

A Categorical Backbone for Composable Optimization

Parameterized Systems, Lawful Interventions, Stochastic Experiments,
and Plugin Interpreters

A doctoral-style architecture study and executable reference implementation

August 2026

Table of Contents

Abstract.....	4
Preface.....	6
Principal thesis claims.....	7
Notation and conventions.....	8
Part I. Reframing the optimization field.....	10
1. From an optimization loop to an optimization doctrine.....	11
2. Empirical starting point: ragopt, ragkit, and applied RAG.....	13
3. Design criteria and non-goals.....	15
4. Two plugin surfaces.....	16
Part II. The categorical and probabilistic backbone.....	18
5. Typed signatures.....	19
6. The free wiring category.....	21
7. Interpreters, effects, and abstract semantics.....	23
8. Parameterized morphisms and the Para construction.....	25
9. Reparameterizations, transformations, and double structure.....	27
10. Optics and lawful intervention.....	28
11. Categorical probability and stochastic trial semantics.....	30
12. Couplings and exact paired experiments.....	31
13. Metrics as orders and resources.....	33
14. Selection, gates, and open decision systems.....	34
15. Campaigns as coalgebras and open dynamical systems.....	36
16. The unified optimization doctrine.....	38
Part III. Software architecture induced by the doctrine.....	40
17. The minimal kernel.....	41
18. Canonical values, identity, and material evidence.....	43
19. Transactional plugin registration.....	45
20. Plans and their interpreters.....	47
21. High-level optimization ports.....	49
22. Candidates and causal dependency closure.....	51
23. Trial custody and comparison.....	53
24. Ordered decisions and candidate selection.....	54
25. Campaign identity, event sourcing, and distributed refinement.....	55
26. Versioning, migration, and security.....	57
27. Package architecture.....	59
Part IV. The executable sandbox.....	61
28. Implementation method and verification status.....	62
29. core: identity and outcomes.....	64
30. plugin: models of signatures.....	65
31. plan: syntax, normalization, and folds.....	66
32. engine: reference execution semantics.....	68
33. para, optic, and prob.....	69
34. metric, experiment, and decision.....	70
35. campaign: executable dynamic semantics.....	71
36. RAG graft: fine-grained plan plus high-level campaign.....	72
37. Quadratic graft: proving domain neutrality.....	75
38. What the sandbox establishes—and what it does not.....	76
Part V. Grafting the backbone onto RAG optimization.....	77

39. From ragopt and ragkit to one optimization field.....	78
40. Indexing optimization as compositional derivation.....	81
41. Query optimization as interpreted evidence flow.....	84
42. Joint index/query spaces and multi-fidelity experiments.....	87
43. From offline campaign to production release.....	90
44. Applied grafts: GEC, RAG-TTC, and Garden.....	92
45. Migration roadmap.....	94
Part VI. Assurance, extensions, and research program.....	96
46. Structural results and proof obligations.....	97
47. Verification strategy.....	101
48. Plugin security and isolation.....	104
49. Distributed refinement.....	107
50. Limits of the doctrine.....	109
51. Research program.....	111
52. Conclusion.....	113
Appendix A. Formal definitions and operational rules.....	114
Appendix B. Sandbox API reference.....	119
Appendix C. Plugin authoring and conformance protocol.....	123
Appendix D. Worked RAG optimization trace.....	127
Appendix E. Campaign event trace and replay.....	131
Appendix F. Empirical mapping to the supplied repositories.....	133
Appendix G. Selected bibliography.....	137
Appendix H. Glossary.....	140

Abstract

Optimization software is usually organized around loops: propose a configuration, run an evaluation, compare measurements, and retain a winner. This description is operationally recognizable but mathematically weak. It says little about how optimized systems compose, how parameters follow composition, how a local intervention changes a global configuration, how stochastic trials should be paired, how effects and resources propagate, how a plugin can extend the field without redefining its laws, or how an interrupted campaign resumes without changing its meaning. In retrieval-augmented generation, these omissions become acute. A candidate can alter corpus admission, chunking, representations, embeddings, index structure, query rewriting, filtering, fusion, reranking, context selection, generation, agent policy, deadlines, caches, or frontend presentation. Each intervention invalidates a different portion of the system and requires a different level of evidence. A flat parameter dictionary and a generic callback loop cannot express this causal structure reliably.

This thesis develops a categorical and probabilistic backbone for a composable optimization field. The construction is deliberately layered. A typed signature describes domain objects and primitive operations. Its free wiring syntax provides sequential composition, monoidal product, permutation, explicit copying, and explicit discarding. Plugins contribute generators and laws, not new composition rules. Every interpretation of a plan—execution, static effect analysis, dependency analysis, resource estimation, identity, provenance, visualization, or deployment—is induced by a structure-preserving fold. Parameterized systems are modeled through the Para construction: a system from A to B with parameter object P is represented by a morphism $P \otimes A \rightarrow B$, and composition tensors parameter objects rather than collecting unrelated strings in a global map. Lawful lenses and more general optics identify local views of a release specification; patches become typed interventions with direct targets, transitive dependency closure, semantic class, and hypothesis.

Stochastic evaluation is modeled with Markov kernels. For parameter θ , a trial is a kernel $K_\theta: X \rightsquigarrow O$ from cases and environment to distributions over outcomes. Paired comparison is therefore a coupling problem, not merely two independent samples. The executable reference uses deterministic, domain-separated seed splitting to provide a common-random-number coupling and exact case/repeat coordinates; a production realization can strengthen this with retained provider responses or explicit coupling kernels. Metrics define oriented preorders rather than a universal scalar field. Hard constraints determine eligibility, Pareto dominance determines partial preference, and lexicographic gates encode policy without pretending that recall, dollars, latency, disclosure, and reliability share a natural additive unit.

An optimization campaign is modeled as an open state-transition system and, equivalently at a high level, a coalgebra whose observations are immutable events. The event reducer is the operational semantics of campaign custody. Candidate registration, exact paired cells, comparisons, decisions, and terminal selection have explicit causal order. Resume is continued transition from a reduced event prefix. The campaign kernel owns identity, pairing, missingness, failure accounting, gate order, and terminal immutability. Domain grafts own the meaning of parameters, cases, execution, measurements, and native artifacts.

The thesis accompanies a self-contained Go implementation, `opfield`, using only the standard library. The implementation contains a canonical envelope and artifact kernel; a transactional plugin registry; a typed plan language; generic structural folds; execution and static-analysis interpreters; Para, lens, finite-probability, metric-order, experiment, decision, and event-sourced campaign packages; and two independent domain grafts. The first is a miniature RAG optimizer with chunking, lexical and hashed-semantic retrieval, dependency-aware build reuse, seven typed interventions, nine evaluation cases, exact pairing, and promotion gates. The second optimizes a noisy quadratic objective without using the plan or RAG layers, demonstrating that the campaign protocol is domain-neutral. The module passes unit tests, `go vet`, and the Go race detector.

The resulting architecture is not a universal optimizer and does not claim that category theory replaces statistics, application semantics, or operational engineering. Its purpose is narrower and stronger: to identify the small algebraic structures that a core must own so that independently developed plugins and applied systems can be grafted onto one optimization field without losing type boundaries, causal identity, reproducibility, or decision semantics.

Preface

This volume develops the mathematical backbone implicit in Chapter 21 of the preceding RAG semantics study. That chapter treated optimization as a typed intervention over the dependency graph of an evolving RAG release. The present work asks what must exist beneath that domain model if indexing optimization, query optimization, answer evaluation, session calibration, backend certification, and production canaries are to share one compositional architecture.

The answer is not “more interfaces” in the ordinary sense. Interfaces are useful only when their laws and composition are controlled. A core made entirely of extension points has no semantics of its own. Conversely, a closed framework that hard-codes every stage cannot accommodate GEC authorization, TTC connected retrieval, Garden widgets, a new ANN backend, or a non-RAG optimization problem without central modification. The design problem is to locate the boundary at which variability becomes a model of stable structure rather than a bypass around it.

The implementation is intentionally a sandbox. It is small enough to inspect end to end and concrete enough to run. It does not include distributed workers, a production artifact service, generalized Bayesian optimization, release activation, or a full statistical library. Those facilities are discussed as interpreters, stores, policies, or domain grafts. The core remains limited to mechanisms whose semantics are sufficiently stable to justify shared ownership.

The mathematical presentation is constructive. Definitions are followed by their software consequences; abstract structures are retained only when they explain a composition rule, a plugin boundary, a law, an identity, a test, or an operational invariant. Formal claims are separated into proven-by-construction properties, executable law checks, proof sketches, and empirical obligations. Natural-language model quality and production performance remain empirical regardless of the elegance of the wiring category.

Principal thesis claims

1. **Optimization requires a domain of composable systems before it requires a search algorithm.** A proposer cannot act meaningfully until system structure, parameterization, intervention, stochastic evaluation, resources, and decisions have explicit semantics.
2. **The correct low-level plugin boundary is a typed signature.** Plugins add objects, primitive generators, codecs, annotations, and laws. The kernel retains sequence, tensor, structural maps, validation, and interpretation.
3. **A free wiring syntax is the simplest stable backbone.** It separates intensional plans from their meanings and gives each interpreter a unique structural extension from plugin generators.
4. **Parameters should compose with systems.** The Para construction models a component as $P \otimes A \rightarrow B$ and makes composite parameter spaces arise from wiring rather than a global namespace.
5. **Candidate patches are optics plus causal declarations.** Lawful focus/update behavior is necessary but not sufficient; every intervention must also name semantic class, direct targets, dependency closure, and evaluation fidelity.
6. **Stochastic comparison is about couplings.** Exact paired coordinates and shared randomness are structural requirements, not reporting conveniences.
7. **Optimization is ordered rather than scalar by default.** Security, integrity, coverage, quality, latency, cost, and user outcome generally form constraints and partial orders; scalar preference is a final policy layer.
8. **A campaign is a dynamic system with durable semantics.** Event-sourced custody makes interruption and resume part of the same transition system rather than an implementation-specific recovery path.
9. **Two plugin surfaces are required.** Fine-grained operation plugins support multiple interpreters; coarse domain grafts wrap existing applications without forcing their internals into the kernel.
10. **The core should be small because its laws are strong, not because it is vague.** Composition, identity, effect declaration, pairing, missingness, gate order, and event validity belong in the kernel. Product meaning and native measurement do not.

Notation and conventions

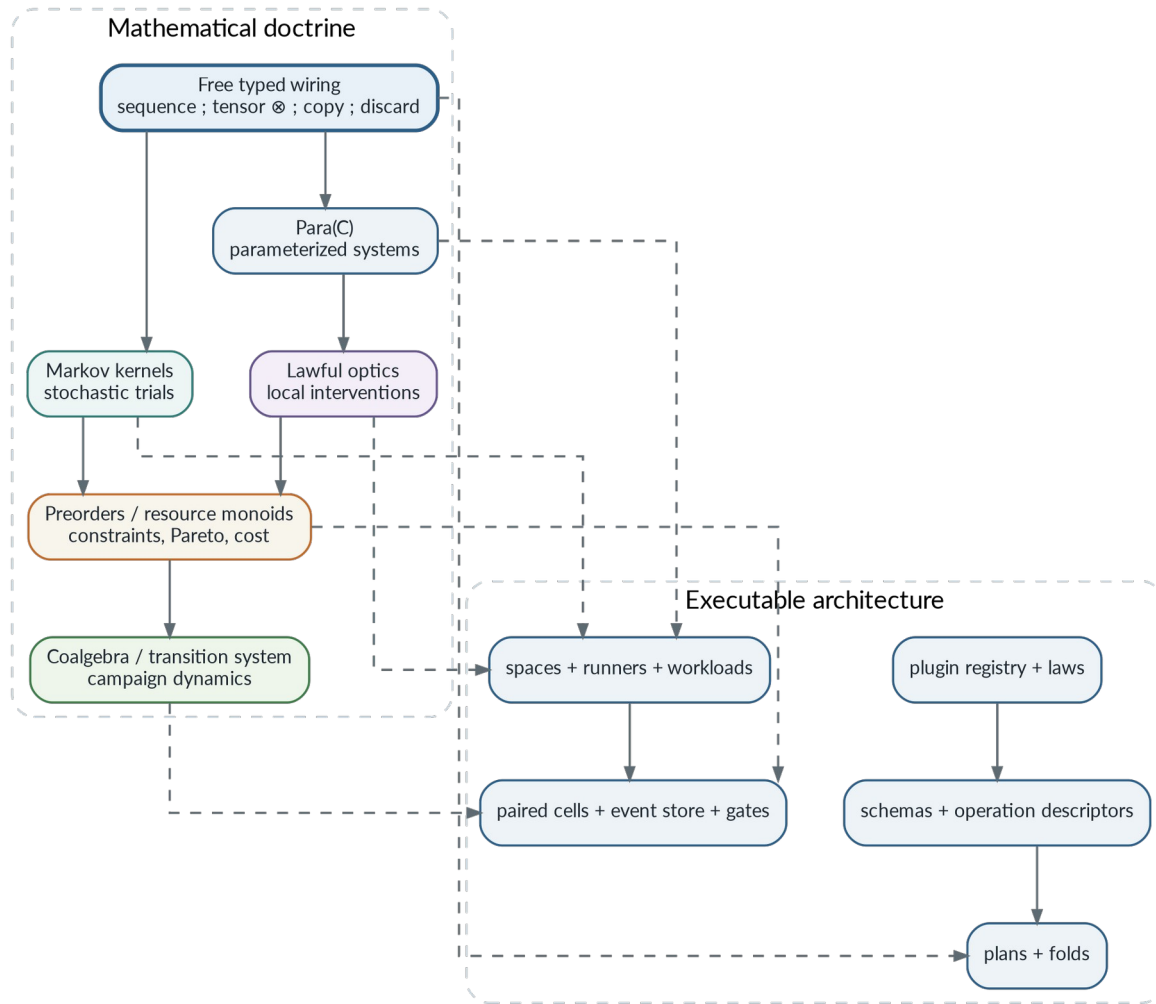
A symmetric monoidal category is written $(C, \otimes, I)$. Objects such as A, B, X, Y denote typed interfaces or data schemas. A morphism $f: A \rightarrow B$ denotes a computation or system component. Sequential composition is written $g \circ f$ or $f; g$. Parallel composition is $f \otimes g$.

A typed signature is $\Sigma = (Ob, Gen, dom, cod, ann)$. Its free wiring category is written $W(\Sigma)$. A parameterized morphism is represented by a pair (P, f) with $f: P \otimes A \rightarrow B$ and written $A \xrightarrow[P]{f} B$. A release or complete configuration is $\theta \in \Theta$. A local parameter view is $\ell: \Theta \rightsquigarrow P$. A candidate intervention is $i: \theta \rightsquigarrow \theta'$.

A Markov kernel from X to Y is written $K: X \rightsquigarrow Y$. $D(Y)$ denotes a distribution over Y . A paired coupling of baseline and candidate kernels is $\Gamma_x \in D(O_b \times O_c)$ with the correct marginals. A metric vector is $m: O \rightarrow R^d$, with each coordinate carrying a direction. The oriented candidate difference is $\Delta_j = s_j(m_j^c - m_j^b)$, where $s_j = 1$ for maximization and $s_j = -1$ for minimization.

A campaign state is $s \in S$, an event is $e \in E$, and the pure reducer is $\rho: S \times E \rightarrow S$. A partial arrow indicates that invalid events are rejected. An event prefix is $e_{1:n}$ and its reduced state is $\rho^{\dot{i}}(s_0, e_{1:n})$.

The Go implementation uses schema IDs and canonical envelopes at runtime because Go cannot directly encode a heterogeneous typed syntax tree with all type equalities checked statically. Typed generic adapters establish the plugin boundary; plan validation establishes the heterogeneous wiring boundary.



The proposed optimization doctrine and its executable realization.

Part I. Reframing the optimization field

1. From an optimization loop to an optimization doctrine

1.1 The usual loop is extensionally under-specified

The standard optimization loop can be expressed in a few lines:

```
while budget remains:
    candidate = propose(history)
    outcome = evaluate(candidate)
    history = update(history, candidate, outcome)
return select(history)
```

This form is useful for explaining search strategy. It is not a sufficient architecture. `candidate` may be a scalar, a prompt file, a whole index, a deployment release, or a policy mutation. `evaluate` may be a pure function, a stochastic provider call, a multi-hour build, or a production canary. `outcome` may be one number, a partial metric vector, a failure, an interaction trace, or an artifact graph. `history` may be in memory, a filesystem, a database, or a durable event stream. `select` may be argmax, a feasibility test, a Pareto policy, or human authorization.

When all of these distinctions are left to callbacks, the loop has almost no denotational content. It cannot tell whether two candidates are comparable, whether an index can be reused, whether a failure remains in the denominator, whether a plugin duplicated an effect, or whether a resumed run is the same experiment.

A doctrine is a small collection of structures that make those questions meaningful across domains. It does not prescribe the search algorithm. It specifies what a system, parameter, intervention, trial, comparison, decision, and campaign are, and how each composes.

1.2 Why RAG exposes the weakness

RAG optimization spans a dependency graph rather than one parameter vector. A chunk-size change modifies knowledge projection, chunk identities, generated representations, embeddings, indexes, retrieval labels, and downstream answers. A fusion-weight change reuses the index but changes ranking and every downstream context. A reranker change alters disclosure, latency, failure behavior, and ranking. A deadline change may be operational under one workload and answer-changing under another. An agent-tool description changes the distribution over search trajectories. A widget projection can change user outcome with no retrieval change.

These interventions have different codomains of evidence. Static laws can reject an invalid chunker or authorization order. Exact-oracle tests can certify an ANN approximation. Retrieval labels can compare ranking. Repeated answer cells can compare grounding. Multi-turn sessions are required for agent policy. Shadow and canary trials are required for production latency and fallback. A generic optimizer must not flatten these levels.

1.3 What the doctrine must own

The doctrine must own precisely the decisions that make evidence composable:

- typed system boundaries;
- sequential and parallel composition;
- explicit duplication and discarding;
- semantic identity;
- parameter composition and reparameterization;

- lawful local update;
- declared effects and dependencies;
- stochastic trial coordinates and couplings;
- metric direction and missingness;
- ordered gate evaluation;
- durable state transition and terminality.

It must not own the meaning of a GEC source role, a TTC product fact, a Garden widget, or an ANN recall threshold. Those remain domain semantics.

1.4 Search algorithms become plugins of the doctrine

Grid search, random search, Bayesian optimization, evolutionary search, gradient descent, LLM proposal, and human curation can all implement a proposer interface once the candidate space and evidence protocol exist. Gradient-based optimization may exploit extra differential structure; Bayesian methods may exploit probabilistic surrogate structure. Neither should be required by the base field.

This inversion is central: the optimizer is no longer the core into which a system is embedded. The compositional system and experiment doctrine is the core; a search algorithm is one controller over it.

2. Empirical starting point: `ragopt`, `ragkit`, and applied RAG

2.1 Existing strengths

The reviewed `ragopt` package already establishes several strong semantic choices. A candidate is an immutable baseline snapshot plus exactly one mutable asset. Evaluation pairs incumbent and challenger at identical case and repeat coordinates. Outcomes distinguish contract validity, abstention, metrics, usage, errors, and native artifacts. The run store copies inputs, appends cells, resumes incomplete runs, and prevents a report from silently ignoring missing cells. Gate policy is ordered rather than one weighted objective.

`ragkit` provides deterministic RAG domain types and functions: documents, chunks, representations, embedders, lexical and vector searchers, retrieval and fusion, reranking contracts, context construction, grounded answer validation, caches, execution controls, and immutable index bundles. RAG-TTC, GEC, and Garden graft product-specific behavior onto those mechanisms.

The supplied snapshot contains 173 Go files and 273 test functions in `ragkit`, 45 Go files and 42 tests in `ragopt`, 515 Go files and 906 tests in RAG-TTC, 200 Go files and 252 tests in GEC, and 70 Go files and 108 tests in the Garden backend scope. The numbers establish that the design problem is not hypothetical. Several mature subsystems already need a shared semantic boundary.

2.2 The remaining gap

The current optimization kernel begins after a product has materialized a candidate and implemented an `Arm`. It does not know the candidate's internal parameter structure, the system plan it modifies, the dependency closure it invalidates, the effects it changes, or the evaluation fidelity required by its semantic class. This is a sound boundary for a generic run harness, but it leaves each application to reinvent the optimization *field* around it.

The current RAG library begins with RAG domain operations. It does not provide a domain-neutral compositional language for systems and parameterization, nor should it. Consequently, a direct attempt to put all optimization composition into `ragkit` would over-specialize the foundation.

The missing layer is between the semantic kernel and the domain packages: a small, typed, compositional doctrine that can describe the shape of an optimized system and the legal way external domains extend it.

2.3 Requirements derived from the codebase

The applied systems imply concrete requirements:

1. A build plan and a query plan must share types and identities but admit different interpreters and resource policies.
2. Query-only candidates must reuse build artifacts without relying on handwritten lists of irrelevant fields.
3. A product runner must retain a native artifact richer than generic metrics.
4. A search tool or full conversation must be usable as one trial without rewriting the application into primitive nodes.
5. A new backend or evaluation stage should be pluggable at a fine-grained level when static analysis is valuable.
6. Authorization and remote disclosure must be visible as effects that a plan policy can reject.

7. Failures and missing metrics must be represented rather than coerced into zero or dropped.
8. A campaign must resume from exact semantic coordinates.
9. The core cannot depend on any one RAG provider, UI framework, judge, or artifact backend.

These requirements motivate two plugin surfaces rather than one.

3. Design criteria and non-goals

3.1 Criteria

Compositionality. The meaning of a composite plan must be determined by meanings of its parts and the composition constructors.

Typed openness. Plugins can add schemas and primitive operations without adding new plan node kinds or changing kernel composition.

Multiple interpretations. One plan can be executed, analyzed, identified, visualized, or compiled to a remote graph.

Parameter locality. Component parameters compose with component wiring and can be addressed through lawful local views.

Stochastic honesty. Randomness, sampling, coupling, failure, and retained material are explicit.

Causal invalidation. An intervention determines downstream artifacts and evaluations through a dependency relation.

Custody. Every candidate, case, repeat, arm, metric, artifact, comparison, and decision has stable identity and durable state.

Policy separation. Eligibility and preference are explicit policy values, not hidden in an optimizer's scoring callback.

Small trusted core. The kernel should be inspectable and stable; product-specific meaning belongs in grafts.

3.2 Non-goals

The doctrine is not intended to prove semantic equivalence of arbitrary application code automatically. It cannot infer whether a prompt change is safe, whether a model answer is true, or whether a corpus label is correct. It provides places to state and test these claims.

It is not a replacement for workflow orchestration. A distributed scheduler can interpret plans or execute trial requests, but queue placement and cluster management need not be part of the categorical kernel.

It is not a requirement that every application operation become a primitive generator. Fine-grained representation is useful only when structural analysis, reuse, or alternate interpretation justifies it.

It is not a universal scalar optimizer. The default decision structure is constraint and partial order.

4. Two plugin surfaces

4.1 Low-level operation plugins

A low-level plugin extends a typed signature. It declares schemas, primitive operation signatures, effects, dependency labels, resource hints, codecs, implementations, and laws. It is appropriate when a component should be visible to several interpreters.

Examples include:

- normalize one source revision;
- chunk a document;
- generate one representation;
- embed a batch;
- build or query one index;
- fuse rankings;
- rerank an authorized pool;
- assemble context;
- validate an answer contract.

The plugin does not define sequence or parallel composition. It cannot define a special identity node. It cannot alter how plan IDs are computed. Those remain kernel laws.

4.2 High-level domain grafts

A high-level graft implements the optimization protocol around an existing system. It supplies a [Space](#), [Proposer](#), [Workload](#), [Runner](#), [Policy](#), and [Store](#). The runner may call an application service, run a CLI, submit a job, or execute a low-level plan.

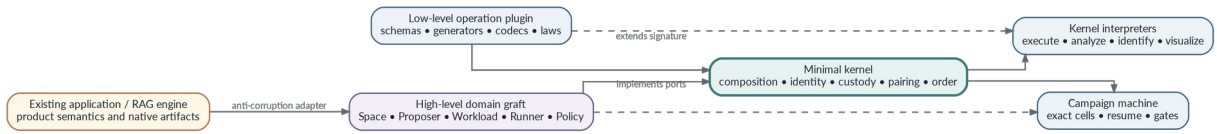
Examples include:

- one complete GEC retrieval-and-answer case;
- one Garden multi-turn calibration conversation;
- one ANN build/query benchmark;
- one production shadow request;
- one noisy mathematical objective.

This surface prevents the architecture from requiring an invasive rewrite before an application can gain exact pairing and durable campaign semantics.

4.3 Why one interface cannot serve both

If every domain is forced into primitive operations, the core becomes a workflow DSL and application semantics leak into shared packages. If every domain is only an opaque runner, the core cannot analyze plans, dependencies, effects, or artifact reuse. The two surfaces occupy different abstraction levels and can coexist.



Low-level plugins extend typed syntax; high-level grafts connect complete domains to campaign custody.

Part II. The categorical and probabilistic backbone

5. Typed signatures

5.1 Definition

A typed operation signature is

$$\Sigma = (O, G, d, c, a),$$

where O is a set of object or schema symbols; G is a set of primitive generators; $d(g)$ and $c(g)$ are finite ordered lists of objects giving the domain and codomain ports of generator g ; and $a(g)$ is a set of annotations.

A port $[A_1, \dots, A_n]$ denotes the tensor $A_1 \otimes \dots \otimes A_n$. The empty port denotes the monoidal unit I .

Annotations are not part of ordinary category theory but are necessary for software interpretation. In this doctrine they include:

- semantic operation and plugin version;
- effect set;
- determinism and cacheability claims;
- dependency labels;
- resource hints;
- data-class or disclosure metadata;
- human description.

5.2 Schemas are semantic objects

A schema is not merely a serialization shape. It identifies the interpretation of a value on a wire. Two JSON records with the same fields but different authority, temporal scope, or units require different schemas. A source chunk and a generated summary may both contain text, but one is evidence and one is a search representation. Treating them as the same object would make an invalid plan type-correct.

Versioned schema IDs provide nominal type safety across plugin boundaries. A codec interprets a schema symbol as a concrete representation and establishes an encode/decode relation. In the sandbox, canonical JSON envelopes combine schema ID, payload, and domain-separated digest.

5.3 Signature union and plugin registration

A registry combines plugin signatures only when object and generator names do not conflict and every operation port resolves. This is a disjoint-union discipline with explicit sharing through previously registered schema IDs. Transactional registration ensures that failed laws or collisions do not leave a partial signature.

The registry is best treated as immutable after release construction. Hot replacement of a generator implementation changes the model of the signature and therefore creates a new registry or release identity.

5.4 Why annotations remain declarative

Effect and dependency declarations cannot be trusted merely because they are fields. Their purpose is to support inspection, policy, and testing. A production plugin may be isolated in a capability sandbox so that

declarations can also be enforced. The mathematical architecture does not confuse a declared effect system with OS security; it supplies the vocabulary and plan position required for enforcement.

6. The free wiring category

6.1 Syntax

From Σ we construct a free typed wiring language $W(\Sigma)$. Its morphisms are generated by:

- each primitive $g: d(g) \rightarrow c(g)$;
- identity $id_A: A \rightarrow A$;
- sequential composition;
- tensor product;
- symmetry or permutation maps;
- explicit copying $\Delta_A: A \rightarrow A \otimes A$;
- explicit discarding $!_A: A \rightarrow I$.

The sandbox represents this syntax as `plan.Plan` nodes: `primitive`, `identity`, `sequence`, `tensor`, `permute`, `copy`, and `drop`.

6.2 Why free syntax is the backbone

Free syntax separates a plan from every particular meaning. The same term can be interpreted as executable code, a dependency graph, a cost expression, a provenance query, a disclosure proof obligation, or a diagram. This is the source of composability: all meanings follow the same wiring.

The free construction also gives plugins a disciplined extension point. A plugin adds generators to Σ ; it does not add an eighth composition constructor. This prevents a plugin from creating a node that static analysis cannot see or that identity hashing treats inconsistently.

6.3 Explicit copy and discard

In an ordinary cartesian category, values can be copied and discarded naturally. Effectful computations generally cannot. Duplicating the output value of a completed deterministic computation is different from executing the computation twice. Discarding a value is different from removing the computation that produced it, especially when the computation writes an artifact or discloses data remotely.

The plan syntax therefore makes structural copy and discard explicit. It resembles a free gs-monoidal or CD-style wiring category rather than assuming all morphisms are cartesian. This aligns with categorical work that treats free gs-monoidal categories as combinatorial term graphs and notes their relevance to computer implementation.

6.4 Equations and normalization

The kernel enforces associative sequence and tensor and removes identities after checking boundary compatibility. Thus

$$(f;g);h = f;(g;h)$$

and

$$(f \otimes g) \otimes h = f \otimes (g \otimes h)$$

receive the same normalized plan representation and digest. Permutation is explicit, so the order of a port remains semantic.

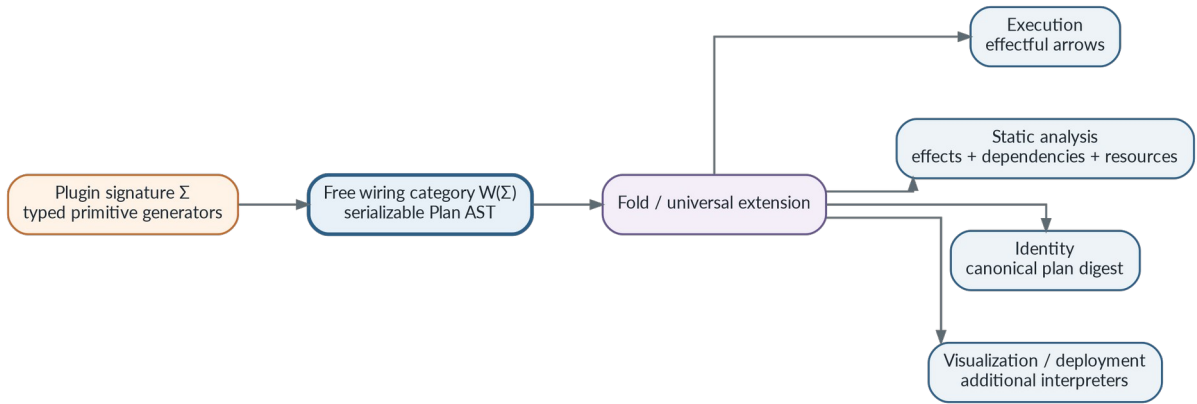
The sandbox does not quotient by every possible gs-monoidal equation. In particular, it does not automatically rewrite copy through arbitrary generators. This conservative syntax preserves intensional structure needed for effects and provenance.

6.5 Universal interpretation

Let D be a semantic domain with meanings for the structural constructors. An assignment F_0 of every object and generator in Σ to D extends uniquely by structural recursion to an interpretation

$$\llbracket - \rrbracket_F : W(\Sigma) \rightarrow D.$$

In the implementation, `plan.Algebra[R]` supplies meanings for the seven constructors and `plan.Fold` performs the extension. This is the practical universal property.



A plugin signature generates one plan syntax; folds induce multiple whole-plan interpretations.

7. Interpreters, effects, and abstract semantics

7.1 Execution as an effectful interpretation

A pure generator would denote a function. Real operations can fail, observe time, read and write artifacts, call networks, use state, sample randomness, or disclose content. A useful denotation has the shape

$$\llbracket f \rrbracket : X \rightarrow T(Y),$$

where T captures effects. One conceptual carrier is

$$T(Y) = \text{Context} \rightarrow \text{Outcome}(Y \times \text{Trace} \times \text{Artifacts} \times \text{Duration}).$$

This can be organized as a Kleisli category when the chosen effect combination forms a monad. Algebraic-effects theory offers another view: primitive effects generate a free theory and handlers interpret them. The sandbox does not encode a monad or handler calculus directly; it uses an explicit `core.Execution` value and an execution algebra. The categorical model explains why sequence and alternate handlers can share syntax.

7.2 Failure has semantic classes

At least four statuses are necessary:

- success;
- domain failure;
- infrastructure failure;
- cancellation.

A query returning a valid abstention can be a success. An answer contract violation can be a domain failure. A corrupt artifact is infrastructure failure. Context cancellation is not evidence that the candidate is low quality. These distinctions determine retry, denominator, and gate behavior.

A plugin implementation error is returned as an attributable execution result. An outer campaign error is reserved for inability to maintain campaign semantics, such as an event-store write failure.

7.3 Static abstract interpretation

The same plan can be folded into a static summary. The sandbox computes operation set, effect set, dependency set, determinism, cacheability, and a resource hint. Sequential and tensor composition use different resource operations:

$$\begin{array}{ll} \text{work}(f;g) & \textcolor{red}{\wr} \text{work}(f) + \text{work}(g), \\ \text{critical}(f;g) & \textcolor{red}{\wr} \text{critical}(f) + \text{critical}(g), \\ \text{memory}(f;g) & \textcolor{red}{\wr} \max(\text{memory}(f), \text{memory}(g)), \\ \text{work}(f \otimes g) & \textcolor{red}{\wr} \text{work}(f) + \text{work}(g), \\ \text{critical}(f \otimes g) & \textcolor{red}{\wr} \max(\text{critical}(f), \text{critical}(g)), \\ \text{memory}(f \otimes g) & \textcolor{red}{\wr} \text{memory}(f) + \text{memory}(g). \end{array}$$

These are estimates, not performance guarantees. Their value is compositional preflight and comparison.

7.4 Effect policy as an interpreter or predicate

A policy can reject a plan whose static interpretation contains forbidden effects or an invalid ordering. A data-class analysis can require every `remote.disclosure` operation to be dominated by an authorization certificate stage. A deterministic-replay policy can reject `clock` and undeclared `random` effects. A local-test policy can replace network generators with artifact replay handlers.

This is a direct benefit of retaining plans as data. A callback-only architecture can log effects after execution; it cannot reliably reject a composition before execution.

7.5 Interpreter coherence

Different interpreters need not produce identical traces, but they must share protected structure. A concurrent execution interpreter may schedule tensor branches differently from the serial reference interpreter. Its observational equivalence may ignore branch interleaving while preserving outputs, failure class, per-operation observations, disclosure set, artifacts, and resource bounds.

Every production interpreter should state its refinement relation to the reference semantics. This is more precise than saying two engines “run the same DAG.”

8. Parameterized morphisms and the Para construction

8.1 Parameter objects

A configurable component is not merely $f : A \rightarrow B$. It is a family indexed by a parameter object P :

$$f : P \otimes A \rightarrow B.$$

Equivalently, each $p \in P$ selects a behavior $f_p : A \rightarrow B$. The object P can be a scalar, a structured record, a prompt artifact, an index backend specification, or a complete release submanifest.

This formulation gives parameters a type and a position in the system.

8.2 Composition

Suppose

$$f : P \otimes A \rightarrow B$$

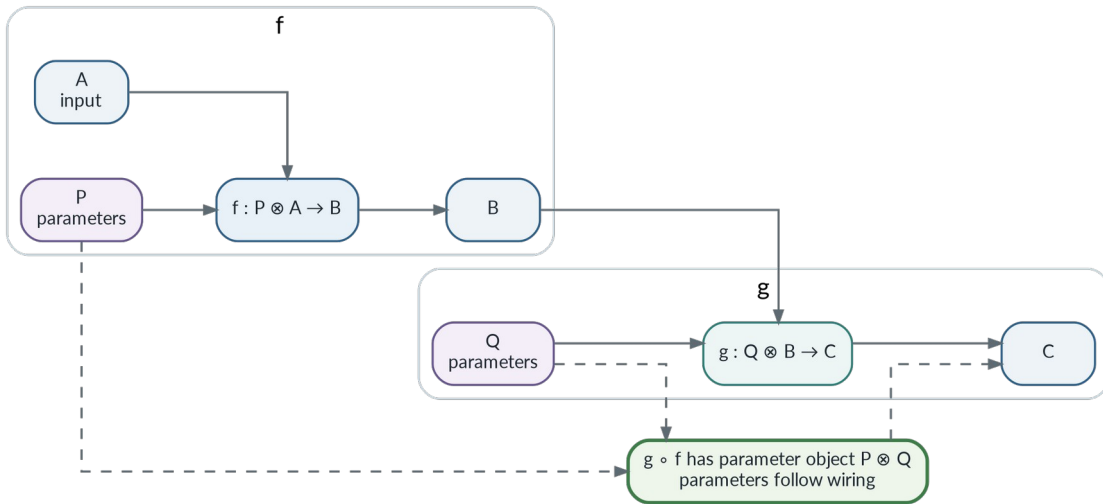
and

$$g : Q \otimes B \rightarrow C.$$

Their parameterized composite has parameter object $P \otimes Q$:

$$(P \otimes Q) \otimes A \cong Q \otimes (P \otimes A) \xrightarrow{id_Q \otimes f} Q \otimes B \xrightarrow{g} C.$$

The crucial consequence is that parameters compose according to system wiring. There is no need for a global untyped map whose keys happen to be understood by distant code.



Composition in Para tensors component parameter objects.

8.3 Reparameterization

An optimizer may use coordinates R that differ from implementation parameters P . A map $r : R \rightarrow P$ induces

$$r^! f = f \circ (r \otimes id_A) : R \otimes A \rightarrow B.$$

Examples include:

- log-space to a positive rate;
- logits to simplex weights;
- a single policy knob tied to several lower-level values;
- a model alias resolved to an immutable provider version;
- a candidate artifact bundle materialized into legacy files.

Reparameterization should be explicit and identified. Otherwise an optimizer’s coordinates can change meaning without a release identity change.

8.4 Para as a category of open models

Under standard conditions, parameterized maps form a category `Para(C)` whose morphisms are parameterized maps modulo suitable reparameterization. Work on compositional learning has used this construction to explain how models and learning algorithms compose, including backpropagation as a monoidal functor and gradient-based learning through parametric lenses.

The present doctrine generalizes the architectural lesson beyond gradients. The parameter object can be discrete and mixed, and the proposer need not be differentiable. Composition still benefits from Para.

8.5 Implementation correspondence

The sandbox’s `para.Parametric[P,A,B]` contains a `Run(P,A)` function. `Compose` pairs parameter objects, `Tensor` pairs independent systems, and `Reparameterize` maps coordinates. Go’s ordinary product structs stand in for monoidal products.

The implementation is intentionally small: it does not quotient reparameterizations or encode associator isomorphisms as first-class values. The thesis-level architecture can use a bicategory or double category when those witnesses must be retained.

9. Reparameterizations, transformations, and double structure

9.1 Why ordinary categories are not the whole story

Optimization involves two kinds of movement:

1. composing systems along their data interfaces;
2. changing how one parameter space represents or controls another.

It is useful to distinguish horizontal system wiring from vertical parameter transformations. A square can express compatibility:

$$\begin{array}{ccc} P \otimes A & \xrightarrow{f} & B \xrightarrow{g} B' \\ \downarrow r \otimes u & \wr & \wr \end{array}$$

Such squares can represent schema migration, materialization, adapter correctness, or a candidate implementation refining an abstract specification.

9.2 Double categories and open systems

Double-category treatments of open dynamical systems distinguish interface wiring from maps between dynamics. This is relevant because an optimization campaign is itself an open dynamical system: it receives proposals and trial completions, changes internal state, and emits decisions and artifacts. Product runners can be grafted through interfaces without exposing all internal state.

The reference implementation does not implement a generic double-category library. It retains the distinction in ordinary interfaces:

- plans compose system operations;
- optics and `Space.Apply` transform specifications;
- runners interpret a spec on a case;
- campaign events transform durable state.

A future formalization can promote compatibility evidence between these layers to explicit 2-cells.

9.3 Refinement squares

A useful production notion is a refinement square between an abstract operation and a concrete plugin implementation. It can state that concrete execution followed by output projection equals abstract execution after input embedding, perhaps up to an observation relation. Such squares would support backend substitution and differential certification.

For example, an ANN backend refines an exact vector-search operation not by equality but by a tolerance relation over ranked outputs and resource improvement. The square's relation becomes part of its certification artifact.

10. Optics and lawful intervention

10.1 The local-update problem

A release specification Θ is large. An optimizer usually changes one focus P : chunk size, vector weight, reranker model, timeout, or prompt. A raw function `set(path, value)` is too weak. It does not state how to read the current focus, whether writes are stable, or whether two updates interfere.

A lens consists of

$$get: \Theta \rightarrow P$$

and

$$put: \Theta \times P \rightarrow \Theta,$$

subject to laws.

10.2 Lens laws

For admissible p, q :

$$put(\theta, get(\theta)) = \theta$$

(get-put),

$$get(put(\theta, p)) = p$$

(put-get), and

$$put(put(\theta, p), q) = put(\theta, q)$$

(put-put).

These laws give a local patch stable meaning. Put-put is particularly important for event histories and adaptive proposers: the current focus depends on the last value, not accidental update history.

10.3 Partial validity

Real configurations impose constraints. Overlap must be less than chunk length; weights must be finite and nonnegative; a model must support the required dimension. `put` may therefore be partial. The laws apply over admissible values. A `Space` must not advertise a patch that its optic rejects.

The sandbox checks lens laws at plugin registration over representative states and values. This is executable evidence, not a universal proof. Property testing can strengthen coverage.

10.4 General optics

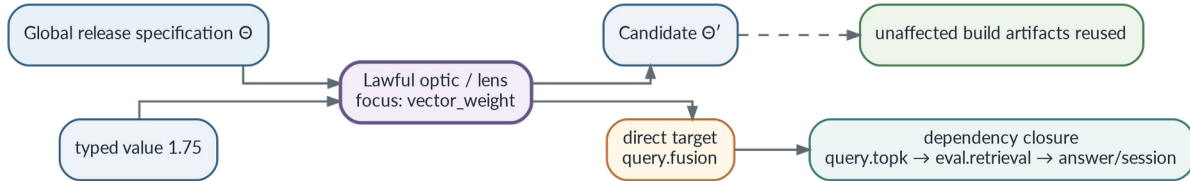
Lenses cover product-like configuration. Prisms can represent optional routes or backend variants; traversals can update homogeneous collections; affine optics can focus on at most one target. Riley's categorical treatment unifies these accessors and provides a general account of lawfulness. The architecture should expose an optic identity and law certificate rather than freeze the core to string paths.

10.5 Patch plus causal declaration

A lawful optic answers “how is the configuration changed?” It does not answer “what does the change invalidate?” A complete patch therefore includes:

$$i = (\ell, p', class, targets, closure, hypothesis).$$

Semantic classes include operational, approximation, relevance, knowledge, policy, interaction, and presentation. Direct targets identify changed dependency nodes; closure is the transitive downstream invalidation set.



A lawful local patch induces a causal dependency closure and reuse boundary.

10.6 Composition of interventions

The initial `ragopt` discipline of exactly one mutation is valuable because it improves attribution. The mathematical backbone can nevertheless compose compatible interventions. Two lenses on independent focuses can tensor; nested lenses can compose. A multi-change candidate then has a structured intervention tree rather than an unordered patch set.

Campaign policy can still require atomic interventions during exploration and allow composed candidates only after individual effects are understood.

11. Categorical probability and stochastic trial semantics

11.1 A trial is a kernel

A deterministic trial is a function. A model-mediated or production trial is more accurately a Markov kernel

$$K_\theta : X \rightsquigarrow O,$$

assigning each case x a distribution $K_\theta(- \mid x)$ over outcomes. Randomness may arise from model decoding, provider load, approximate search, timing, data arrival, or agent choices.

Markov categories provide a compositional setting in which stochastic maps, copying, discarding, conditional independence, and statistical notions can be treated abstractly. The doctrine uses this structure to define trial composition without choosing one probability representation.

11.2 Composition

For kernels $K : X \rightsquigarrow Y$ and $L : Y \rightsquigarrow Z$:

$$(L \circ K)(z \mid x) = \int_Y L(z \mid y) K(y \mid x).$$

For finite distributions the integral is a sum. The sandbox implements `prob.Bind` and `prob.Compose` directly.

Independent parallel composition uses product kernels. Correlated stages require a joint kernel rather than tensor.

11.3 Deterministic maps inside stochastic semantics

A deterministic function $f : X \rightarrow Y$ embeds as a Dirac kernel $\delta_f : X \rightsquigarrow Y$. This lets deterministic preprocessing, stochastic generation, and deterministic validation compose in one semantic target.

Deterministic structure is also necessary for lawful copying. A sampled outcome can be copied after sampling; copying a stochastic generator and running two samples is a different plan. Explicit copy in the wiring syntax preserves this distinction.

11.4 Partiality and failure

A probability distribution over only successful values loses failure probability. One option is an outcome object

$$O = Y + \text{DomainFailure} + \text{InfrastructureFailure} + \text{Cancelled}.$$

The kernel then distributes mass across terminal classes. Another option uses partial Markov categories or subprobability. The sandbox uses explicit terminal statuses in samples. This is sufficient for exact denominator custody and avoids treating a failed trial as absent probability mass.

11.5 Retained material versus abstract distribution

A production system rarely knows the full kernel. It obtains samples. Reproducibility is material: inputs, seed, provider identity, response, trace, and artifacts are retained. The categorical kernel is the denotational model; the campaign ledger is sampled evidence.

12. Couplings and exact paired experiments

12.1 Independent samples are not the default comparison

To compare baseline K_b and candidate K_c , one could sample independently. This adds avoidable variance and weakens causal attribution. A paired experiment chooses a coupling

$$\Gamma_x \in D(O_b \times O_c)$$

whose marginals are $K_b(- \mid x)$ and $K_c(- \mid x)$.

The paired difference is measured on joint samples $(o_b, o_c) \sim \Gamma_x$.

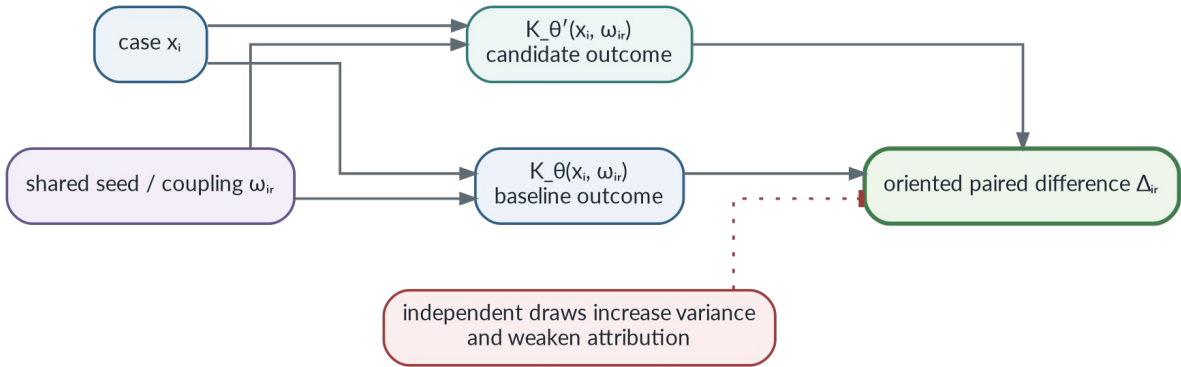
12.2 Common random numbers

A practical coupling uses one seed $\omega_{i,r}$ for case i and repeat r :

$$o_b = F(\theta_b, x_i, \omega_{i,r}), o_c = F(\theta_c, x_i, \omega_{i,r}).$$

This is valid only where the runner interprets the seed consistently. Remote providers may not expose deterministic seeding. Stronger coupling can replay identical retrieval candidates, provider responses, or traffic conditions where doing so does not invalidate the intervention.

The campaign identity domain-separates seeds by campaign, case, and repeat. Baseline and candidate use the same seed coordinate.



Paired comparison is an explicit coupling of baseline and candidate kernels.

12.3 Exact coordinates

A trial coordinate includes case ID, repeat, arm, and candidate identity. Comparison projects baseline and candidate onto common case/repeat keys and requires exactly one terminal result from each arm. Missing and duplicate cells are errors.

This is more than bookkeeping. The matrix of exact coordinates is the empirical object being compared. Dropping difficult failures changes the estimand.

12.4 Coupling validity under interventions

Not every shared artifact is a valid coupling. Replaying baseline retrieval candidates when testing a new chunker would erase the candidate's intended effect. Replaying provider generation while testing only fusion weights may be useful for retrieval diagnosis but cannot measure answer-distribution change. The intervention's dependency closure determines what can be shared.

This gives a categorical interpretation to evaluation reuse: reuse is permitted only outside the causal downstream cone of the intervention.

13. Metrics as orders and resources

13.1 Directional metric spaces

A metric definition is (j, s_j, u_j) : identity, direction, and unit. Direction orients all differences so positive means better:

$$\Delta_j = s_j(m_j^c - m_j^b).$$

This operation is valid without making units commensurate.

13.2 Product preorder and Pareto dominance

Given complete metric vectors, candidate a weakly dominates b when it is no worse in every oriented coordinate. It strictly dominates when at least one coordinate is better. This defines a partial order after quotienting observational equality.

The Pareto front is the set of undominated eligible candidates. It preserves trade-offs rather than hiding them in arbitrary weights.

13.3 Constraints first

Security, contract validity, missingness, and hard capacity are not preferences. They define the feasible subset:

$$F = \{\theta \in \Theta \mid c_k(\theta) \leq 0 \forall k\}.$$

Preference operates only on F . A candidate cannot compensate for unauthorized disclosure with higher recall.

13.4 Resources as ordered commutative monoids

Resources combine and admit convertibility or feasibility orders. Work, dollars, token budget, storage, and provider calls can often be modeled as commutative monoids under addition with an order. Parallel critical path and peak memory require richer algebra.

The architecture should permit resource interpretations that are monotone under composition. Fritz’s resource-theory account of ordered commutative monoids supplies a general language for combination and convertibility. The software consequence is to keep resource vectors structured and allow policy to apply monotones, rather than forcing one cost scalar into every operation descriptor.

13.5 Lawvere-style quantitative enrichment

Some properties can be treated as enriched distances or costs. Latency bounds compose additively in sequence and by maximum in ideal parallel execution. Approximation error may compose under domain-specific bounds. This suggests enrichment of the wiring category in a quantale or ordered algebra.

The sandbox’s `CostHint` is a first approximation, not a general enrichment. The thesis leaves the richer carrier pluggable through interpreters.

14. Selection, gates, and open decision systems

14.1 Decision is not objective evaluation

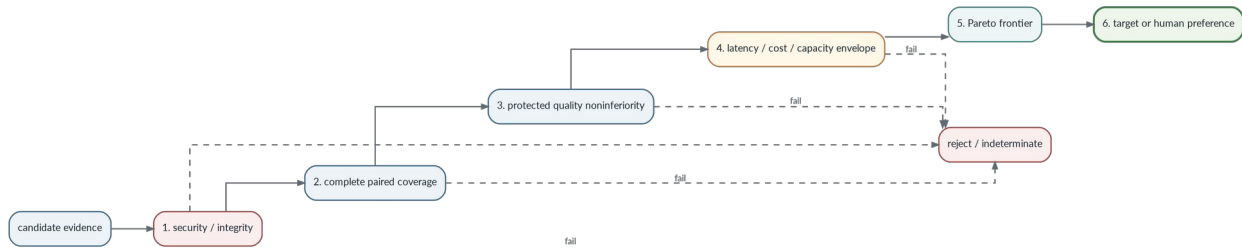
An evaluator maps outcomes to measurements. A decision policy maps a comparison report to `pass`, `fail`, or `indeterminate`, plus checks and optional preference score. Keeping these separate prevents the runner from deciding its own promotion.

14.2 Lexicographic gate program

A gate sequence is evaluated in order:

1. security and integrity;
2. complete paired coverage;
3. protected-stratum noninferiority;
4. target improvement;
5. resource envelope;
6. Pareto or product preference.

The first failed or indeterminate gate terminates evaluation. This order is part of policy identity.



Eligibility is constraint-first; Pareto and preference apply only to surviving candidates.

14.3 Three-valued logic

`Indeterminate` is distinct from failure. Missing metrics, insufficient samples, or an unavailable required stratum mean the policy cannot decide. Treating indeterminate as pass is unsafe; treating it as an intrinsic candidate failure may also be misleading. Operationally it blocks promotion and requests more or corrected evidence.

14.4 Selection functions and open games

A selection function maps a context or payoff continuation to preferred choices. Compositional game theory treats open games as systems whose local behavior depends on an environment and whose equilibria compose. Optimization components have a related open character: a local candidate is valuable only relative to downstream metrics, constraints, and environment.

The present architecture does not model every optimizer as an open game. It adopts the narrower lesson that decision behavior should be an explicit composable object with inputs, outputs, and continuation context, not hidden inside a scalar callback.

14.5 Human decision as a policy interpreter

Human review can be represented as a terminal gate that consumes the complete promotion report and emits a signed decision event. This preserves the distinction between evidence generation and organizational authority. Human judgment is not made “automatic” by placing it in the same event model; it becomes auditable.

15. Campaigns as coalgebras and open dynamical systems

15.1 State and transition

A campaign has state

$$s = (id, candidates, trials, reports, decisions, terminal).$$

Its next action depends on missing semantic work: register a candidate, execute a missing coordinate, compare complete pairs, decide a report, or terminate.

This can be viewed as a coalgebra

$$\gamma: S \rightarrow F(S)$$

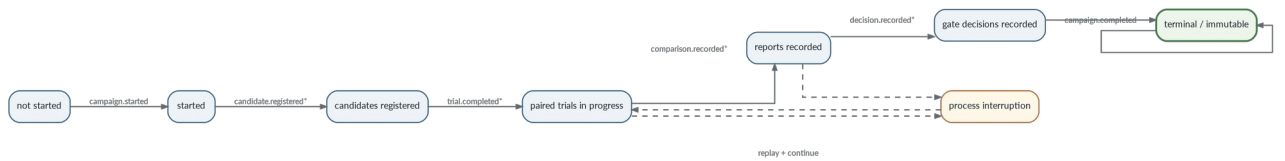
for a functor describing commands, observations, and next state, or as a labelled transition system. The implementation exposes the transition trace as events and the state update as a pure reducer.

15.2 Event-sourced operational semantics

Each event has a sequence number, campaign identity, type, timestamp, and typed payload. The reducer rejects:

- a non-start event first;
- sequence gaps;
- campaign identity mismatch;
- duplicate candidates or cells;
- report before candidate;
- decision before report;
- nonterminal trial result;
- any event after completion.

This event language is the small-step operational semantics of the campaign.



The event-sourced campaign state machine.

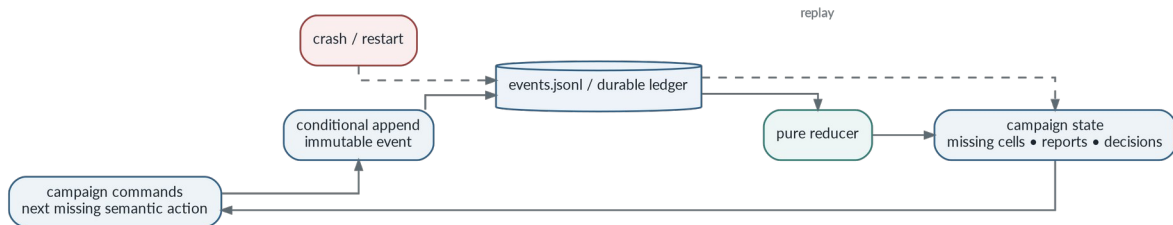
15.3 Resume as semantic continuation

Let $E = P \cdot U$ be an event history split at interruption. Replaying P yields state S_p . Continuing from S_p emits only missing events U . Correct resume requires

$$\rho^i(s_0, P \cdot U) = \rho^i(\rho^i(s_0, P), U).$$

This is reducer associativity. The engine's command selection must also be idempotent with respect to completed semantic coordinates.

The sandbox injects an event-store interruption, resumes, verifies no duplicate cells, and verifies that rerunning a terminal campaign appends nothing.



Commands, durable append, pure reduction, and replay form the resume loop.

15.4 Open interfaces

A campaign is open to proposers, runners, workloads, policies, and stores. Each component can be replaced through a narrow interface. The campaign identity binds their IDs and the material baseline/candidates, so replacement does not silently continue an old campaign.

This is the high-level plugin architecture. Its compositionality is operational rather than fine-grained categorical wiring, but the same design rule applies: the kernel owns the transition laws; grafts supply behavior at ports.

16. The unified optimization doctrine

16.1 Definition

An optimization doctrine O consists of:

1. a typed monoidal base C or free wiring presentation $W(\Sigma)$;
2. a category of parameterized systems $Para(C)$;
3. a class of lawful optics over parameter/release objects;
4. a stochastic semantic category K for trials;
5. an ordered resource and metric semantics V ;
6. an artifact and identity theory A ;
7. a campaign transition system S ;
8. a family of structure-preserving interpretations linking these layers.

A domain graft supplies a model of the relevant signatures and interfaces plus law evidence.

16.2 Optimization problem in the doctrine

A problem instance includes:

- baseline parameter/release θ_0 ;
- legal intervention family $I(\theta_0)$;
- workload distribution or finite suite W ;
- trial kernel family K_θ ;
- metric map m ;
- feasibility/gate policy G ;
- proposal controller P ;
- budget/resource policy B .

The optimizer seeks evidence for an eligible preferred candidate, not merely

$$\arg \max_{\theta} J(\theta).$$

A more faithful expression is

$$\text{Select}_G \left(\text{Pareto} \left(\left(\theta, \widehat{K}_\theta, m, \text{trace} \right) \mid \theta \in I(\theta_0), \text{evidence complete} \right) \right).$$

16.3 Plugin theorem, informally

Given a plugin signature extension $\Sigma \xrightarrow{G} \Sigma'$ and a valid model of its generators in each required interpreter, every existing plan remains valid with unchanged meaning, and every new plan receives a meaning by the same fold. No existing interpreter requires a new plan-node case.

This is the core extensibility result. In software terms, the syntax is closed under constructors and open under generators and interpreters.

16.4 Scope of proof

The free-syntax extension property is structural. The claim that a plugin operation correctly implements its declared generator is a separate refinement obligation. The claim that a candidate is better is statistical and policy-relative. The doctrine does not conflate these proof levels.

Part III. Software architecture induced by the doctrine

17. The minimal kernel

17.1 What “minimal” means

A minimal core is not a collection of empty interfaces. It is the smallest implementation that owns the laws whose divergence would make evidence incomparable. In this architecture the trusted semantic kernel consists of:

- canonical schema-bearing values and domain-separated identity;
- typed operation descriptors;
- a fixed plan syntax and validator;
- a structural fold interface;
- explicit execution outcomes and observations;
- immutable artifact references;
- exact trial coordinates and paired comparison;
- metric directions and missingness;
- ordered gate semantics;
- an event vocabulary and reducer.

This is larger than a `func(any)` any plugin host and much smaller than a RAG framework or workflow engine.

17.2 Kernel-owned versus plugin-owned laws

Kernel-owned laws apply in every domain:

- associativity and identities of sequence/tensor;
- port compatibility;
- canonical envelope validation;
- uniqueness of registry IDs;
- exact coordinate pairing;
- failure/missingness retention;
- gate order;
- event causal order and terminal immutability.

Plugin-owned laws depend on domain meaning:

- chunk ranges reconstruct source text;
- ANN recall meets a relation to an exact oracle;
- a filter enforces authorization;
- an answer citation resolves to admitted evidence;
- a Garden widget field has structured provenance;
- a quadratic loss is calculated correctly.

The registry can require plugin law evidence, but the core does not define the laws’ content.

17.3 Trusted computing base

The in-process sandbox trusts plugin code. Its semantic trusted base is the `core`, `plan`, `plugin`, `engine`, `experiment`, `decision`, and `campaign` packages. A production host can reduce the security trusted base by running plugins behind capability-restricted process boundaries. The typed signature and event protocol survive that change.

The important principle is that plugin trust and plugin semantic freedom are separate. Even trusted code should not be allowed to invent a hidden composition operator or alter campaign pairing.

17.4 Closed constructors, open generators, open interpretations

The plan constructor set should change rarely because each new constructor affects every interpreter and proof. Generator sets should change frequently through plugins. Interpreter sets should also change frequently through new folds.

This is a two-dimensional expression of the expression problem:

- extending domain operations does not modify interpreters;
- extending interpretations does not modify domain operations;
- extending core syntax is a versioned architectural event.

18. Canonical values, identity, and material evidence

18.1 Schema-bearing envelopes

A cross-plugin value is represented as

$$E = (\text{schema}, \text{payload}, \text{digest}).$$

The digest is

$$H(\text{domain} \parallel \text{schema} \parallel \text{canonicalPayload}),$$

using length-delimited, domain-separated hashing. Validation recomputes canonical form and digest. A value cannot be decoded under the wrong schema.

The envelope is not intended to replace typed in-process values. Generic typed adapters decode at the operation boundary, invoke an ordinary Go function, and encode at the output boundary.

18.2 Identity strata

At least four identities should remain distinct:

1. **Schema identity:** what a value means and how it is encoded.
2. **Plan identity:** what structural operations and generator versions are wired.
3. **Material identity:** exact bytes of a retained artifact or sampled output.
4. **Campaign identity:** which baseline, candidates, workload, runner, policy, and repeat scheme constitute the experiment.

A production architecture adds release and execution identities. Conflating them creates either unsound cache reuse or needless cache misses.

18.3 Canonicalization boundary

The sandbox defines a versioned canonical JSON subset by recursively sorting object keys, emitting no insignificant whitespace, validating finite numbers, and rejecting trailing values. It does not claim automatic compatibility with every language's numerical formatting. Cross-language use requires golden vectors and a declared codec version or adoption of a standard canonical format.

The semantic rule is independent of JSON: identity must be computed from a canonical representation whose version is itself identified.

18.4 Domain separation

The same bytes used for an envelope, artifact, plan, workload, candidate, and campaign receive different digests because each hash includes a domain tag. This prevents accidental substitution across identity roles.

Domain separation also applies to random seeds. A seed for one case/repeat coordinate should not be reused as a candidate ID or artifact key.

18.5 Immutable artifacts

Native artifacts preserve domain evidence too rich for generic metrics: channel rankings, provider transcripts, traces, generated answers, index manifests, or frontend snapshots. An artifact reference includes content digest, media type, and size. Reads verify content.

The local file store writes a temporary file, syncs it, and renames it. It is appropriate for the sandbox. A production artifact store adds conditional publication, tenant authorization, encryption, retention, garbage collection, and provenance links.

18.6 Material reproducibility

For stochastic or remote operations, semantic reproducibility may be impossible because the provider kernel changes. Material reproducibility means that the exact sampled input/output and metadata are retained. A replay interpreter can then reconstruct downstream behavior from artifacts.

Optimization reports should link both semantic specifications and material evidence.

19. Transactional plugin registration

19.1 Manifest

A plugin manifest carries stable ID, version, and description. It can be extended with:

- compatible host API range;
- signer and artifact digest;
- schema migrations;
- required capabilities;
- declared data classes;
- law-suite or certification artifact;
- deterministic build provenance.

Manifest version is not a substitute for operation versions. An operation's semantic ID should change when its input-output relation or effect contract changes.

19.2 Builder transaction

Plugin installation occurs in an isolated builder. The plugin registers local schemas and operations. The host validates:

- nonempty IDs and versions;
- unique local IDs;
- operation ownership by the manifest;
- valid ports;
- references to local or existing schemas;
- absence of global collisions;
- successful laws.

Only then are all additions committed to the registry. Failure leaves the registry unchanged.

This transaction prevents a common plugin failure mode: a partially installed plugin that makes later errors dependent on registration order.

19.3 Typed adapters

The sandbox ships generic adapters for one, two, and three inputs. Their generic type parameters connect Go values to codecs, while descriptors connect runtime ports. The adapter performs arity checking, decoding, execution, encoding, observation preservation, and duration measurement.

A direct `Operation` implementation is appropriate for streaming, variable arity, or special artifact behavior. It remains subject to the same descriptor and result contract.

19.4 Law admission

A `Law` is a named check. The RAG plugin uses it to check lens laws. Other plugins can register golden identities, normalization invariants, or finite oracle comparisons.

Executing arbitrary law code during production registration is not always desirable. A mature host can separate:

- development-time property suites;
- build-time certification;
- release-time attestation verification;
- runtime lightweight invariant checks.

The registry's semantic rule is that a plugin is not eligible until the required law evidence succeeds.

19.5 Plugin capability and effect policy

A plugin operation declares effects such as CPU, clock, random, state, artifact read/write, network, and remote disclosure. A host policy can reject plugins or plans whose effects exceed an allow-list. Capability objects can enforce the declaration by making network or artifact access impossible without host-supplied handles.

Effect declarations should be specific enough for policy. `network` alone cannot express whether customer text leaves a trust zone. A production extension can attach data-class and purpose annotations to remote effects.

20. Plans and their interpreters

20.1 Serializable intensional representation

A plan stores kind, input and output ports, operation ID, children, and permutation. It is serializable and content-addressable. It contains no function pointers. This permits:

- review before execution;
- caching of compilation;
- remote transmission;
- static policy checks;
- visual explanation;
- comparison between releases;
- plan-ID references in traces.

The registry supplies the primitive meanings at interpretation time.

20.2 Validation before fold

`Validate` recursively checks:

- non-nil plan;
- valid ports;
- identity boundary equality;
- primitive existence and signature agreement;
- sequence child compatibility and boundary agreement;
- tensor concatenation agreement;
- valid permutation;
- copy/drop shapes;
- known kind.

This catches descriptor drift: a serialized plan cannot silently execute against a plugin whose operation signature changed under the same ID.

20.3 The fold algebra

An algebra over carrier R provides:

```
type Algebra[R any] interface {
  Identity(core.Port) (R, error)
  Primitive(core.OperationDescriptor) (R, error)
  Sequence(core.Port, core.Port, []R) (R, error)
  Tensor(core.Port, core.Port, []R) (R, error)
  Permuted(core.Port, core.Port, []int) (R, error)
  Copy(core.SchemaID) (R, error)
  Drop(core.SchemaID) (R, error)
}
```

Fold is the catamorphism of the plan syntax. It is the principal plugin-proof boundary: no interpretation can skip a child without expressing that behavior in its algebra, and no domain plugin can inject a node unknown to the fold.

20.4 Execution interpreter

The execution algebra compiles each node into an arrow over envelope frames. Sequential composition feeds outputs to the next arrow and stops on failure. Tensor splits the frame according to child input arities and concatenates results. The reference tensor interpreter runs branches serially; a parallel interpreter can preserve the same port behavior with a declared observation-order relation.

Before execution, the executor validates every input envelope against the compiled port. After successful execution it validates every output.

20.5 Static-analysis interpreter

The analysis algebra combines operation IDs, effects, dependency labels, determinism, cacheability, and costs. Because it folds the same plan, an operation cannot be executed without appearing in the analysis unless the execution interpreter or plugin violates its contract.

Analysis results can become gate inputs. Examples:

- reject a candidate plan that introduces `remote.disclosure`;
- require every operation to be deterministic for an exact replay suite;
- estimate whether a build fits a memory envelope;
- compute which dependency nodes require fresh artifacts.

20.6 Additional interpreters

A mature architecture should add interpreters as separate packages:

DOT/diagram. Render operation boxes, typed wires, effects, and artifacts.

Provenance. Compute a query that maps outputs back to input schemas and operations.

Disclosure. Track data classifications and authorization certificates through wires.

Deployment. Compile supported generators into a workflow or service graph.

Simulator. Replace expensive primitives with calibrated models.

Replay. Resolve primitive results from retained artifacts.

Proof obligations. Produce a set of plugin laws or refinement certificates required by the plan.

The interpreter model avoids a “god interface” on `Plan` with methods for every future use.

21. High-level optimization ports

21.1 Space

A `Space` defines baseline specification, schema, legal patches, and application. It is the high-level presentation of a parameterized family. The baseline is a canonical envelope. Applying a patch yields a new envelope under the same release/spec schema.

A production `Space` should also expose or validate a dependency graph, locks, constraint schema, and a fidelity requirement function. The sandbox records direct targets and closure in each patch.

21.2 Proposer

A proposer maps current space and baseline to candidate patches. The reference `AllPatches` enumerates the space. Adaptive proposal requires campaign-state input and proposal events. The interface can be generalized to:

$$P : S \rightsquigarrow D(I + Stop),$$

making the proposer a stochastic policy over campaign state.

The core should not privilege one search method. A gradient plugin can require differentiable structure; a Bayesian plugin can require feature embeddings and uncertainty; an LLM proposer can emit typed patch values validated by the space.

21.3 Workload

A workload has content identity and immutable cases. Each case has stable ID, groups, and a typed input envelope. Group labels support protected strata but should not be trusted as the sole source of authorization or domain policy.

A distributional workload can be materialized into sampled cases with a sampling manifest. Production shadow traffic can be represented as a stream-backed workload whose admitted sample set is frozen for one campaign.

21.4 Runner

A runner interprets one specification on one case and coordinate. It declares metric definitions and returns one terminal trial result. This is the principal graft point for existing systems.

The runner is responsible for:

- domain-specific setup and execution;
- release or artifact resolution;
- honoring the supplied seed or documenting inability;
- recording native observations;
- projecting finite metrics;
- retaining native artifacts;
- classifying domain versus infrastructure failure.

The kernel validates coordinate integrity and metric finiteness.

21.5 Policy

A policy consumes a paired report and metric definitions. It emits a three-valued decision with ordered checks and optional score. Policy identity is part of campaign identity.

A production policy can include statistical confidence intervals, multiple-comparison control, protected group gates, operational SLOs, and human signature. The kernel only requires deterministic interpretation of the retained report and policy inputs.

21.6 Store

The store loads an event prefix and appends an event. This small interface is sufficient because the reducer owns state semantics. A production store adds expected sequence and fencing to make append linearizable across processes. It may expose snapshots as derived acceleration, but events remain authoritative.

21.7 Ports rather than callbacks

These interfaces are ports of an open campaign machine. Their IDs are material to campaign identity. A plugin cannot be swapped mid-run without creating an identity mismatch. This turns ordinary dependency injection into a semantic composition boundary.

22. Candidates and causal dependency closure

22.1 Candidate value

A candidate contains:

- baseline envelope;
- resulting specification envelope;
- one patch;
- content-derived candidate ID.

The ID binds both the declared intervention and the actual resulting spec. This prevents two different patch applications from sharing an identity merely because their patch labels match.

22.2 Dependency graph

Let $D=(N, E)$ be a directed acyclic or generally directed dependency graph over semantic nodes such as:

```
corpus.normalize
index.chunk
index.representation
index.embedding
index.lexical
index.vector
query.rewrite
query.channels
query.filter
query.fusion
query.rerank
answer.context
answer.generate
eval.retrieval
eval.answer
eval.session
```

A patch's direct target set T induces closure

$$cl(T) = \{n \mid \exists t \in T, t \leadsto n\} \cup T.$$

The planner must compute this closure. Handwritten closure remains an inspectable prototype but can be incomplete.

22.3 Artifact reuse theorem

An artifact at node n is reusable between baseline and candidate if:

1. $n \notin cl(T)$;
2. all semantic inputs to n have equal identities;
3. the artifact verifies under the same schema and operation version;
4. reuse does not violate an evaluation coupling requirement.

This is a semantic cache criterion, not simply “the file exists.”

22.4 Evaluation invalidation

The same closure determines required fidelity. If `eval.retrieval` is affected but `eval.answer` is not considered, a retrieval-only comparison may suffice for an approximation claim. In practice downstream user behavior can still be affected, so policies map semantic classes and target nodes to mandatory evaluation levels.

A candidate changing `answer.generate` cannot be promoted from cached retrieval metrics. A candidate changing only an event-store implementation may require state-machine and load evidence but not language-quality reruns if refinement is established.

22.5 Interventions over evolving baselines

A candidate is relative to one baseline. Applying it to a later release is a rebase operation requiring optic reapplication, validation, and new identity. It is not the same candidate even if the focused value is unchanged, because unaffected context and dependency artifacts may differ.

23. Trial custody and comparison

23.1 Coordinate matrix

For candidates c , cases i , and repeats r , the engine creates baseline and candidate cells. The baseline can be shared across candidate comparisons only when baseline spec, workload, runner, and seed coordinate are identical. Its coordinate carries a baseline identity distinct from every candidate.

The reference engine stores one baseline cell per case/repeat and one candidate cell per candidate/case/repeat.

23.2 Failure-preserving results

A trial result includes status, failure, metrics, output, observations, artifact, and timestamps. `Completed` means terminal, not successful. Comparison counts failed baseline and candidate cells. Metrics may be absent on failures, yielding explicit missingness.

This avoids survivorship bias from dropping provider errors or invalid contracts. A gate can require zero failures or compare failure rates.

23.3 Metric validation

The runner declares definitions before execution. Each result's metric map is checked for known IDs and finite values. Unknown or nonfinite metrics turn the cell into an infrastructure failure attributable to the runner boundary.

Metric absence is permitted and counted. This is necessary for heterogeneous outcomes, but policy must decide whether missingness is acceptable.

23.4 Paired report

For each common case/repeat key, the report stores the full baseline and candidate result, oriented deltas, and missing metrics. Aggregates include count, missing count, baseline mean, candidate mean, mean oriented delta, and minimum/maximum oriented delta.

The report is deliberately not a statistical conclusion. It is exact descriptive evidence consumed by policies or more advanced statistical gates.

23.5 Native artifacts remain authoritative

Generic reports cannot contain every domain detail. A RAG trial artifact can contain rankings, contexts, citations, and traces. A production request artifact can contain SLO telemetry. The generic outcome links to the native artifact and projects only metrics used by cross-domain campaign machinery.

Diagnosis must begin from native evidence rather than reconstructing domain behavior from metric names.

24. Ordered decisions and candidate selection

24.1 Gate interface

A gate evaluates one report under metric definitions and returns named status, message, and numeric values. The gate is pure with respect to retained evidence. External human or production decisions should first be materialized as signed evidence or explicit events.

The sandbox provides:

- coverage gate;
- mean oriented-delta gate;
- worst-case oriented-delta gate;
- absolute candidate-mean gate.

24.2 Policy sequence

`decision.Sequence` applies gates in order and stops on fail or indeterminate. If all pass, it reads a target aggregate as score. This simple mechanism encodes the key separation between eligibility and preference.

A richer policy may produce a partial order or set of eligible candidates. The campaign terminal selection can then be a separate selection function. The reference chooses the highest passing target score with deterministic candidate-ID tie-break.

24.3 Pareto-aware extension

The `metric` package implements dominance and Pareto front. A production campaign can accumulate eligible candidate-level summary vectors and compute the frontier before human or policy selection.

Candidate-level vectors should include uncertainty or gate evidence where needed. A point estimate Pareto front is not sufficient for stochastic promotion.

24.4 Policy monotonicity

A hard gate should be monotone with respect to evidence refinement where possible. For example, discovering a missing failed cell must not improve coverage. Adding a stricter earlier security gate cannot make a previously failing candidate pass.

Three-valued logic helps formalize this: partial evidence is indeterminate, and completion can refine it to pass or fail.

25. Campaign identity, event sourcing, and distributed refinement

25.1 Campaign identity

The sandbox computes campaign ID from:

- campaign name and repeat count;
- space ID;
- proposer ID;
- workload digest;
- runner ID;
- policy ID;
- baseline digest;
- sorted candidate IDs.

Execution timestamps, worker count, and event path are excluded. Changing semantic inputs makes an existing event file incompatible rather than silently continuing.

Adaptive campaigns require identity over proposer version and initial policy, with candidate proposals added as events. The terminal campaign identity can then be a Merkle root of the event log or a run root referencing all proposal material.

25.2 Append-only store

The local JSONL store serializes in one process, appends one JSON event, and syncs the file. It is deliberately transparent. The reducer verifies sequence and causality on load.

A distributed store should expose:

```
Append(ctx, expectedSequence, fence, event) error
```

and guarantee at most one successful append for an expected sequence/fence. Workers submit trial completions to a coordinator or an idempotent cell-commit API.

25.3 Command/event separation

“Run trial” is a command; `trial.completed` is an event. A command may be retried. The event is appended once under coordinate uniqueness. This is the correct place to handle at-least-once worker delivery.

The event reducer should never call providers or artifacts. It is pure and deterministic.

25.4 Snapshots

A large event log can be accelerated by a state snapshot identified by event sequence and log prefix digest. On load, the store verifies the snapshot and reduces only the suffix. The snapshot is derived and disposable; the event prefix remains authoritative.

25.5 Cancellation

Cancellation has two scopes:

- cancelling one trial produces a terminal cancelled cell;
- cancelling a campaign produces a terminal campaign event or a policy-defined paused state.

The sandbox models cancelled trial status but not a campaign-cancel event. A production extension should add explicit transitions rather than infer cancellation from process exit.

25.6 Terminal immutability

A terminal campaign cannot accept more evidence. New evidence creates a new campaign or a versioned continuation whose relationship is explicit. This prevents retroactive movement of promotion criteria.

26. Versioning, migration, and security

26.1 Version dimensions

The architecture distinguishes:

- schema version;
- operation version;
- plugin manifest version;
- plan schema version;
- codec/canonicalization version;
- space/proposer/runner/policy version;
- workload and label-policy identity;
- event schema version;
- artifact media schema.

A single semantic version on the executable cannot safely substitute for these identities.

26.2 Schema migration as a plan

A migration from A_1 to A_2 should be a typed operation $m: A_1 \rightarrow A_2$, with its own effects and laws. In-place reinterpretation of old payload bytes under a new schema ID is forbidden.

Migration of a terminal campaign does not append converted events to the old log. It produces a new derived report or event stream whose provenance names the source campaign and migration.

26.3 Plugin compatibility

A host should verify a plugin's API range and operation/schema contracts. Existing serialized plans detect signature drift. Behavioral compatibility requires law/differential evidence.

If a plugin implementation changes under the same operation ID but claims semantic equivalence, release certification should include a refinement report. Caches can be reused only under that declared equivalence policy.

26.4 Untrusted plugins

The in-process Go interface is not a security sandbox. For untrusted plugins, suitable hosts include:

- separate processes over canonical RPC envelopes;
- WASM components with capability-limited imports;
- container jobs with signed manifests;
- remote services with mTLS and policy certificates.

The registry can store a proxy operation whose execution interpreter calls the isolated host. Static interpreters remain local because descriptors are data.

26.5 Supply-chain identity

A production manifest should bind plugin binary/module digest, build provenance, signer, and declared source revision. Operation semantic ID alone does not identify the executable that ran.

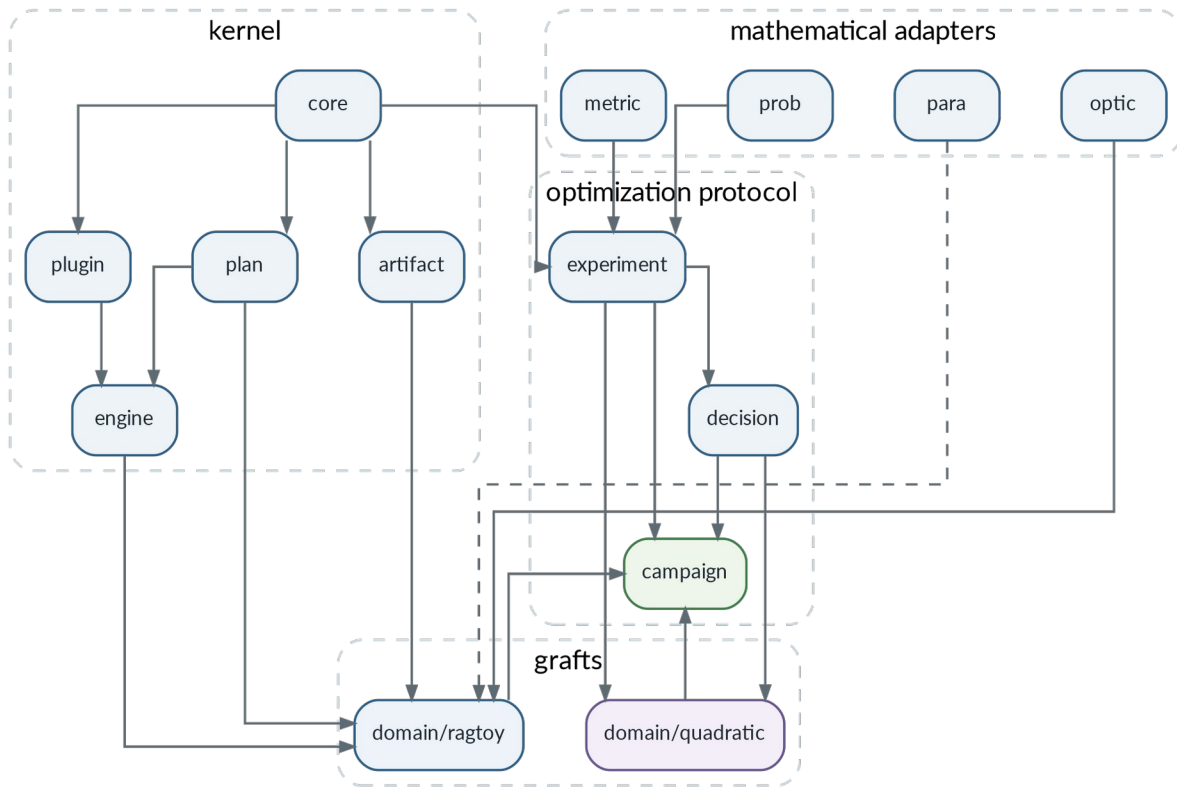
Material trial traces should record both semantic operation version and implementation artifact identity.

26.6 Data security

Canonical envelopes may contain sensitive data. Content addressing does not imply public addressing. Artifact and event stores require authorization and encryption. Digests can leak equality across tenants if shared indiscriminately. A multi-tenant deployment may use scoped namespaces or keyed digests for sensitive material while retaining semantic IDs inside an authorized boundary.

27. Package architecture

The reference module separates the doctrine into focused packages rather than one framework package.



Reference implementation package architecture.

Package	Stable responsibility
core	schemas, ports, canonical envelopes, digests, effects, outcomes
plugin	codecs, typed operations, transactional registry, laws
plan	free typed wiring syntax, validation, identity, fold, analysis
engine	reference execution interpretation
artifact	content-addressed material store interface and local implementation
para	parameterized maps, composition, tensor, reparameterization
optic	lawful lenses and executable law checks
prob	finite distributions, kernels, deterministic seed splitting
metric	directions, oriented differences, Pareto dominance
experiment	spaces, patches, candidates, workloads, cells, comparison
decision	three-valued gates and ordered policies
campaign	event vocabulary, reducer, engine, resume, store
domain/ragtoy	fine- and coarse-grained RAG graft
domain/quadratic	unrelated high-level graft

The package graph is intentionally not perfectly layered in mathematical order. Go packages are organized around stable implementation responsibilities and acyclic dependencies. The doctrine supplies the conceptual unification.

Part IV. The executable sandbox

28. Implementation method and verification status

28.1 Goals

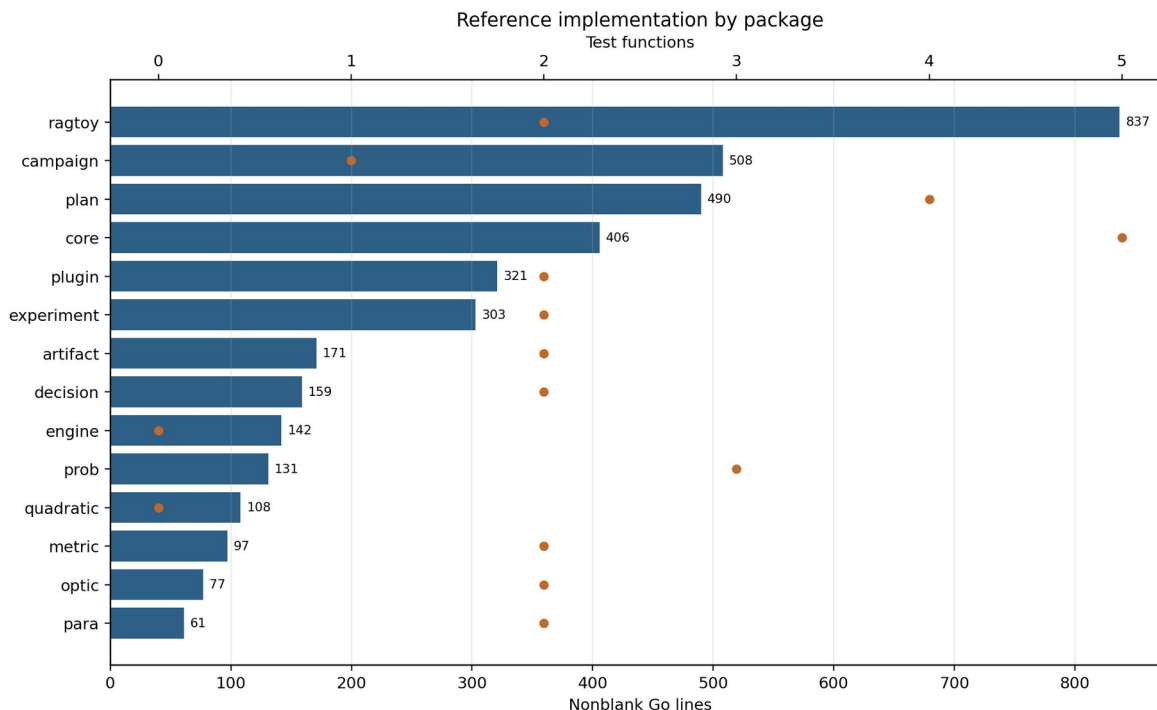
The implementation was built to answer five questions experimentally:

1. Can a useful typed wiring syntax remain small?
2. Can plugins add operations without changing the syntax or interpreters?
3. Can the same plan be executed and statically analyzed through a generic fold?
4. Can a domain-neutral campaign kernel support both RAG and an unrelated stochastic problem?
5. Can dependency-aware reuse, exact pairing, gates, and resume be demonstrated without external services?

The module is deliberately standard-library-only. It uses Go 1.23 language and library features and avoids code generation, reflection-heavy dependency injection, databases, and network providers.

28.2 Scale

The reference implementation contains approximately 3,700 nonblank Go lines across fourteen packages, plus documentation, examples, and scripts. The RAG graft is the largest package because it contains a corpus, chunker, index, retrieval functions, evaluation data, plugin, space, runner, policy, and tests. The categorical kernel remains distributed across small packages whose individual responsibilities can be inspected.



Reference implementation size and test distribution.

28.3 Verification performed

The final module was run through:

```
go test ./...  
go vet ./...  
go test -race ./...
```

All completed successfully. Statement coverage from the normal test run is about 64 percent overall, with the RAG graft and probability package above 80 percent and intentionally thin command/reference-interpreter packages lower. Coverage is not used as a correctness claim; it identifies where future law tests would be useful.

28.4 Reproduction

The repository root contains:

```
make verify  
make demo-rag  
make demo-quadratic  
./scripts/reproduce.sh
```

The reproduction script deletes generated output, reruns verification, and executes both domains. Generated campaign timestamps are execution identity and will differ; structural plan IDs, workload/candidate IDs, decisions, and deterministic trial behavior remain stable under the same code and inputs.

29. `core`: identity and outcomes

29.1 Domain-separated digest

`core.Sum` hashes a domain tag and a length-delimited sequence of byte parts with SHA-256. The result is rendered as `sha256:<hex>`. Validation checks prefix, length, and hexadecimal encoding before a digest is used as a filesystem key.

Length delimiting avoids ambiguity such as `(ab,c)` versus `(a,bc)`. Domain separation prevents a plan digest from being substituted as an artifact reference merely because the raw hash happens to match.

29.2 Canonical JSON

`CanonicalJSON` marshals a typed value and then canonicalizes it. `CanonicalizeJSON` uses `json.Number`, recursively sorts object keys, emits compact arrays/objects, validates finite numbers, and ensures the decoder reaches EOF after one value. Tests cover key order, invalid numbers, and multiple/trailing values.

The codec is versioned by usage domains rather than advertised as a complete standards implementation. A production cross-language deployment would add golden vectors or use deterministic CBOR/JCS with explicit version.

29.3 Envelope validation

`NewEnvelope` creates canonical payload and digest. `Envelope.Validate` re-canonicalizes the payload, checks byte equality, and recomputes digest. `DecodeEnvelope[T]` then requires exact schema and disallows unknown JSON fields.

Disallowing unknown fields is a deliberate compatibility choice at plugin boundaries. Schema migration must be explicit; silent forward-field loss would invalidate identity and semantics.

29.4 Effects and costs

`OperationDescriptor` includes typed ports, effects, determinism, cacheability, dependencies, and a cost hint. Effects are normalized as a sorted unique set for stable analysis. Cost fields are finite primitive values; a production descriptor should validate all numbers and may separate estimated from measured cost.

29.5 Execution outcome

`core.Execution` contains status, outputs, failure, observations, artifacts, and duration. Helpers construct success and failure values. The operation adapter returns infrastructure failure for arity, decode, encode, or implementation-boundary errors. Domain operations can return richer classified outcomes by implementing `Operation` directly.

30. `plugin`: models of signatures

30.1 Registry data structures

The registry holds maps of schemas, operations, and plugin manifests behind an RW mutex. Registration uses a private builder and commits only after validation and laws. Read methods return operation descriptors and sorted operation lists.

The registry is safe for concurrent reads and serialized registration. The intended lifecycle registers plugins before campaign execution; dynamic mutation would require registry-version identity and plan recompilation.

30.2 Typed unary operation

A unary adapter has the conceptual form:

```
type Unary[I, O any] struct {
  Desc core.OperationDescriptor
  In   Codec[I]
  Out  Codec[O]
  Run  func(context.Context, I) (O, []core.Observation, error)
}
```

The adapter is generic but the registry stores it behind the `Operation` interface. Its descriptor is the existential type witness: it preserves the input/output schema IDs after the concrete generic types have been erased into the heterogeneous registry.

30.3 Transactionality test

One test installs a plugin whose law always fails and verifies that the registry still contains no operations. Another registers two plugins that attempt to own the same schema and verifies collision rejection. These tests exercise the semantic atomicity of plugin installation.

30.4 RAG plugin law

`domain/ragtoy.Plugin.Laws` checks every configuration lens over representative valid states and values. Registration therefore establishes that the candidate space's advertised local updates satisfy get-put, put-get, and put-put on those fixtures.

This is a useful pattern: domain law evidence is attached to the plugin that owns the semantics, while the registry owns the admission transaction.

31. `plan`: syntax, normalization, and folds

31.1 Constructors

`Identity`, `Primitive`, `Copy`, `Drop`, and `Permute` construct atomic terms. `Seq` and `Tensor` flatten nested nodes. `Seq` checks all boundaries before removing identity nodes, preventing an ill-typed identity from concealing a mismatch.

Empty sequence/tensor yields identity on the unit port. A single nonidentity child is returned directly. This normalization creates stable representations without a separate rewrite pass.

31.2 Plan identity

`plan.ID` canonicalizes the normalized plan and hashes it under `opfield/plan/v1`. Tests construct left- and right-associated sequences/tensors and verify equal IDs. Identity insertion also preserves the ID.

Plan identity is intensional modulo the implemented normalization. Two different primitive decompositions with extensionally equal functions remain different plans, as they should for effects, provenance, and plugin versioning.

31.3 Validation

A primitive plan stores its input/output ports as captured at construction. At validation, the registry descriptor is looked up and compared. If a plugin changes a signature under an existing operation ID, the plan fails with signature drift.

This is a strong reason to serialize ports in the plan rather than storing only operation IDs: the plan carries the contract under which it was constructed.

31.4 Generic fold

The `Algebra[R]` interface and `Fold` make the plan an initial-algebra-like syntax. `Fold` validates before interpreting, then recursively supplies child interpretations to sequence and tensor methods.

A new interpreter can be added in one package without changing `Plan`, the registry, or plugins. Conversely, a new generator registered by a plugin is handled by every interpreter through `Primitive(descriptor)`.

31.5 Analysis algebra

The analysis carrier is:

```
type Analysis struct {
  Operations  []core.OperationID
  Effects     []core.Effect
  Dependencies []string
  Deterministic bool
  Cacheable   bool
  Cost        core.CostHint
}
```

Structural maps are deterministic and cacheable with zero declared cost. Primitive analysis copies descriptor annotations. Sequence/tensor combine through the algebra described in Chapter 7.

31.6 Structural tests

The plan tests create a plugin with increment, double, and square operations. They verify:

- sequence associativity and identity yield stable IDs;
- the composite executes correctly;
- tensor associativity yields stable ID;
- copy and permutation preserve two outputs;
- signature drift is rejected;
- static analysis sees both operations and preserves deterministic/cacheable conjunction.

These are finite executable instances of the wiring laws.

32. `engine`: reference execution semantics

32.1 Compilation

`Executor.Compile` folds a plan into a `Compiled` value with input port, output port, and `Arrow`:

```
type Arrow func(context.Context, []core.Envelope) core.Execution
```

Compilation resolves primitive operations once. Execution validates the input frame, context state, and successful output frame.

32.2 Sequential composition

The compiled sequence clones the input frame, executes children in order, accumulates observations and artifact references, and returns immediately on first failure while preserving prior evidence.

This implements fail-fast operational semantics. A different interpreter could implement error accumulation where types permit; it would be a distinct execution policy and identity.

32.3 Tensor composition

The reference tensor interpreter slices the input frame by child arity, evaluates children in canonical order, and concatenates outputs. It is semantically parallel but operationally serial. This choice keeps the reference deterministic and simple.

A concurrent interpreter must define cancellation when one branch fails, stable output ordering, observation branch IDs, and resource admission. It can still implement the same plan algebra.

32.4 Primitive observations

When a plugin observation lacks an operation ID, the primitive interpreter attaches the descriptor ID. This ensures traces can attribute observations without requiring each typed function to repeat its own identity.

33. `para`, `optic`, and `prob`

33.1 Para implementation

`para.Parametric[P,A,B]` wraps `Run(P,A)(B,error)`. `Compose` returns a parameter object `Pair[P,Q]`; `Tensor` composes independent inputs and outputs; `Reparameterize` maps new coordinates into old parameters.

The tests verify a composed add-then-multiply system and a string-length reparameterization. A more formal implementation would include identity and associator tests across nested product representations.

33.2 Lens implementation

`optic.Lens[S,A]` contains ID, get, partial put, and equality functions. Equality is explicit because floating-point or semantic records may require a domain relation rather than `Go ==`.

`CheckLaws` enumerates states and values. Invalid values rejected by `Put` are skipped; therefore the supplied fixture set must cover the advertised valid domain. Property generators are the natural extension.

`optic.Compose` builds a nested lens by reading the outer focus, updating it through the inner lens, and writing it back.

33.3 Finite distributions

`prob.Dist[T]` is a map from comparable values to normalized weights. `NewDist` rejects negative, nonfinite, and zero-total mass. `Pure`, `Bind`, `Map`, and `Product` implement finite distribution composition.

Tests verify left identity, right identity, and associativity of bind within numerical tolerance. Kernel composition and tensor are tested on deterministic kernels.

33.4 Seeds

A `Seed` is derived from a string by SHA-256 and can be split by a label. Split seeds are deterministic and domain-separated. The seed produces a local pseudorandom generator for trial implementations.

The seed is not a cryptographic random capability and should not be used for secrets. It is a reproducible coupling coordinate.

34. `metric`, `experiment`, and `decision`

34.1 Metric vector

A metric definition carries ID, direction, unit, and description. A vector validates known IDs and finite values. `OrientedDelta` implements direction normalization. `Dominates` and `ParetoFront` provide exact point-estimate order operations.

Tests verify minimize/maximize behavior and Pareto filtering.

34.2 Candidate construction

`experiment.NewCandidate` validates baseline and resulting spec envelopes, validates patch identity, canonicalizes the patch, and hashes all three. The candidate retains both baseline and result so reports are self-describing.

The sandbox does not permit a candidate with multiple patches. Structured composed interventions can be added later without changing the campaign coordinate concept.

34.3 Workload identity

`NewSuite` rejects empty and duplicate case IDs and validates every case envelope. It hashes suite name and cases. Case order is preserved in the digest in the reference implementation; because engine execution uses sorted candidate IDs and workload order only for scheduling, a future suite canonicalization could sort cases by ID if order is intended to be nonsemantic.

34.4 Comparison

`Compare` builds unique baseline and candidate maps keyed by case/repeat. It rejects duplicate baseline or candidate cells and missing candidate pairs. It stores full pairs and calculates per-metric accumulators.

A production comparison should also reject an extra candidate cell absent from the baseline map, verify arm/candidate coordinate identities more strictly, and support group-level aggregates and confidence procedures. The reference demonstrates the essential exact-pair behavior.

34.5 Gates

Coverage can require both complete pairs and zero failed arms. Mean and worst-case gates operate on oriented deltas. Absolute gates operate on candidate means. The sequence returns at the first fail or indeterminate.

The RAG policy uses recall as target but first requires complete successful coverage, nonnegative worst-case recall, nonnegative mean recall, bounded context words, and nonnegative worst-case hit rate.

35. `campaign`: executable dynamic semantics

35.1 Engine setup

The engine validates config and plugin presence, validates metric definitions, loads baseline, checks the space schema, obtains patches, applies them, constructs candidates, sorts candidates by ID, and computes campaign identity.

This means invalid candidate application fails before any event is written. The campaign's candidate set is fixed for the reference nonadaptive engine.

35.2 State reconstruction

The engine loads all events and calls `Reduce`. It creates an `appendEvent` closure that assigns next sequence, timestamp, and campaign ID, writes the event, and then applies it to in-memory state. Store failure prevents state transition.

If the state is terminal, the engine returns without mutation.

35.3 Trial scheduling

For each candidate, case, and repeat, the engine derives one shared seed. It ensures a baseline cell exists and then a candidate cell. Because baseline coordinate uses a common baseline ID, baseline results are reused across all candidates.

The runner's returned coordinate is checked. Metric validation occurs after the runner returns. Any boundary violation is converted into a terminal infrastructure-failure cell rather than aborting event custody.

35.4 Report, decision, and completion

After a candidate's cells are present, the engine gathers baseline and candidate trials, sorts them, compares, records the report, applies policy, and records the decision. After all candidates, it selects the highest-scoring passing candidate with deterministic ID tie-break and appends completion.

This schedule is sequential and simple. A distributed implementation can execute cells concurrently because coordinates and event validity are explicit.

35.5 Resume test

The test store fails after a fixed number of appends. The first engine run returns an error with a valid event prefix. A second run loads the prefix and continues. It verifies terminality, nonempty selection, no duplicate trial keys, and no new events on another terminal rerun.

This test demonstrates that resume semantics derive from event state, not from a special checkpoint code path.

36. RAG graft: fine-grained plan plus high-level campaign

36.1 Domain values

The miniature RAG domain defines:

- `Spec`: chunk words, overlap, lexical weight, vector weight, and top-k;
- `Corpus` and documents;
- `ChunkSet`;
- `Index` with lexical terms and hashed semantic vectors;
- `CaseInput` with query and relevant document IDs;
- `RetrievalResult` and `TrialOutput`;
- `ParameterValue` for typed patch payloads.

The system is deterministic and local so architecture can be tested without network noise.

36.2 Primitive operations

The plugin registers four operations:

$$\begin{aligned} & \text{chunk} : \text{Spec} \otimes \text{Corpus} \rightarrow \text{ChunkSet}, \\ & \text{index} : \text{ChunkSet} \rightarrow \text{Index}, \\ & \text{retrieve} : \text{Index} \otimes \text{Spec} \otimes \text{Case} \rightarrow \text{RetrievalResult}, \end{aligned}$$

and

$$\text{measure} : \text{RetrievalResult} \rightarrow \text{TrialOutput}.$$

Each declares dependencies and static cost.

36.3 Build plan

The build plan must feed both spec and corpus into chunking and then index the chunks. It is a sequence:

```
[Spec, Corpus] --chunk--> [ChunkSet] --index--> [Index]
```

Its stable plan digest in the generated example is:

```
sha256:85b789bed04ef6b1de388011166964e8d1174427cc19723a52b4cd22c307ed57
```

Static analysis reports operations `ragtoy.chunk/v1` and `ragtoy.index/v1`, effects `artifact.write` and `cpu`, dependencies over corpus normalization/chunking/lexical/vector index, deterministic/cacheable true, work 6, critical path 6, and memory hint 1 MiB.

36.4 Query plan

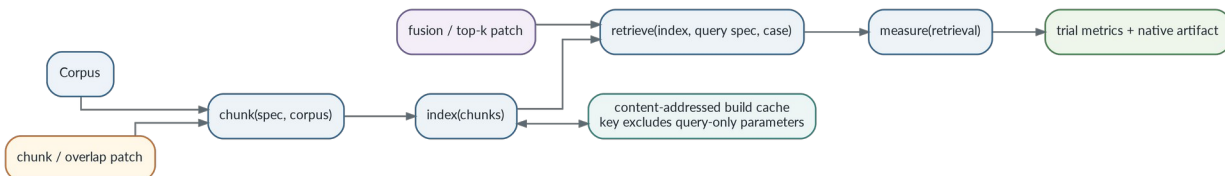
The query plan is:

```
[Index, Spec, Case] --retrieve--> [Retrieval] --measure--> [Trial]
```

Its digest is:


```
sha256:af800326216f634edf702fb2ce25e93db4cedacaa8bf655959b4bb85415d29b7
```

Static analysis reports artifact read and CPU effects, retrieval/evaluation dependencies, work 4, and critical path 4.



The RAG graft separates build-changing and query-only interventions.

36.5 Retrieval algorithm

The toy index tokenizes chunk text for lexical overlap and builds a fixed hashed semantic vector. Query execution computes lexical and vector scores, combines them under weights, applies deterministic tie-breaking, and returns top-k chunks. The evaluator calculates recall, reciprocal rank, hit rate, number of scored chunks, and context words.

This is not intended as a competitive retrieval implementation. It is complex enough to create meaningful dependency and metric interactions.

36.6 Lawful space

Default spec is 24-word chunks, 4-word overlap, equal lexical/vector weights, and top-k 2. The space advertises seven interventions:

- chunk size 14;
- chunk size 34;
- overlap 8;
- lexical weight 1.75;
- vector weight 1.75;
- top-k 4;
- top-k 1.

Each patch uses a named lens and typed value. Chunk/overlap patches target build nodes and close through indexes and evaluation. Weight/top-k patches target query nodes and do not invalidate the build.

36.7 Build reuse

The runner computes a build key from only chunk size, overlap, and corpus envelope digest. If a verified index envelope artifact exists, it is reused. Query-only candidate cells therefore reuse the same build; chunk-changing candidates build distinct indexes.

This is a concrete demonstration of parameter-dependency separation. In a production system the key would be generated from the dependency graph and semantic operation identities rather than a handwritten struct.

36.8 Trial execution

For each cell, the runner decodes spec and case, resolves/builds index, executes the query plan, decodes trial output, projects metrics, writes a content-addressed native trial artifact, and returns observations including whether the build cache hit.

Infrastructure failures become terminal trial results.

36.9 Workload

The sample suite contains nine cases, including exact terms, semantic paraphrases, and at least one case with multiple relevant documents. This creates a trade-off: top-k 1 and a high vector weight lose recall on some cases, while larger chunks improve mean recall under the toy corpus.

36.10 Campaign result

With three repeats, the campaign evaluates seven candidates over nine cases. Because the domain is deterministic, repeats provide identical values but exercise coordinate and custody semantics. The campaign records 239 events.

Results are:

Patch	Decision	Candidate mean recall	Oriented mean recall change
chunk-14	pass	0.916667	0.000000
chunk-34	pass	0.944444	0.027778
lexical-1.75	pass	0.916667	0.000000
overlap-8	pass	0.916667	0.000000
topk-1	fail	0.861111	-0.055556
topk-4	pass	0.916667	0.000000
vector-1.75	fail	0.861111	-0.055556

The selected candidate is [chunk-34](#), the only passing candidate with positive target score. The result is not a general claim about chunk size; it demonstrates the full causal path from lawful intervention to build invalidation, paired cells, gates, and deterministic selection.

37. Quadratic graft: proving domain neutrality

37.1 Domain

The quadratic domain has spec (x, σ) and cases with target t . It reports

$$distance = |x - t|$$

and

$$loss = (x - t)^2 + \epsilon,$$

where ϵ is deterministic pseudo-random noise derived from the paired seed and bounded by σ .

The space proposes $x \in \{1, 2, 3, 4\}$ from baseline $x=0$. Both metrics are minimized.

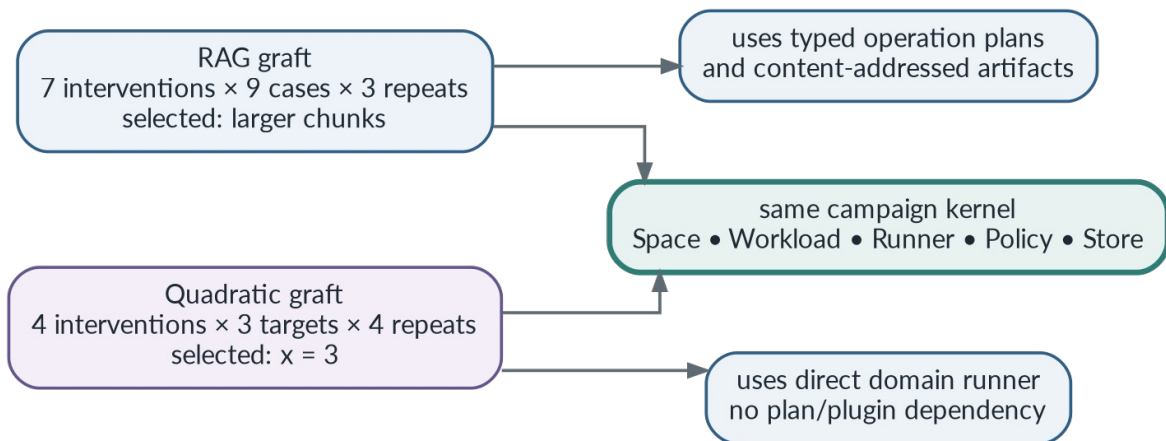
37.2 No low-level plugin dependency

The quadratic runner does not use `plugin`, `plan`, `engine`, or `artifact`. It implements only the high-level optimization interfaces. This demonstrates that existing systems can gain campaign custody without adopting fine-grained plan syntax.

37.3 Campaign result

The workload has targets 2.8, 3.0, and 3.2 and four repeats. All candidates improve over baseline. The oriented loss scores are approximately 5, 8, 9, and 8 for $x=1, 2, 3, 4$. The selected candidate is $x=3$.

The campaign records 74 events. The same state machine, exact pairing, metric orientation, decision sequence, and selection function used for RAG apply unchanged.



Two unrelated domains graft onto the same campaign interfaces.

38. What the sandbox establishes—and what it does not

38.1 Established by construction and tests

The sandbox establishes:

- plugins can extend generators without changing plan syntax;
- one structural fold supports execution and analysis;
- normalized sequence/tensor identities are stable for tested laws;
- typed codecs and plan validation reject schema drift;
- local RAG patches can be law-checked;
- build-relevant and query-only parameters can have different cache identities;
- exact paired cells and deterministic seed coupling are reusable across domains;
- metric direction and gate order can remain domain-neutral;
- interruption/resume can be event-semantic;
- a complete application can integrate at a coarse runner boundary.

38.2 Not established

The implementation does not prove the general categorical coherence of every Go algebra. It does not prove plugin implementations meet descriptors. It does not provide statistical uncertainty, adaptive proposals, distributed linearizability, production security, or release activation. The toy RAG outcomes do not validate a real retrieval strategy.

These are extension and assurance tasks described in later chapters. The sandbox's value is to make their interfaces and laws precise enough that stronger implementations can be substituted without redesigning the field.

Part V. Grafting the backbone onto RAG optimization

39. From `ragopt` and `ragkit` to one optimization field

39.1 Existing strengths

The supplied code already contains two substantial but deliberately different centers of gravity. `ragkit` owns reusable RAG-domain mechanisms: document and chunk identity, derived representations, embeddings, lexical and vector indexes, immutable bundles, retrieval, fusion, answer-context construction, grounded contracts, evaluation metrics, and within-stage execution controls. `ragopt` owns experiment custody: immutable candidates, repeated baseline/candidate cells, exact pairing, durable run state, comparison, ordered gates, and promotion evidence.

Neither package is defective because it does not contain the other. The architectural problem is that their composition currently occurs in application-specific commands and adapters. The RAG-TTC tool-evaluation adapter, GEC parameter sweeps, Garden calibration runner, and ANN bakeoff each independently decide:

- what a candidate changes;
- which artifacts can be reused;
- which cases and repeats constitute a fair comparison;
- what failures enter the denominator;
- how native observations become generic metrics;
- what decision sequence makes a candidate eligible.

The categorical backbone is the missing middle. It does not merge `ragkit` and `ragopt`. It gives them a common doctrine:

```
ragkit values and operations
  ↓ register typed generators / implement domain ports
opfield plan + intervention + experiment semantics
  ↓ exact campaign protocol
ragopt-compatible custody and decision
```

The same doctrine also accepts complete product runners that do not expose their internal plans. This is important because GEC and Garden cannot be reduced to a chain of generic retrieval calls without losing authorization, conversation, structured facts, and frontend projection.

39.2 Proposed ownership boundary

The target ownership is:

Concern	Owner
RAG document, chunk, representation, evidence, release, and query semantics	<code>ragkit</code>
Primitive RAG operation implementations and descriptors	<code>ragkit</code> plugins or product adapters
Free wiring, parameter composition, optics, effect algebra, plan identity	small optimization backbone
Candidate and trial protocol, paired comparison, gates, durable campaign state	<code>ragopt</code> or the backbone's campaign packages
Product-specific source meaning, authorization, prompts, structured facts, judges, widgets	GEC, TTC, and Garden
Deployment activation and traffic routing	product release/runtime layer

The reference implementation places all pieces in one module for inspectability. A production repository can preserve the boundary as separate modules. The decisive dependency rule is that the optimization kernel imports neither `ragkit` nor any product package. A thin RAG specialization may import both.

39.3 Mapping the low-level surface

Representative `ragkit` operations can be registered as generators:

RAG operation	Typed generator	Important annotations
normalize source revision	Revision $\rightarrow$ Document*	policy, parser, deterministic, source disclosure
chunk document	Document $\rightarrow$ Chunk*	chunker identity, locality, work
generate representation	Chunk $\rightarrow$ Representation*	provider, prompt, stochastic material, cost
embed batch	Representation* $\rightarrow$ Vector*	provider, model, rate/cost, cacheable
build lexical index	Chunk/Representation* $\rightarrow$ LexicalIndex	artifact write, analyzer identity
build vector index	Vector* $\rightarrow$ VectorIndex	exact/approximate capability, memory
rewrite query	Query $\rightarrow$ QueryVariant*	stochastic or deterministic, fallback
lexical/vector search	Index $\times$ Query $\rightarrow$ Ranking	filter capability, latency
collapse/fuse	Ranking* $\rightarrow$ Ranking	deterministic order, finite scores
authorize	Subject $\times$ Ranking $\rightarrow$ AuthorizedRanking	policy effect, proof object
rerank	AuthorizedRanking $\times$ Evidence $\rightarrow$ Ranking	remote disclosure, provider, fallback
admit context	Ranking $\times$ Budget $\rightarrow$ EvidenceContext	boundedness, diversity
generate answer	Conversation $\times$ Context $\rightarrow$ Draft	stochastic, provider disclosure
validate contract	Draft $\times$ Evidence $\rightarrow$ ValidatedAnswer	deterministic safety kernel

The type system should distinguish `Ranking` from `AuthorizedRanking`; otherwise an ordinary sequence can send an unfiltered ranking to a remote reranker. A static effect interpreter can add a second defense by rejecting any path where an operation annotated `remote.text` is not dominated by an authorization proof.

39.4 Mapping the high-level surface

Existing applications can adopt the coarse protocol immediately:

- `Space` exposes a typed release/configuration view and lawful patches.
- `Proposer` enumerates or adaptively generates candidate patches.
- `Workload` supplies immutable cases and strata.
- `Runner` executes the actual application and writes a native artifact.
- `Policy` translates product constraints into ordered gates.
- `Store` supplies durable campaign events and artifacts.

A GEC runner may call `knowledge.Service.Search` and its answer judge. A Garden runner may create a real conversation, submit turns and choices, wait for a terminal projection, and evaluate widgets. A RAG-TTC runner may materialize a candidate tool configuration and execute a full agent loop. None must first expose every internal operation as a plan.

Later, selected internal operations can move to the low-level surface when alternate interpretation has value. This provides an incremental migration path rather than an architecture rewrite.

39.5 The RAG optimization doctrine

For a RAG release configuration R , let θ be its parameter object, S a source snapshot, X a workload case, and ρ an environment. A complete domain graft supplies:

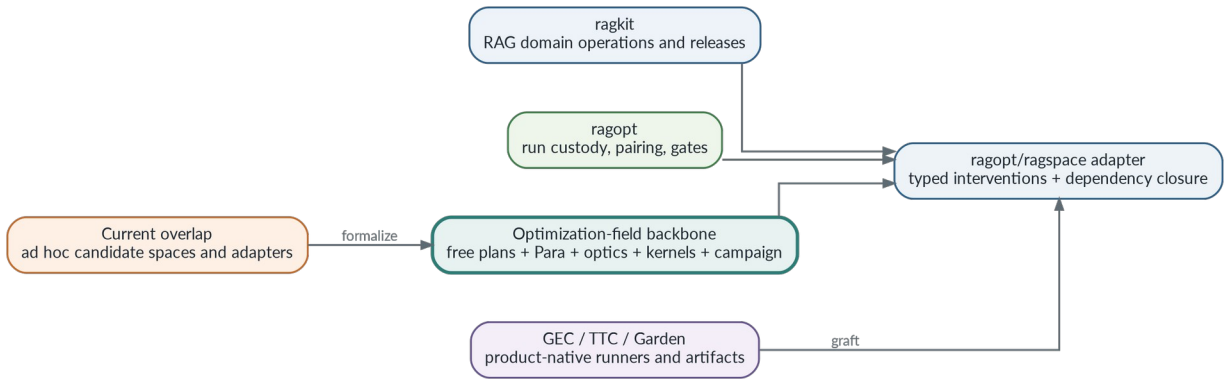
$$\text{build}:\Theta_B \otimes S \rightsquigarrow I,$$

$$\text{query}:\Theta_Q \otimes I \otimes X \rightsquigarrow O,$$

and a decision relation over distributions of O . The combined parameter object is not necessarily a Cartesian vector with uniform meaning. It is assembled from component parameter objects through wiring:

$$\Theta = \Theta_B \otimes \Theta_Q \otimes \Theta_A \otimes \Theta_P.$$

Here θ_B may alter corpus derivation, θ_Q retrieval, θ_A answer/agent behavior, and θ_P presentation. An optic selects one lawful local view, while the dependency graph determines which downstream artifact and evaluation objects must be reinterpreted.



The backbone maps current RAG packages into generators, domain ports, and campaign custody without merging their ownership.

40. Indexing optimization as compositional derivation

40.1 Indexing is a family of parameterized morphisms

Index construction is often treated as a single black-box function from corpus to bundle. For optimization, the useful denotation is a composition:

$$S \xrightarrow{N_{\theta_s}} D \xrightarrow{C_{\theta_c}} K \xrightarrow{P_{\theta_p}} R \xrightarrow{E_{\theta_e}} V \xrightarrow{B_{\theta_b}} I.$$

The intermediate objects are normalized documents D , chunks K , searchable representations R , vectors V , and physical/logical indexes I . Each component is parameterized and may have different effects, locality, determinism, and artifact semantics.

In $\text{Para}(\mathcal{C})$, composition yields one parameterized system whose parameter object is the tensor product of component parameters. This makes invalidation structural. A change focused on θ_C cannot affect normalization identity unless the dependency graph explicitly contains a backward relation; it does affect all morphisms downstream of chunking.

40.2 Derivation keys as semantic interpretation

A cache key should be the interpretation of the relevant plan prefix and inputs, not a hand-maintained struct. For a stage f with descriptor d_f , parameters p , and input artifacts a , define:

$$key_f = H(schema_f, d_f, p, a, policy_f).$$

For a composite plan, the artifact interpreter computes keys inductively. Sequence consumes the prior artifact references; tensor computes independent keys for branches; structural maps rearrange references without generating new semantic material. Operational settings such as worker count are excluded unless they can alter material output.

The key law is:

If two stage invocations have equal semantic keys and all referenced artifacts verify, their outputs may be reused only under the stage's declared observational equivalence.

For deterministic stages, this is exact material equivalence. For stochastic provider stages, reuse means replaying retained material from a prior sampled outcome, not claiming the provider distribution is deterministic.

40.3 Locality and affected closure

Each generator declares an invalidation relation over dependency labels. Document-local chunking may declare:

```
source.revision → normalize.document → chunk.document
chunk.document → representation.chunk → embedding.representation
embedding.representation → vector.index
chunk.document → lexical.index
```

A candidate patch has direct targets T . The core computes least transitive closure:

$$T^{\ell} = \mu X. T \cup succ(X).$$

Only artifacts whose dependency labels intersect T^6 are semantically invalid. This is a conservative statement: plugins may overdeclare dependencies and lose reuse, but they may not underdeclare without violating conformance.

A stronger implementation can index dependencies by logical item, not merely stage kind. If one source document changes, the impact plan can identify its chunks, representations, vectors, lexical records, and evaluation cases. The categorical doctrine remains the same; the dependency object becomes a finite relation or provenance semiring rather than a set of labels.

40.4 Incremental indexing as a change-action plugin

For evolving corpora, a stage can expose an incremental action. Let X carry changes ΔX and Y carry changes ΔY . An incremental implementation is:

$$Df : X \otimes \Delta X \rightarrow \Delta Y$$

satisfying the change-action law:

$$f(x \oplus \delta x) = f(x) \oplus Df(x, \delta x).$$

The kernel should not require every plugin to implement Df . It can always recompute f on the affected partition. A plugin that supplies an incremental interpreter must pass a differential/full-recompute conformance suite.

This is another reason plugins register generators and algebras rather than callbacks. The same logical generator may have:

- a full execution interpretation;
- an incremental execution interpretation;
- a cost interpretation;
- a provenance interpretation;
- a verification interpretation.

40.5 Physical backend capabilities

An index backend plugin needs more than `Search`. Its manifest should declare a capability algebra:

```
type IndexCapabilities struct {
    FullBuild      bool
    Upsert         bool
    Delete         bool
    SnapshotRead   bool
    FilterPushdown bool
    ExactScores    bool
    DeterministicBuild bool
    Compact        bool
}
```

A plan precondition can require, for example, filter pushdown before remote reranking. A candidate changing the backend is admissible only if the target plan's required capabilities are a subset of those declared by the plugin.

Capability declarations are claims. Conformance tests and certification artifacts provide evidence. The registry can attach a certificate digest to a plugin version and reject uncertified implementations in a production profile.

40.6 Indexing objectives

Indexing optimization must evaluate more than retrieval relevance. A candidate can be ordered over:

- corpus coverage and admission correctness;
- clean-build and incremental equivalence;
- update amplification;
- build latency and throughput;
- provider calls and monetary cost;
- index bytes and memory;
- exact or approximate retrieval quality;
- deletion latency;
- snapshot-open and recovery behavior;
- source-to-active freshness.

These metrics do not naturally form one scalar. Security and integrity are hard gates. Approximation quality may be a noninferiority constraint. Build cost and freshness define a Pareto frontier among eligible candidates.

40.7 Example: chunking intervention

Suppose a release uses chunk length 800, overlap 80, contextual representation prompt p , embedding model m , lexical analyzer a , and vector backend v . A candidate changes chunk length to 600 through a lawful lens.

The direct target is `chunk.spec`. Closure includes chunks, all chunk-derived representations, embeddings, both indexes, release identity, retrieval results, answer outcomes, and session projections. It does not include source capture or normalization.

The campaign may share:

- source snapshot and normalized documents;
- workload cases and labels expressed against source spans;
- provider-independent query inputs;
- baseline/candidate paired randomness for generated representations if a coupling is defined.

It may not share:

- chunk-ID labels copied from the baseline;
- candidate embeddings or indexes;
- answer outcomes generated from baseline evidence.

This example shows why the candidate's optic and dependency declaration are part of causal validity rather than implementation metadata.

41. Query optimization as interpreted evidence flow

41.1 Query plans

A query plan is a morphism whose input includes a release-pinned query context and whose output includes both user-facing outcome and trace:

$$Q_\theta: U \otimes C \otimes X \rightarrow O \otimes T.$$

U is subject/authorization context, C conversation or query state, X the request, O the outcome, and T a trace object. For stochastic stages the executable meaning is a Markov kernel.

A useful fine-grained plan is:

```
query
  → rewrite/route
  → lexical ⊗ vector ⊗ structured/connected channels
  → collapse
  → authorize
  → fuse
  → optional remote rerank
  → hydrate/admit evidence
  → answer or agent continuation
  → validate
  → presentation projection
```

The precise order is product policy. The type/effect system should prevent unsafe orders.

41.2 Authorization as type and effect

Let `CandidateSet` be untrusted with respect to subject policy and `AuthorizedSet[p]` carry a proof under policy identity p . A remote reranker generator has domain:

$$AuthorizedSet[p] \otimes HydratedEvidence[p] \rightarrow Ranking.$$

It cannot be connected to a raw candidate set by type alone. The authorization operation consumes subject, release, policy, and candidate metadata and emits both the filtered set and a certificate.

An effect interpreter additionally tracks data classes and trust boundaries. It rejects a plan if source text with class d reaches provider r without a policy relation $d \leq allowed(r)$. This is an example of a static interpretation whose algebra combines annotations over sequence and tensor.

41.3 Rewrite and route plugins

Query rewriting can be deterministic, stochastic, or stateful. A synonym expander is a pure morphism. HyDE generation is a stochastic kernel with provider effects. An intent router may depend on conversation state and may produce a route distribution.

The plan should distinguish alternatives from parallel composition. If only one route is selected, the semantics is a coproduct/choice or controlled morphism, not tensor. The minimal sandbox omits general branching to keep the kernel small. A production extension can add an explicit sum type and case-analysis constructor, preserving free structure. It should not encode branching inside opaque plugin code when route analysis or alternate interpretation matters.

41.4 Fusion and reranking laws

A fusion plugin should declare:

- channel identities and deterministic order;
- finite-score validation;
- collapse identity;
- tie-breaking total order;
- treatment of missing channels;
- monotonicity or invariance claims where applicable.

For weighted reciprocal-rank fusion:

$$RRF(d) = \sum_c \frac{w_c}{k + r_c(d)}.$$

The deterministic reference algebra can use rational arithmetic for tests, while production uses validated floating-point values and stable tie-breaks.

A reranker plugin declares its input text composition, pool depth, model identity, fallback, and whether its scores replace or blend with prior rank. Failure must produce an explicit outcome branch. A fail-open reranker and a fail-closed reranker are different plan semantics even when provider success is common.

41.5 Context and answer policy

Context admission is a constrained selection morphism. Its parameter object can include evidence count, token/rune budget, source diversity, authority ordering, and recency. Its output should retain rejected-candidate reasons so optimization can distinguish retrieval miss from admission miss.

Answer generation is usually stochastic and expensive. The evaluation kernel should preserve:

- input release and evidence identities;
- model/prompt/decoder identity;
- provider attempts and fallback;
- token/cost/latency observations;
- raw retained response where policy permits;
- contract validation and repair trajectory;
- final answer or abstention.

A candidate changing only fusion may reuse channel rankings but must regenerate downstream answer outcomes if evidence changes. A candidate changing only answer wording may reuse context artifacts. The dependency graph determines this boundary.

41.6 Agentic query semantics

For an agent, one turn is an open dynamical system. State includes conversation, tool budget, evidence ledger, and release lease. One step consumes a model/tool event and emits observations:

$$\alpha : S \rightarrow D(O \times S + F).$$

Optimization can target route descriptions, system prompts, allowed tools, iteration limits, evidence-reuse rules, or model profiles. The evaluation unit is the complete trajectory, not one tool call. Exact trial coordinates bind the initial conversation snapshot and environment. Tool-call IDs and side effects require idempotency semantics.

Low-level search operations may still be visible as plan nodes inside the agent tool. The high-level runner retains the complete trajectory as the native artifact.

41.7 Query objectives and strata

Relevant dimensions include:

- authorized recall, MRR, nDCG, and authority selection;
- evidence diversity and duplicate collapse;
- grounding and citation validity;
- answer completeness and abstention;
- tool-call count and trajectory length;
- fallback and degradation rate;
- remote disclosure volume;
- p50/p95/p99 latency;
- provider calls, tokens, and cost;
- frontend terminal delivery and widget validity.

The workload should be stratified by subject, route, query class, corpus density, freshness, and expected source authority. A global mean cannot protect a restricted administrative stratum or a rare unsupported-question class.

42. Joint index/query spaces and multi-fidelity experiments

42.1 Why independent tuning is insufficient

Index and query parameters interact. Chunk length affects the number and semantic granularity of retrieval units, which changes appropriate channel depth, rerank pool, and context budget. An embedding model changes vector score distribution and the fusion weight that works well. ANN approximation can be masked or amplified by lexical fusion. A generated representation improves discovery but may increase duplicate collapse and index cost.

The joint system is:

$$\theta = (\theta_I, \theta_Q, \theta_A, \theta_S),$$

where indexing, query, answer/agent, and serving parameters compose. Exhaustive evaluation of the product space is infeasible. The backbone therefore needs factorized proposals and multi-fidelity evidence, not a monolithic grid.

42.2 Hierarchical spaces

A *Space* can expose a tree or dependent family rather than a flat map. For example:

```

backend = exact-sqlite
└ no ANN parameters
backend = hnsw
├ M
├ efConstruction
└ efSearch
reranker = disabled
└ no rerank pool/model
reranker = remote-cross-encoder
├ provider/model
├ pool
├ timeout
└ fallback

```

Dependent parameters can be modeled as sums and products in the configuration schema. Optics focus only valid branches. The proposer cannot create a candidate with `efSearch` for an exact backend or a rerank pool when reranking is disabled.

42.3 Fidelity as an indexed evaluation category

Let fidelities form a preorder:

$$F_0 \leq F_1 \leq \dots \leq F_n$$

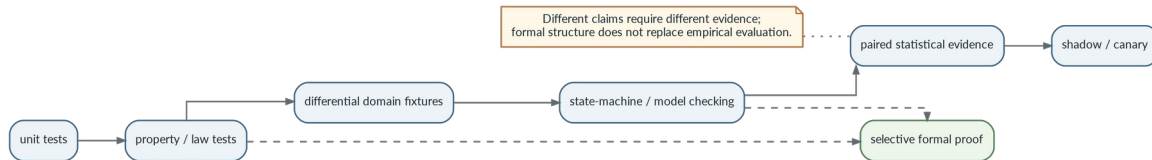
where higher levels are more expensive and observe more behavior. A typical chain is:

1. schema and static-law checks;
2. artifact/incremental equivalence;
3. retrieval benchmark;
4. repeated answer benchmark;
5. agent/session calibration;
6. refresh/load simulation;

7. production shadow;
8. canary.

An intervention has a minimum admissible fidelity based on semantic class. Lower fidelities may reject it; they cannot promote it beyond the highest changed behavior. This can be encoded as a functor from intervention classes to required evidence stages or as a policy relation.

The campaign engine in the sandbox executes one fixed workload. A production engine can model each fidelity as a subcampaign whose passing terminal event authorizes creation of the next. The event link preserves provenance across the ladder.



Assurance increases from static laws through paired trials, shadow traffic, and canary activation.

42.4 Shared artifacts without biased comparisons

Artifact reuse improves experiment efficiency but can invalidate causality when shared material depends on the candidate. The dependency interpretation decides admissible sharing.

For paired baseline/candidate evaluation:

- share immutable source snapshot, cases, labels, and environment coordinates;
- share prefix artifacts whose semantic keys are equal;
- do not share stochastic outputs downstream of changed inputs unless replay is explicitly the coupling being studied;
- record cache hits and artifact identities in each trial trace;
- preserve baseline artifacts even when a candidate fails.

The trial report should state the common prefix and divergent suffix. This makes the comparison inspectable as a commuting diagram rather than an opaque pair of metrics.

42.5 Adaptive proposers

A Bayesian optimizer, evolutionary search, gradient method, or language-model proposer is a plugin over campaign history. It receives a read-only observation view and emits a candidate patch plus hypothesis. It does not write events or decide eligibility.

An adaptive campaign cannot fix the complete candidate set in its initial identity. Its identity instead includes proposer implementation, prior/base release, space, workload, policy, budget, and deterministic random root. Candidate proposal events become part of the history. Resume must reproduce the next proposal from the same event prefix or retain the proposal as material.

Selection bias is explicit. Candidates generated after seeing development results require a hidden holdout or nested evaluation before promotion. The campaign protocol should distinguish development, selection, and confirmation workloads.

42.6 Gradient-based extensions

When a component exposes differentiable structure, the Para doctrine and reverse-derivative categories provide a compositional account of learning. A parameterized morphism $f : P \otimes A \rightarrow B$ can be paired with a reverse derivative that propagates cotangents to parameters and inputs. Composition then induces backpropagation structurally.

This is valuable for learned fusion, differentiable retrieval surrogates, or trainable rerankers, but it is not the universal optimizer. Many RAG interventions are discrete, constrained, stateful, expensive, or non-differentiable. The common field should admit gradient-based proposers as one model while retaining the same candidate, trial, artifact, and decision semantics.

42.7 Multi-objective decision

Let each candidate report vector $m(c) \in R^d$ with directions normalized so higher is better. Eligibility is:

$$eligible(c) = \bigwedge_{j \in H} g_j(c),$$

for hard gates H . Among eligible candidates, dominance is:

$$c_1 > c_2 \Leftrightarrow \forall i m_i(c_1) \geq m_i(c_2) \wedge \exists i m_i(c_1) > m_i(c_2).$$

The Pareto front retains candidates not dominated. Product policy may then choose by a lexicographic preference, budget, or human review. The kernel should never infer that one percentage point of recall is worth a particular dollar or millisecond without an explicit policy artifact.

43. From offline campaign to production release

43.1 Promotion is a morphism with authority

An optimization report does not deploy itself. Promotion consumes an eligible candidate release, a gate report, an expected active release, an actor/capability, and rollout policy:

$PromotionInput := EligibleRelease \otimes GateReport \otimes Authority \otimes ExpectedHead$,
and

$$promote: PromotionInput \rightarrow ActivationEvent + Conflict.$$

This operation belongs to the product release layer. The optimization kernel creates evidence and a promotion proposal. Keeping activation outside the campaign prevents an automated proposer from acquiring deployment authority merely by producing metrics.

43.2 Shadow as an interpreter

A production shadow runner is a high-level graft. It receives a real request envelope, executes the candidate release under the same authorization and provider policy, suppresses customer-visible effects, and stores a native trace. It can share deterministic request input with the incumbent, but it must not duplicate side-effecting tools.

The shadow interpreter declares which effects are disabled, simulated, or redirected. Static plan analysis can reject a candidate with an unshadowable side effect. Metrics compare evidence, answer, latency, cost, and failure under real traffic strata.

43.3 Canary as controlled routing

Canary evaluation adds a routing policy and online stopping rule. One semantic model is a Markov decision process whose state includes cumulative evidence, operational health, and cohort allocation. Actions increase, hold, reduce, or roll back traffic. Hard safety signals have immediate rollback authority.

The campaign event stream should retain:

- routing allocation and eligibility rules;
- request/case sampling identity;
- release leases for incumbent and candidate;
- metric windows and statistical procedure;
- every stop/continue decision;
- activation or rollback event reference.

Repeated peeking requires sequentially valid statistical methods or conservative predefined thresholds. A plain fixed-sample p -value recomputed after every request does not preserve its nominal error rate.

43.4 Release identity and campaign identity

A candidate configuration is not necessarily a loadable release. Build/materialization creates a release manifest. The campaign links:

```

candidate patch
→ materialized release
→ trial outcomes
→ comparison report
→ eligibility decision
→ promotion proposal
→ activation event

```

Each arrow is an immutable reference. A release can participate in multiple campaigns; a campaign can evaluate multiple materializations if stochastic build material is part of the experiment. The identity scheme must state whether generated representations or model outputs are fixed material or resampled trial effects.

43.5 Continuous corpus change

Optimization and content refresh are different interventions. During an optimization campaign, the source snapshot should normally be fixed to preserve causal comparison. In production, the active corpus changes. The candidate policy can be rebased onto a newer source snapshot only through a new materialized release and at least a refresh/regression validation.

A useful two-dimensional release lattice is:

		policy/config generation		
		c0	c1	c2
source	s0	R00	R01	R02
source	s1	R10	R11	R12
source	s2	R20	R21	R22

Content refresh moves vertically; optimization moves horizontally. A promotion report from R_{01} over R_{00} does not automatically justify R_{21} over R_{20} . The architecture can reuse evidence where assumptions hold, but it must represent the rebase.

43.6 Runtime observations feed future campaigns

Production traces are observations, not direct objective truth. They can generate new workload cases, incident strata, or proposer hypotheses. A mining plugin must retain sampling and privacy policy. User clicks, follow-ups, and abandonment are confounded by presentation and population shifts; they should not be treated as unbiased relevance labels.

The event-sourced campaign boundary helps prevent accidental leakage. Production data becomes a versioned workload artifact, and candidate generation begins from a declared observation cutoff.

44. Applied grafts: GEC, RAG-TTC, and Garden

44.1 GEC

The immediate coarse graft is a [Runner](#) around the existing knowledge service and answer judge. Its native artifact should include authorized candidate sets, channel ranks, synonym expansion, reranker request identity, fallback, admitted evidence, answer, judge dimensions, latency, and usage.

The [Space](#) can initially expose:

- RRF rank constant and channel weights;
- lexical/vector depths;
- synonym asset revision;
- rerank pool, blend, timeout, and model;
- context budget;
- answer model/prompt under separate campaigns.

Authorization policy is locked and evaluated as a hard law. Before any relevance experiment, the query plan must be corrected so authorization dominates remote reranking. A provider-spy law then becomes part of plugin admission.

Fine-grained registration can follow: GEC lexical expansion, weighted RRF, authorization, reranker composition, and evidence admission become visible operations. The GEC product package retains scope/role meaning and tool/UI projection.

44.2 RAG-TTC

RAG-TTC is the strongest candidate for full plan adoption because it already exposes indexing stages, caches, ANN experiments, search routes, tool loops, and native artifacts. The first migration removes its copied common substrate and uses [ragkit](#) as the single RAG implementation.

Low-level plugins can cover committed-Git capture, chunk/representation/embedding stages, exact and ANN indexes, route-specific retrieval, connected augmentation, and answer validation. High-level grafts retain complete tool-loop and conversation semantics.

The parameter dependency graph should distinguish:

- source/admission and normalization;
- chunk and representation generation;
- embedding and index backend;
- route/channel/fusion/rerank;
- answer/tool configuration;
- session/runtime policy.

Current ANN bakeoff becomes a specialized campaign whose hard gates include deletion/filter soundness, exact-oracle recall, update-sequence behavior, memory, latency, reopen, and compaction. Current tool evaluation becomes a high-fidelity campaign linked to the same candidate release.

44.3 Garden

Garden’s optimization unit is frequently a multi-turn user outcome. A coarse runner should create an isolated conversation, execute prompts and selections, retain every terminal snapshot and widget event, and project metrics only after native expectations are evaluated.

Its space can include intent routing, structured-first versus retrieve-first policy, connected retrieval, tool descriptions, context/evidence budgets, product-fact augmentation, widget conflict policy, and answer profile. Product fact schemas and widget field meaning remain Garden-owned.

The low-level surface is useful for shared retrieval and evidence operations. Presentation remains a product interpreter. A widget payload is eligible only when every customer-visible field is linked to admitted evidence under the trial’s release. This can be expressed as a law over the native artifact and enforced before quality scoring.

44.4 Cross-product reuse

The shared kernel can reuse:

- typed candidate and trial coordinates;
- dependency and artifact semantics;
- metric direction and missingness;
- exact paired comparison;
- event-sourced campaign state;
- gate sequencing and Pareto analysis;
- release/promotion references;
- plugin manifests and conformance results.

It should not reuse by force:

- GEC access roles and judge rubric;
- TTC connected-retrieval routes;
- Garden product facts, choices, or widget schemas;
- product-specific prompts and human review policy.

The architecture succeeds when these product domains can remain different while their optimization evidence composes through the same backbone.

45. Migration roadmap

45.1 Phase 0: freeze empirical behavior

Capture golden and differential fixtures for current indexing, retrieval, answer, agent, and frontend paths. Record identities, ranks, evidence, failures, native artifacts, and current experiment decisions. Classify intentional defects such as unauthorized pre-rerank disclosure separately from compatibility requirements.

45.2 Phase 1: adopt coarse campaign ports

Wrap one current RAG-TTC tool evaluation, one GEC retrieval suite, and one Garden calibration suite as high-level runners. Move exact coordinates, paired comparison, missingness, gates, and event custody to the shared campaign protocol. Do not change domain execution yet.

This phase tests domain neutrality and produces immediate reproducibility improvements with low migration risk.

45.3 Phase 2: introduce typed spaces and optics

Replace stringly parameter maps with versioned configuration schemas and lawful lenses. Generate candidate patches as immutable values with hypotheses and semantic classes. Add dependency labels conservatively. Continue using existing build/query code behind runners.

Acceptance requires lens-law tests, schema compatibility checks, and explicit invalidation reports for every candidate.

45.4 Phase 3: register high-value primitive operations

Start with operations where alternate interpretation pays for the abstraction:

- chunking/representation/embedding and index build;
- lexical/vector channel search;
- collapse/fusion;
- authorization and remote rerank;
- evidence admission and contract validation.

Create execution and static-analysis algebras. Compare plan execution with legacy paths using differential fixtures. Avoid translating entire chat frameworks into the plan language.

45.5 Phase 4: semantic artifact keys and build reuse

Derive cache and artifact keys from plan prefixes, parameters, and inputs. Replace handwritten build-relevance checks. Add full versus incremental equivalence tests. Record common-prefix/divergent-suffix diagrams in experiment reports.

45.6 Phase 5: multi-fidelity campaigns

Link static laws, retrieval, answers, sessions, refresh/load, shadow, and canary as staged evidence. Add confirmation holdouts and statistical uncertainty. Introduce adaptive proposers only after event semantics for proposal history are stable.

45.7 Phase 6: release integration

Materialize candidate patches into behavior-complete releases. Connect promotion reports to compare-and-swap activation without granting campaign code deployment authority. Add shadow and canary runners with release-pinned traces.

45.8 Phase 7: distributed refinement

Replace local event/artifact stores and sequential engine with distributed implementations that refine the same protocol. Preserve stable coordinates, idempotent commands, append-only events, fencing, and terminal immutability. Verify linearizability of the critical store/activation boundaries.

45.9 Exit criteria

The field is successfully established when:

- two unrelated domains and all three RAG products use the same campaign semantics;
- at least two interpreters consume one fine-grained plan;
- candidate dependency closure determines artifact reuse;
- missing or failed cells cannot disappear from comparison;
- plugin registration is transactional and law-gated;
- production promotion resolves to exact release and report identities;
- no product must put its domain semantics into the kernel to participate.

Part VI. Assurance, extensions, and research program

46. Structural results and proof obligations

46.1 Free-extension result

Let Σ be a typed signature and $W(\Sigma)$ the free strict symmetric monoidal category with explicit copy and discard maps admitted by the chosen wiring doctrine. Let D be a target category carrying the corresponding structure. A generator interpretation J assigns every object of Σ to an object of D and every primitive generator to a morphism with matching domain and codomain.

By freeness, there is a unique structure-preserving functor

$$\hat{J}: W(\Sigma) \rightarrow D$$

extending J . This is the mathematical basis of `plan.Fold`: once an interpreter supplies meanings for primitives and structural constructors, every valid plan has one induced meaning.

Now extend the signature to $\Sigma' = \Sigma + \Delta$. Any interpretation J' that agrees with J on Σ induces $\hat{J}'$. For every old plan $p \in W(\Sigma)$,

$$\hat{J}'(p) = \hat{J}(p).$$

This is the noninterference property desired from operation plugins. New generators do not change old meanings. The software proof depends on stable canonical descriptors and the absence of mutable global registry behavior during a campaign.

46.2 Normalization and identity

The sandbox normalizes nested sequence and tensor nodes before hashing. Desired laws are:

$$(f;g);h \equiv f;(g;h),$$

$$(f \otimes g) \otimes h \equiv f \otimes (g \otimes h),$$

$$id;f \equiv f \equiv f;id,$$

and corresponding unit laws for tensor. In a strict presentation, associators and unitors are erased by normalization. Permutations retain explicit identity because port order is semantically visible.

The proof obligation is that normalization is terminating, idempotent, and sound with respect to every interpreter:

$$N(N(p)) = N(p)$$

and

$$\hat{J}(N(p)) = \hat{J}(p).$$

The sandbox tests representative laws. A production kernel should use property tests over generated well-typed plans and a small mechanized proof or exhaustive finite model for the normalization rewrite system.

46.3 Type preservation

Let $p: A \rightarrow B$ be a validated plan and let an execution environment provide an envelope matching A . If every primitive implementation satisfies its declared input/output codecs, then execution either returns an attributable failure outcome or an envelope matching B .

This is a progress-and-preservation style property. It is conditional on plugin refinement. The kernel checks structural port compatibility and envelope schema/digest validity. Typed adapters discharge ordinary decode/encode boundaries. A malicious or defective plugin can still lie about its output; post-execution validation turns that lie into infrastructure failure rather than allowing it to contaminate downstream operations.

46.4 Tensor independence

For pure deterministic operations $f: A \rightarrow B$ and $g: C \rightarrow D$, the tensor execution should be observationally equivalent to independent execution:

$$\llbracket f \otimes g \rrbracket(a, c) = (\llbracket f \rrbracket(a), \llbracket g \rrbracket(c)).$$

With effects, equality is weakened. A concurrent interpreter may interleave observations and resource use. It must preserve branch-local inputs, outputs, artifacts, and protected effects. If two branches contend for a shared provider quota, their joint latency distribution may not factor. Resource analysis and interpreter refinement must state this explicitly.

The core therefore distinguishes syntactic independence from operational independence. Tensor permits parallel interpretation; it does not promise absence of shared external resources.

46.5 Para associativity

Given parameterized maps

$$f: P \otimes A \rightarrow B, g: Q \otimes B \rightarrow C, h: R \otimes C \rightarrow D,$$

the two ways of composing produce parameter objects isomorphic to $P \otimes Q \otimes R$ and equal behavior up to associators and symmetries. In the sandbox, nested Go product structs expose associativity at the representation level; helper functions construct a canonical pairing convention.

A production schema should canonicalize parameter products or name component fields so that associator choices do not create accidental candidate identities. The mathematical category treats them as coherent isomorphisms; serialized software must choose one normal form.

46.6 Optic law and patch determinacy

For a lawful total lens $L: S \multimap A$, the three lens laws imply that applying a patch to focus A has history-independent last-write semantics. If $\text{Patch}(L, a)$ denotes the endomorphism $s \mapsto \text{put}(s, a)$, then:

$$\text{Patch}(L, a); \text{Patch}(L, b) = \text{Patch}(L, b).$$

For two disjoint commuting lenses L_A and L_B :

$$\text{Patch}(L_A, a); \text{Patch}(L_B, b) = \text{Patch}(L_B, b); \text{Patch}(L_A, a).$$

Commutativity is not automatic for overlapping or constrained optics. A multi-patch candidate should carry either an order or a commutation witness. The sandbox restricts candidates to one patch, matching the supplied `ragopt` discipline and making causal interpretation clearer.

46.7 Dependency-closure soundness

Let $G=(V, E)$ be the dependency graph and $T \subseteq V$ direct targets. Let $C = reach_G(T)$. An artifact labeled by node $v \notin C$ is eligible for reuse only if its semantic key excludes every changed value and every transitive output of changed nodes.

The desired soundness theorem is:

If plugin dependency declarations are complete and semantic keys are compositional, then reusing artifacts outside C does not change the candidate denotation.

The theorem is conditional because dependency completeness is domain knowledge. Conformance can test it through mutation analysis: change each advertised parameter, execute both fresh and reuse paths, and compare outputs. Provenance traces can also detect undeclared reads by instrumenting configuration access in development builds.

46.8 Coupling unbiasedness

Suppose baseline and candidate outcomes have marginals $K_b(x)$ and $K_c(x)$. Any coupling Γ_x with these marginals gives an unbiased paired estimator of mean difference:

$$E_{(Y_b, Y_c) \sim \Gamma_x} [m(Y_c) - m(Y_b)] = E_{K_c(x)}[m] - E_{K_b(x)}[m].$$

The coupling changes variance, not expectation. Common random numbers can reduce variance when responses are positively correlated. They can increase variance otherwise. Pairing is therefore a structural requirement for exact coordinates, while the choice of coupling is a statistical design decision that must be retained in campaign identity.

Provider APIs may not expose stable seeds. A material replay coupling can hold upstream provider output fixed to isolate downstream changes, but it answers a conditional causal question rather than the full live-distribution question. Campaign reports must state which coupling was used.

46.9 Failure-preserving comparison

Let each coordinate produce a total trial result in a sum type:

$$R = Success(M, A) + DomainFail(F) + InfraFail(F) + Cancelled(F).$$

Comparison is a total function over pairs of R . It never projects only successful metric maps before checking coverage. A hard coverage gate can require all intended coordinates, and a success gate can separately bound failure classes.

This design prevents survivorship bias where a candidate appears strong because hard cases failed and disappeared. It also prevents infrastructure outages from being silently interpreted as low metric values.

46.10 Gate monotonicity

A lexicographic gate sequence evaluates $g_1, \dots, g_n$ and returns on first fail or indeterminate. If a new hard gate is prepended, a previously ineligible candidate cannot become eligible. If a stricter version of an earlier predicate replaces it, later favorable metrics cannot compensate.

This monotonicity is desirable for security and integrity policy. It differs from weighted scoring, where adding a penalty can be offset by unrelated gains.

46.11 Event-reducer safety

The campaign reducer should satisfy:

1. deterministic reduction;
2. prefix validity;
3. terminal immutability;
4. coordinate uniqueness;
5. causal order of report and decision;
6. campaign identity consistency.

For valid histories P and U :

$$\rho^i(s_0, P \cdot U) = \rho^i(\rho^i(s_0, P), U).$$

The store and command engine add operational obligations: append atomicity, expected sequence, fencing in distributed execution, and idempotent command selection. The sandbox proves the reducer laws through tests and demonstrates resume under an injected append failure. It does not prove distributed linearizability.

46.12 Plugin extension theorem in software form

A practical theorem schema for each plugin version is:

- registration is atomic;
- all schema and operation IDs are unique;
- every operation's port schemas exist;
- codecs round-trip a conformance corpus;
- static descriptor is canonical and immutable;
- implementation outputs validate against the descriptor;
- declared laws pass their certificate suite;
- existing plans retain IDs and interpreter outputs after registration.

The last property can be tested by snapshotting old registry descriptors and running old golden plans before and after plugin addition. This is the executable counterpart of signature-extension conservativity.

47. Verification strategy

47.1 Assurance ladder

Different claims require different methods:

Claim	Appropriate evidence
schema and port compatibility	compile/validation tests
canonical identity	golden vectors and property tests
sequence/tensor normalization	property tests and proof of rewrite system
plugin implementation refinement	differential and conformance tests
dependency closure	mutation tests and provenance instrumentation
incremental/full equivalence	generated change-sequence tests
campaign reducer safety	state-machine tests and model checking
stochastic quality	paired statistical evaluation
distributed store semantics	linearizability/fault-injection tests
production utility	shadow/canary evidence and human review

No single formalism establishes all rows. The backbone is valuable because it localizes the claim boundaries.

47.2 Property-based plan generation

Generate schemas and well-typed primitive descriptors, then recursively generate plans using identity, sequence, tensor, permutation, copy, and drop. Check:

- validation succeeds for generated plans;
- normalized IDs are invariant under reassociation and units;
- every interpreter fold terminates;
- execution reference and an alternate interpreter agree under the stated observation relation;
- malformed permutations and port mismatches fail;
- unknown node kinds and operation IDs fail closed.

Shrinking should preserve type correctness so counterexamples remain intelligible.

47.3 Plugin conformance harness

A plugin package should export a manifest and a test factory. The host supplies generic tests:

- descriptor canonicalization;
- codec round-trip and invalid-payload rejection;
- operation determinism where declared;
- cache-key sensitivity to semantic inputs;
- no undeclared effect under an instrumented environment;
- law checks over generated fixtures;

- cancellation and deadline behavior;
- output schema validation;
- panic containment or process isolation.

Domain-specific tests augment the generic harness. An ANN plugin receives an exact oracle suite; an authorization plugin receives adversarial scope cases; a Garden projection plugin receives provenance fixtures.

47.4 Metamorphic optimization tests

Optimization software has useful metamorphic properties:

- adding an irrelevant metric must not change earlier hard-gate results;
- permuting candidate enumeration must not change reports or deterministic selection;
- permuting case execution order must not change terminal state;
- increasing repeats adds coordinates without changing existing coordinate identity;
- resuming after any event prefix yields the same terminal state as uninterrupted execution;
- adding a plugin unused by the campaign must not change campaign identity;
- changing a runner, workload, coupling, metric definition, or policy must change campaign identity.

These tests catch hidden dependence on map order, wall clock, or mutable registries.

47.5 Statistical conformance

A stochastic runner can be tested against known kernels. Generate Bernoulli, Gaussian-like finite, or quadratic objectives where expected differences are known. Verify:

- seed splitting produces stable distinct substreams;
- paired reports match hand calculations;
- missing cells and failures remain visible;
- confidence intervals have approximate coverage under simulation;
- sequential policies control the intended error rate under their assumptions;
- Pareto front and metric orientation are correct.

The sandbox includes finite distributions and paired deterministic noise but not a statistical inference package. That should be an extension module rather than part of the identity/custody kernel.

47.6 Model checking campaign protocols

A bounded TLA+ or explicit-state model can include:

```
candidates, cells, reports, decisions, terminal,
commands, events, workerLeases, appendPosition
```

Actions register, schedule, complete, compare, decide, cancel, fail append, resume, and complete. Safety invariants include uniqueness and causal order. Liveness can state that under fair scheduling and a functioning store, every finite campaign eventually terminates.

A distributed refinement adds worker claims and fencing. The model should explore duplicate completions, stale workers, coordinator failover, and concurrent compare commands.

47.7 RAG-specific differential tests

For each candidate class:

- execute the legacy application path;
- execute the plan/graft path with identical release and retained provider material;
- compare ranked evidence, contribution traces, authorization decisions, context, answer contract, native artifact, and frontend projection under an explicit normalization;
- classify intended changes.

Security corrections should deliberately change disclosure traces. Deterministic tie-order improvements may change ranks only for equal scores. Identity-epoch changes should not be hidden as compatibility failures.

47.8 Race, load, and failure testing

The reference module passes Go's race detector, but production assurance also needs:

- concurrent registry reads after freeze;
- campaign workers completing the same coordinate;
- event append conflicts and coordinator failover;
- artifact write interruption and verification;
- provider timeout and retry storms;
- build/query resource contention;
- canary stop while trials are in flight;
- plugin process crash or malformed response;
- release revocation during a shadow trial.

The semantic outcome for each failure must be specified before injecting it.

47.9 Reproducibility levels

A campaign can declare one of several reproducibility classes:

1. **Structural:** same plans, candidates, coordinates, and policies.
2. **Material:** all provider and derived outputs retained; exact replay is possible.
3. **Seeded:** deterministic software and providers under recorded seeds/environment.
4. **Distributional:** only the stochastic kernel/model version is identified; repeated statistics should agree.
5. **Observational:** production environment cannot be replayed; retained trace supports audit only.

Reports should not claim exact reproducibility when only distributional identity exists.

48. Plugin security and isolation

48.1 Plugins are executable supply-chain inputs

A plugin can read data, invoke networks, consume resources, forge metrics, or corrupt process state. Type descriptors do not make code safe. The architecture separates semantic extensibility from execution trust.

A production manifest should contain:

- plugin ID, semantic version, build/source digest;
- publisher/signature and review status;
- supported kernel/schema versions;
- generators and codecs;
- declared effects, data classes, endpoints, and resource bounds;
- law/conformance certificate references;
- required secrets and capabilities;
- process/runtime isolation profile.

Campaign and release identities bind the exact plugin digest, not merely a friendly name.

48.2 Registry freeze

Registration occurs through a builder transaction. Once committed, a registry snapshot is immutable. A campaign captures its registry identity. Dynamic registration during execution would make plan resolution time-dependent and could change old plan meanings.

Hot plugin upgrades therefore create a new registry/release generation. In-flight campaigns continue under the old snapshot or stop with an explicit incompatibility event.

48.3 Capability-based execution

Rather than handing a plugin a general context with filesystem, network, secrets, and clocks, the host supplies narrow capabilities:

```
type Capabilities struct {
  Artifacts artifact.Client
  Clock      clock.Reader
  Random     prob.Source
  Network    network.PolicyClient
  Secrets    secret.ScopedReader
  Observe    trace.Sink
}
```

The manifest determines which fields are populated. Network clients enforce endpoint and data-policy constraints. Artifact clients constrain namespaces. A deterministic test interpreter supplies virtual clock and seeded randomness.

In-process Go cannot reliably sandbox malicious code. Untrusted or high-risk plugins should run in a separate process, container, WebAssembly runtime, or remote service with authenticated typed RPC.

48.4 Typed RPC boundary

A process-isolated operation protocol contains:

- registry/plugin/operation identity;
- request/campaign/trial coordinate;
- input envelope references;
- deadline and idempotency key;
- granted capability tokens;
- output envelope and artifact references;
- declared observations and usage;
- terminal status and attributable failure.

The host validates all output as if the process were untrusted. Large artifacts move through content-addressed storage, not arbitrary paths. The RPC protocol is an interpreter of the same primitive signature, so process isolation does not change plan semantics.

48.5 Metric and artifact fraud

A domain runner controls native measurement and can lie. The kernel cannot infer truth from a metric map. Trust can be strengthened by:

- evaluator separation from candidate implementation;
- immutable raw/native artifacts;
- independent recomputation of generic metrics;
- hidden cases and policy tests;
- signed provider/request logs;
- deterministic reference implementations;
- human inspection for promotion.

The campaign records who produced each artifact and metric. It does not elevate plugin output to unquestioned fact.

48.6 Data minimization

Plans and traces should carry artifact references and digests by default, not duplicate corpus/query text. Effect analysis can calculate disclosure sets. Product policy determines retention and redaction.

A plugin manifest declares whether it needs raw source, normalized text, metadata only, vectors, or aggregate metrics. The host can reject an unnecessarily broad requirement or insert a projection operation before the plugin.

48.7 Denial-of-service and quotas

Static resource hints are advisory. Runtime enforcement uses deadlines, memory/process limits, provider quotas, artifact size limits, and output cardinality bounds. A plugin that exceeds its contract yields an infrastructure failure and may be quarantined.

A proposer also requires limits. It cannot emit an unbounded candidate stream or recursively create campaigns without a budget. Candidate-count and materialization budgets are campaign policy.

48.8 Schema evolution attacks

A plugin should not reinterpret an existing schema ID with different meaning. Schema IDs are immutable and content/domain separated. Evolution creates a new ID and an explicit migration operation. The registry rejects duplicate IDs with unequal descriptors.

Unknown fields and forward compatibility must be specified per schema. Silent field dropping can invalidate candidate identity or security policy.

49. Distributed refinement

49.1 Separation of semantic and physical graphs

The plan is a semantic graph. A scheduler may split, batch, fuse, place, retry, or parallelize its operations. These transformations form a physical execution plan that must refine the semantic plan.

Examples:

- batching many `embed` morphisms into one provider request;
- fusing decode/encode boundaries inside one process;
- executing tensor branches concurrently;
- scheduling build stages on remote workers;
- replaying a cached artifact instead of invoking the operation.

Each optimization needs a refinement witness or conformance test preserving protected observations.

49.2 Command protocol

Workers should receive idempotent commands keyed by semantic coordinate:

```
type TrialCommand struct {
    CampaignID core.Digest
    CandidateID core.Digest
    CaseID     core.Digest
    Repeat     int
    Arm        experiment.Arm
    Seed       uint64
    RunnerID   string
    Deadline   time.Time
    Fence      uint64
}
```

A completion event is accepted only once for the key and current fence. Duplicate attempts can write identical content-addressed artifacts, but only one terminal cell enters campaign state.

49.3 Store linearizability

The campaign event stream requires an append operation with expected next sequence or an equivalent compare-and-swap. In a partition, two coordinators may propose the same next event. At most one succeeds. The loser reloads and recomputes commands.

Snapshots are derived caches and never authority. An invalid or stale snapshot is discarded and events replayed.

49.4 Exactly-once effect is local, not global

It is unnecessary and usually impossible to guarantee exactly-once execution of every trial. The protocol guarantees at-most-one accepted terminal result per semantic coordinate. Provider calls and work may repeat. Side-effecting domain operations must be idempotent, simulated, or excluded from trials.

Promotion/activation is a separate compare-and-swap effect with an idempotency key. This is the only exactly-once semantic effect required at the release head.

49.5 Deterministic scheduling identity

Execution order is normally operational identity, not campaign semantic identity, because exact coordinates make reports order-independent. It becomes semantic when time/environment drift can affect outcomes. A paired interleaving schedule can then be part of the coupling policy:

```
case 1 repeat 1: baseline, candidate
case 1 repeat 2: candidate, baseline
...
```

The report records schedule and wall-clock strata. This is useful for remote provider drift.

49.6 Distributed tensor and cancellation

A tensor node can launch branches concurrently. If one fails, policy decides whether to cancel the other, retain partial artifacts, or wait for complete observations. This policy is part of the execution interpreter, not the abstract tensor.

Campaign cancellation stops scheduling new commands, requests cancellation of active work, accepts or rejects late completions under policy, and appends one terminal cancellation event. Resume after cancellation creates a new campaign unless the cancellation event is explicitly reversible, which the reference doctrine avoids.

49.7 Federated or privacy-preserving evaluation

Some workloads cannot leave a product or tenant boundary. A remote evaluator can execute trial commands and return signed aggregate/native artifact references. Exact coordinate identity and metric schema remain shared. The central campaign need not receive raw evidence.

Secure aggregation or differential privacy can be modeled as an evaluation plugin whose noise/privacy budget is part of the metric semantics and campaign identity. Privacy accounting is an ordered resource, not an annotation.

50. Limits of the doctrine

50.1 Category theory does not supply domain truth

The construction can prove that a plan is well typed, an interpreter is structurally compositional, or a campaign retains exact pairs. It cannot prove that retrieved text is relevant, a judge is calibrated, a source is authoritative, or a user is helped. Those are domain and empirical claims.

Formal structure prevents several classes of category error. It does not eliminate measurement error.

50.2 Freeness can expose too much syntax

A fully reified plan for every internal function can become verbose, brittle, and expensive to version. The correct granularity is guided by interpretation value. An operation should be a primitive when it needs independent identity, policy, reuse, observation, replacement, or alternate execution. Ordinary local code can remain inside a plugin implementation.

50.3 Effects are difficult to combine universally

Network, state, nondeterminism, exceptions, cancellation, streaming, and resource constraints do not have one trivial monad in production software. Algebraic effects and handlers provide a conceptual model, but a practical Go implementation may use explicit outcomes and capability interfaces. The doctrine requires declared composition and interpreters, not one universal effect stack.

50.4 Markov kernels idealize providers

A remote language model is not necessarily a stationary kernel identified by model name and seed. Providers change infrastructure, hidden prompts, batching, moderation, and model weights. Distributional identity is therefore an operational claim with uncertainty. Retained material and temporal strata remain necessary.

50.5 Optic laws can conflict with global constraints

A local lens is most natural for independent product fields. Configuration validity can couple distant fields. Partial optics, validated reparameterizations, or dependent spaces are required. The system should reject invalid candidates rather than weaken laws to make every path writable.

50.6 Dependency graphs are declarations

The core can compute closure exactly from the graph, but it cannot know the graph is complete. Undeclared ambient reads—environment variables, current time, mutable files, provider aliases—are common sources of unsound reuse. Capability restriction, provenance instrumentation, and mutation tests are essential.

50.7 Decision policies remain political and product-relative

Lexicographic gates make policy explicit; they do not make it neutral. Choosing a noninferiority margin, cost budget, safety threshold, or human reviewer is an organizational decision. The artifact should identify and expose that policy rather than presenting it as mathematical inevitability.

50.8 Distributed production adds failure modes

The sandbox is local and sequential. A distributed implementation must establish store consistency, fencing, artifact durability, worker identity, secret policy, and operational SLOs. The semantics guide refinement but do not supply infrastructure automatically.

50.9 Adaptive optimization complicates inference

A fixed paired campaign is statistically simpler than adaptive candidate generation, early stopping, and repeated holdout use. A generalized engine must treat proposal history, selection bias, multiplicity, and confirmation evidence explicitly. Search sophistication should not arrive before experiment custody.

51. Research program

51.1 Mechanized free-plan kernel

Formalize the wiring syntax, normalization, typing, and folds in Lean, Coq, or Agda. Extract golden normal forms and test vectors for the Go implementation. Prove conservativity under signature extension and soundness of the static effect algebra.

A modest target is preferable to proving the entire system: schemas, ports, sequence, tensor, permutation, copy, discard, and finite effect summaries.

51.2 General optics and dependent spaces

Replace the lens-only sandbox with a serializable optic language supporting products, sums, optional branches, traversals, and validated reparameterizations. Study composition with dependency declarations so that focusing a parameter can automatically derive semantic targets.

A useful result would be an optic whose residual/context object directly generates an invalidation witness.

51.3 Provenance-semiring interpretation

Interpret plans and incremental builds into provenance polynomials or semimodules. This can provide item-level explanations of which source revisions, parameters, and operations contributed to an artifact or metric. It can also compute precise invalidation and common-prefix sharing.

The challenge is keeping provenance compact for large corpora. Factorized or Merkle-linked representations are likely necessary.

51.4 Categorical stochastic campaigns

Develop an explicit Markov-category semantics for trial plans, couplings, repeated measures, missingness, and adaptive proposals. Distinguish random-variable coupling from shared material replay. Connect statistical estimators to the categorical construction without burying assumptions in implementation code.

51.5 Resource-enriched optimization

Model latency, dollars, tokens, memory, energy, privacy, and disclosure as ordered commutative monoids or enriched hom-values. Investigate when sequential/tensor resource summaries are exact, upper bounds, or empirical distributions. This can support static budget rejection and compositional capacity planning.

51.6 Open campaign composition

Treat campaigns, build coordinators, release managers, and canary controllers as open dynamical systems with compatible interfaces. Study composition of their safety properties and how event traces implement categorical wiring at runtime.

A practical question is whether an offline campaign, shadow campaign, and canary can be composed as one higher-level protocol while retaining each subsystem's durable state.

51.7 Certified plugin refinement

Define machine-readable refinement contracts between abstract generators and concrete implementations. Examples include exact/ANN retrieval relations, local/remote reranker equivalence, legacy/new query path compatibility, and simulator/production runner correspondence.

Certificates can combine proof, exhaustive finite checking, property tests, benchmark distributions, and signed human review.

51.8 Learning proposers as ordinary plugins

Implement grid, random, Bayesian, evolutionary, gradient, and language-model proposers over the same observation interface. Compare their sample efficiency only after preserving candidate identity, selection history, and confirmation holdout. This separates optimizer research from experiment-engine correctness.

51.9 Cross-language SDK

Specify canonical schemas and RPC for plugins in Go, Python, Rust, and TypeScript. Use a language-neutral canonical encoding and golden vectors. Preserve typed operation descriptors and campaign coordinates while allowing domain-native implementation.

The kernel should remain language-agnostic at the protocol level even if one trusted implementation is in Go.

51.10 RAG field benchmark

Construct a benchmark that evaluates optimization architectures rather than only retrievers. It should include:

- indexing and query interventions with known invalidation closure;
- stochastic provider stubs;
- authorization constraints;
- incremental corpus changes;
- answer and agent outcomes;
- frontend projections;
- interrupted/distributed campaign execution;
- promotion and rollback decisions.

Success would measure whether a system produces correct, reproducible evidence under changes and failures, not merely whether it finds a high score.

52. Conclusion

Optimization needs a semantics of systems before it needs a more sophisticated search loop. The central construction of this thesis is a small but strong doctrine:

- a typed signature names domain objects and primitive operations;
- a free wiring syntax owns sequence, tensor, and structural composition;
- plugins extend generators and models without extending composition rules;
- structural folds induce execution, effects, dependencies, resources, identity, and provenance;
- parameterized morphisms make configuration compose with systems;
- optics make local interventions lawful and explicit;
- Markov kernels and couplings make stochastic comparison honest;
- ordered metrics and gates preserve constraints and partial preference;
- event-sourced campaigns give optimization a durable operational semantics.

The plugin boundary follows from the mathematics. Fine-grained plugins are models of a signature and are valuable when multiple interpretations matter. Coarse domain grafts implement stable campaign ports and let existing systems participate without invasive decomposition. The core remains simple not by omitting semantics, but by owning only semantics that must be universal: composition, identity, pairing, missingness, gate order, and event validity.

The executable `opfield` sandbox demonstrates the construction in ordinary Go. It composes typed plans, interprets them in more than one way, checks lawful interventions, couples exact paired trials, resumes an interrupted campaign, and optimizes both a miniature RAG system and an unrelated stochastic quadratic objective. Its limitations are explicit: it is a reference kernel, not a production scheduler, statistical package, or proof assistant.

Applied to RAG, the doctrine turns indexing and querying into one compositional field without collapsing their distinctions. A chunking intervention changes a build prefix and invalidates downstream artifacts. A fusion intervention changes only a query suffix. An agent-policy intervention requires trajectory evaluation. A serving-policy intervention may be operational or outcome-changing depending on deadlines and fallback. Every candidate carries its causal claim, and every promotion is backed by exact release, workload, trace, and decision identities.

This architecture makes room for future methods—Bayesian search, differentiable learning, language-model proposers, incremental indexing, distributed workers, shadow traffic, and canary control—without making any one of them the definition of optimization. They become plugins and interpreters grafted onto a field whose basic laws remain stable.

The resulting research program is both formal and practical. The free-plan and reducer kernels are small enough to prove or model-check. Plugin refinement and quality remain testable and empirical. Product-specific semantics remain in the products. That division of labor is the principal result: a composable optimization architecture can be open to new domains and methods while preserving a small, reviewable semantic backbone.

Appendix A. Formal definitions and operational rules

A.1 Typed signatures

A typed operation signature is a tuple

$$\Sigma = (Ob, Gen, dom, cod, ann),$$

where:

- Ob is a set of atomic schema objects;
- Gen is a set of primitive operation symbols;
- $dom, cod: Gen \rightarrow Ob^{\dot{c}}$ assign finite ordered ports;
- ann assigns versioned semantic annotations such as effects, dependencies, determinism, cacheability, cost, and plugin identity.

Ports form the free strict monoid $Ob^{\dot{c}}$ under concatenation. The empty port is I .

A plugin extension Δ is valid relative to Σ when its schema and generator IDs are fresh, every referenced schema resolves in $\Sigma + \Delta$, descriptors validate, and all registration laws pass. The committed registry represents the coproduct signature $\Sigma + \Delta$.

A.2 Plan syntax

For ports A, B , plans are generated by:

$$\begin{array}{c}
 \frac{\square}{id_A: A \rightarrow A} \\
 \frac{g \in Gen \quad dom(g) = A \quad cod(g) = B}{prim(g): A \rightarrow B} \\
 \frac{f: A \rightarrow B \quad g: B \rightarrow C}{f; g: A \rightarrow C} \\
 \frac{f: A \rightarrow B \quad g: C \rightarrow D}{f \otimes g: A \otimes C \rightarrow B \otimes D} \\
 \frac{\pi \text{ a permutation of } \{1, \dots, |A|\}}{permute_{A, \pi}: A \rightarrow \pi A} \\
 \frac{\square \quad \square}{copy_A: A \rightarrow A \otimes A \quad drop_A: A \rightarrow I}
 \end{array}$$

The sandbox permits copy/drop for every port because it models data envelopes, not arbitrary linear resources. A stronger effect system can restrict copying and discarding of affine, secret, expensive, or stateful values.

A.3 Validation judgments

Registry judgment $R \vdash p : A \rightarrow B$ is defined recursively. Primitive validation requires that the currently registered descriptor equal the descriptor captured by the plan. This detects semantic signature drift even when an operation ID is reused incorrectly.

A plan is executable only after:

1. structural validation;
2. static policy validation;
3. input envelope validation;
4. interpreter-specific preconditions.

The first is kernel-owned. The second can inspect effects/capabilities. The third checks schemas, canonical payloads, and digests. The fourth includes resource admission or provider health.

A.4 Structural congruence

The intended congruence includes strict monoidal category laws and permutation coherence. The implementation chooses canonical sequence/tensor flattening and removes identities. It does not identify arbitrary permutation expressions unless their serialized permutation vectors coincide.

Copy/drop operations suggest a gs-monoidal or Markov-category-like wiring doctrine: data may be copied and discarded, but operations need not be natural with respect to copying. In particular, copying an input before a stochastic operation is not the same as running the stochastic operation once and copying its output. Explicit nodes preserve that distinction.

A.5 Algebra and fold

For result carrier R , an algebra consists of functions:

```
Identity : Port → R
Primitive: OperationDescriptor → R
Sequence : List[R] → R
Tensor   : List[R] → R
Permute  : Port × Permutation → R
Copy     : Port → R
Drop     : Port → R
```

Fold is the unique catamorphism induced by this algebra over validated syntax. Its equations are definitional:

$$\begin{aligned} fold_\alpha(prim(g)) &= \alpha_{prim}(g), \\ fold_\alpha(f;g) &= \alpha_{seq}(fold_\alpha(f), fold_\alpha(g)), \end{aligned}$$

with analogous equations for all constructors.

An interpreter is lawful when its algebra satisfies the intended structural equations. The kernel can test but cannot universally prove an arbitrary Go algebra lawful.

A.6 Execution configurations

A reference execution configuration is

$$\langle p, \vec{e}, \kappa \rangle,$$

where p is a plan, $\vec{e}$ an input envelope frame, and κ execution capabilities/context. A terminal result is:

$$Execution = (status, outputs, observations, artifacts, failure, duration).$$

For a primitive operation g :

$$\frac{validate(\vec{e}, dom(g)) \implies (g, \kappa, \vec{e}) = r}{\langle g, \vec{e}, \kappa \rangle \Downarrow r}.$$

For sequence:

$$\frac{\langle f, \vec{e}, \kappa \rangle \Downarrow r_1 \text{ and } r_1.status = success \text{ and } \langle g, r_1.outputs, \kappa \rangle \Downarrow r_2}{\langle f; g, \vec{e}, \kappa \rangle \Downarrow r_1 \diamond r_2}.$$

If r_1 is non-success, the sequence terminates with r_1 . The operator $\diamond$ concatenates observations/artifacts and uses the later outputs/status.

For tensor, the input frame is split according to branch arities. Both branches execute under the interpreter's scheduling policy. The reference implementation executes serially and concatenates outputs in branch order. A concurrent interpreter must preserve that output order.

A.7 Static analysis algebra

A summary is:

$$A = (Ops, Effects, Deps, det, cache, W, C, M),$$

with operation/effect/dependency sets, determinism/cacheability booleans, total work W , critical path C , and memory hint M .

Identity contributes empty sets, true booleans, and zero resources. Primitive contributes its descriptor. Sequence combines sets by union, booleans by conjunction, work and critical path by addition, and memory by maximum. Tensor combines sets/booleans similarly, work by addition, critical path by maximum, and memory by addition.

These equations form one abstract interpretation. A probabilistic cost interpreter could replace scalar hints with distributions; a policy interpreter could use a security lattice.

A.8 Parameterized systems

For a strict symmetric monoidal category C , a parameterized arrow $A \rightarrow B$ is an equivalence class of pairs (P, f) with $f: P \otimes A \rightarrow B$. The sandbox retains the representative rather than quotienting by reparameterization.

Composition is:

$$(Q, g) \circ (P, f) = \left(P \otimes Q, (P \otimes Q) \otimes A \xrightarrow{\sigma} Q \otimes (P \otimes A) \xrightarrow{id \otimes f} Q \otimes B \xrightarrow{g} C \right).$$

A reparameterization $r: R \rightarrow P$ maps (P, f) to $(R, f \circ (r \otimes id))$.

A.9 Optics and interventions

A simple total lens is (get, put) . An intervention value is:

$$i = (id, optic, value, class, targets, closure, hypothesis).$$

Applying i to baseline θ is partial:

$$apply: \Theta \times I \rightarrow \Theta + Invalid.$$

A candidate is:

$$c = (baselineID, intervention, resultEnvelope, candidateID),$$

where candidate identity is content-derived. The sandbox enforces one intervention. A generalized candidate may contain a typed patch plan with explicit order and commutation evidence.

A.10 Stochastic semantics

A finite distribution on X is a normalized function $\mu: X \rightarrow [0, 1]$ with finite support. A Markov kernel is $K: X \rightarrow D(Y)$. Composition is:

$$(L \circ K)(x)(z) = \sum_y K(x)(y) L(y)(z).$$

Tensor in the finite implementation constructs independent products:

$$(K \otimes L)(x, u)(y, v) = K(x)(y) L(u)(v).$$

Paired experiment coupling is not tensor. It is a kernel

$$\Gamma: X \rightarrow D(Y_b \times Y_c)$$

whose marginals are the arm kernels. The sandbox realizes a coupling by deriving one seed from campaign/case/repeat and passing it to both arms.

A.11 Trial coordinates and reports

A coordinate is:

$$q = (campaign, candidate, case, repeat, arm).$$

A paired coordinate omits arm. The report for candidate c is computed only when exactly one terminal baseline and candidate result exist for every required pair.

For metric definition $(id, direction, unit)$ and successful values b_i, c_i , oriented difference is:

$$\delta_i = direction \cdot (c_i - b_i),$$

where direction is +1 for maximize and -1 for minimize. Missing values remain missing. Failures are counted separately and can make gates indeterminate or failed.

A.12 Decisions

A gate is a partial decision function:

$$g: Report \rightarrow Pass(reason) + Fail(reason) + Indeterminate(reason).$$

A sequence evaluates left to right. A candidate is eligible only when all gates pass. Preference score is defined only for eligible candidates.

Pareto dominance is defined over complete oriented metric vectors. Missing dimensions prevent dominance unless policy explicitly defines an imputation, which should be rare and visible.

A.13 Campaign transition system

State contains campaign identity, started flag, candidates, cells, reports, decisions, and terminal result. Events are:

```
CampaignStarted
CandidateRegistered
TrialRecorded
ReportRecorded
DecisionRecorded
CampaignCompleted
CampaignCancelled (extension)
```

Transition $s \xrightarrow{e} s'$ is defined by the reducer. Invalid transitions return an error and do not change state. The engine derives commands from missing work, executes them, appends an event, then applies it. The store append is the linearization point for local custody.

The reference engine uses a nonadaptive finite candidate list and sequential scheduling. These are implementation choices, not doctrine laws.

Appendix B. Sandbox API reference

B.1 Build and run

The module is self-contained and uses only the Go standard library.

```
cd opfield
make verify
make demo-rag
make demo-quadratic
```

Equivalent direct commands are:

```
go test ./...
go vet ./...
go test -race ./...

go run ./cmd/opfield-demo \
  -domain rag \
  -out ./demo-out/rag \
  -repeats 3

go run ./cmd/opfield-demo \
  -domain quadratic \
  -out ./demo-out/quadratic \
  -repeats 4
```

Re-running without reset replays `events.jsonl`, skips terminal coordinates, and leaves a completed campaign unchanged.

B.2 core

Important values:

```
type SchemaID string
type OperationID string
type PluginID string

type Schema struct {
    ID          SchemaID
    Version     string
    Description string
}

type Port []SchemaID

type Envelope struct {
    Schema  SchemaID
    Payload json.RawMessage
    Digest  Digest
}
```

`Port.Tensor` concatenates wires. `Envelope.Validate` verifies schema presence, canonical JSON, and domain-separated digest. `DecodeEnvelope[T]` rejects unknown fields.

`OperationDescriptor` records plugin, version, input/output ports, effects, dependencies, determinism, cacheability, and cost hints. `Execution` records typed outputs, observations, artifact references, status, failure, and duration.

B.3 plugin

```
type Plugin interface {
    Manifest() Manifest
    Install(*Builder) error
    Laws() []Law
}

type Operation interface {
    Descriptor() core.OperationDescriptor
    Execute(context.Context, []core.Envelope) core.Execution
}
```

Registration is transactional. `Install` writes only to a private builder. Laws execute before the global registry lock commits schemas and operations. Collisions or unresolved schemas reject the entire plugin.

Typed adapters reduce boundary boilerplate:

```
plugin.Unary[I,O]
plugin.Binary[A,B,O]
plugin.Ternary[A,B,C,O]
```

Each takes codecs and an ordinary typed `Run` function. Errors from domain code become domain failures; decode/encode/arity errors become infrastructure failures.

B.4 plan

Constructors include:

```
plan.Identity(port)
plan.Primitive(descriptor)
plan.Seq(parts...)
plan.Tensor(parts...)
plan.Permute(port, order)
plan.Copy(port)
plan.Drop(port)
```

`Seq` and `Tensor` validate adjacent/combined ports, flatten nested nodes, and remove identities where legal. `Plan.ID` hashes the canonical normalized syntax. `Validate` resolves primitive descriptors against a registry.

The generic extension point is:

```
type Algebra[R any] struct {
    Identity func(core.Port) (R, error)
    Primitive func(core.OperationDescriptor) (R, error)
    Sequence func([]R) (R, error)
    Tensor func([]R) (R, error)
    Permute func(core.Port, []int) (R, error)
    Copy func(core.Port) (R, error)
    Drop func(core.Port) (R, error)
}

func Fold[R any](p *Plan, a Algebra[R]) (R, error)
```

`plan.Analyze` is one supplied fold.

B.5 engine

```
type Executor struct {
    Registry *plugin.Registry
}

func (e Executor) Run(
    ctx context.Context,
    p *plan.Plan,
    inputs []core.Envelope,
) core.Execution
```


The executor validates the plan and inputs, folds to an executable arrow, and runs it. Sequence stops on failure. Tensor preserves branch output order. The reference tensor is serial.

B.6 artifact

`artifact.Store` offers content-addressed `Put` and verified `Get`. The local implementation stores files under digest-derived safe paths. Media type is included in the digest domain. `PutJSON` canonicalizes values before storage.

The store is suitable for the sandbox, not a multi-tenant production artifact service.

B.7 optic

```
type Lens[S,A any] struct {
  Name string
  Get  func(S) A
  Put  func(S, A) (S, error)
}
```

`CheckLaws` tests get-put, put-get, and put-put over provided states/values and equality functions. Partial validation is supported through `Put` errors; fixtures must contain admissible values.

B.8 para

```
type Parametric[P,A,B any] struct {
  Run func(P, A) B
}
```

The package provides composition, tensor, and reparameterization. It is an executable illustration of `Para`, not a runtime reflection system.

B.9 prob

`Dist[T]` is a normalized finite distribution. `Kernel[A,B]` maps an input to `Dist[B]`. The package provides pure distributions, bind/composition, tensor, expectation for finite numeric projections, and deterministic seed derivation.

B.10 metric

```
type Definition struct {
  ID          string
  Direction Direction
  Unit        string
}

type Vector map[string]float64
```

Functions validate finite metrics, orient deltas, test Pareto dominance, and compute a deterministic Pareto front.

B.11 experiment

Core high-level ports include:

```

type Space interface {
    ID() string
    Schema() core.SchemaID
    Baseline(context.Context) (core.Envelope, error)
    Patches(context.Context, core.Envelope) ([]Patch, error)
    Apply(context.Context, core.Envelope, Patch) (core.Envelope, error)
}

type Runner interface {
    ID() string
    MetricDefinitions() []metric.Definition
    Run(context.Context, TrialRequest) TrialResult
}

```

The package also defines candidates, workloads, trial coordinates, semantic classes, paired reports, and comparison.

A `Patch` contains optic identity, typed value, semantic classes, direct targets, closure, and hypothesis. Production code should calculate or verify closure rather than trusting a literal list.

B.12 decision

A policy is an ordered sequence of gates. Supplied helpers cover coverage, failure-free execution, mean and worst-case oriented deltas, absolute candidate bounds, and preference score. Results are `pass`, `fail`, or `indeterminate` with reasons.

B.13 campaign

```

type Store interface {
    Load(context.Context) ([]Event, error)
    Append(context.Context, Event) error
}

```

The engine binds space, proposer, workload, runner, policy, store, repeats, and seed root. It computes campaign identity, replays state, executes missing cells, records reports/decisions, and appends terminal selection.

The JSONL store fsyncs append operations and serializes writers in-process. A distributed store requires expected-position CAS and fencing.

B.14 RAG toy plugin

The RAG plugin registers schemas for spec, corpus, chunks, index, case, retrieval, and trial output. It registers `chunk`, `index`, `retrieve`, and `measure` operations plus lens laws.

The runner owns a build and query plan, content-addressed artifacts, and an in-memory map from semantic build keys to artifact references. It writes native trial artifacts and projects recall, MRR, hit rate, scored chunks, and context words.

B.15 Quadratic graft

The quadratic graft implements only the high-level interfaces. It demonstrates that `campaign` does not depend on `plan` or `plugin`. Deterministic seed-derived noise exercises pairing without external providers.

Appendix C. Plugin authoring and conformance protocol

C.1 When to create a plugin

Create a low-level operation plugin when at least one of the following is true:

- the operation must appear in static effect, dependency, resource, or policy analysis;
- alternate implementations or backends should substitute under one signature;
- the output deserves independent artifact identity and reuse;
- the operation crosses a trust or provider boundary;
- a remote/distributed interpreter must schedule it;
- a law or certification suite applies to it independently;
- the operation is shared across products.

Do not create a primitive merely because a function exists. Internal parsing helpers, formatting functions, and tightly coupled local algorithms can remain inside one implementation.

Create a high-level graft when the optimized unit is already a coherent application action: a complete search request, answer request, agent turn, conversation, build benchmark, or production shadow. Fine-grained decomposition can follow selectively.

C.2 Manifest checklist

A plugin manifest should include:

```
identity
  plugin ID
  semantic version
  source/build digest
  publisher/signature
  supported kernel protocol

surface
  schemas and codec versions
  operations and typed ports
  effects and data classes
  dependency labels
  determinism/cacheability class
  resource hints
  capabilities and preconditions

assurance
  law suite ID
  conformance artifact digest
  benchmark/certification class
  known limitations
  security review status
```

The sandbox manifest is intentionally smaller. Production fields can be added without changing plan syntax because they decorate primitive descriptors.

C.3 Registration protocol

Recommended registration steps are:

1. create a private builder transaction;
2. validate manifest identity and compatibility;

3. register schema descriptors and codecs;
4. register operation descriptors and implementations;
5. resolve every input/output schema;
6. run generic codec and descriptor checks;
7. run plugin-declared laws;
8. verify conformance certificate policy;
9. acquire registry commit lock;
10. recheck global collisions;
11. commit one immutable registry snapshot.

On any failure, no global state changes. The plugin should not start background goroutines or open resources during `Install`; runtime resources belong to interpreter construction and leases.

C.4 Codec conformance

For each codec C_A and fixture set:

- `Decode(Encode(x)) = x` under declared equality;
- encoding is canonical and deterministic;
- envelope schema equals A ;
- unknown fields, wrong schema, malformed JSON, nonfinite numbers, and wrong digest fail;
- maximum payload size is enforced where required;
- cross-language golden vectors agree;
- secrets are not included unless the schema explicitly represents them.

A codec migration is a primitive operation from old schema to new schema. It should not silently decode two meanings under one ID.

C.5 Operation conformance

Generic tests invoke each operation with:

- correct inputs;
- wrong arity;
- wrong schemas;
- corrupted payload/digest;
- cancelled context;
- expired deadline;
- boundary-sized inputs;
- an instrumented capability environment.

The operation must return a terminal `Execution`. Panics should be contained by a process boundary or converted to infrastructure failure in trusted in-process hosts.

If `Deterministic` is true, repeated runs over equal canonical inputs and fixed capabilities must produce equal protected outputs and artifacts. Observed wall-clock duration may differ. If `Cacheable` is true, the plugin must specify the equivalence class under which replay is valid.

C.6 Effect conformance

Declared effect sets are upper bounds. An instrumented host records filesystem namespaces, network endpoints, provider calls, random sources, clocks, secrets, and artifact operations. Observed effects must be contained in the declaration.

This is not a complete security proof. Native code can evade user-space instrumentation. Process isolation and operating-system policy enforce the boundary for untrusted plugins.

C.7 Dependency conformance

For every configuration field and representative input mutation:

1. compute the plugin-declared affected closure;
2. run a clean candidate execution;
3. run an execution reusing artifacts outside the closure;
4. compare protected outcomes and traces;
5. fail if a reused artifact changes candidate meaning.

A development-only configuration proxy can record field reads by operation, producing an observed dependency graph. The declared graph must cover observed reads. Ambient environment access is prohibited or converted into explicit capability/configuration inputs.

C.8 Domain law examples

Chunker. Every output chunk references one input document, has valid range, reconstructs declared text, follows deterministic order, and has stable identity.

Embedding. Output dimension and normalization match model spec; equal semantic keys replay valid vectors; nonfinite vectors fail.

ANN index. Every result belongs to snapshot and filter; deleted items are absent; recall relation to exact oracle holds under certified envelope; reopen/compaction preserve relation.

Authorization. Returned and remotely disclosed candidates are subsets of allowed set; policy identity and subject are bound; missing labels fail closed.

Reranker. Candidate identity is preserved; scores are finite; pool depth is bounded; provider disclosure matches certificate; fallback is explicit.

Answer validator. Every accepted citation resolves to admitted evidence from the release; unsupported output becomes a typed failure/abstention.

Frontend projection. Every widget field has declared provenance; snapshot plus suffix converges; duplicate/stale events do not corrupt state.

C.9 High-level graft conformance

A runner should satisfy:

- returned coordinate exactly equals requested coordinate;
- result is terminal and finite;
- release and workload identities are retained in native artifact;
- domain failures are returned as trial results, not outer campaign errors;
- infrastructure errors are attributable;
- repeated idempotency keys do not duplicate side effects;
- every generic metric can be recomputed or audited from the native artifact;
- cancellation is propagated and represented;
- no model-controlled case field grants authorization or release selection.

A workload should supply stable case IDs, canonical case inputs, strata/tags, and immutable workload identity. A proposer should be pure over its declared history or materialize its output as an event.

C.10 Compatibility policy

Semantic versioning should distinguish:

- additive new plugin generators;
- descriptor-compatible implementation fixes;
- behavior changes requiring new operation ID/version;
- schema evolution requiring migration;
- law/certification changes;
- kernel plan-syntax changes.

An old plan is resumable only when its registry, schemas, operations, runner, workload, metrics, policy, and campaign protocol remain compatible. Otherwise, start a new campaign and link it to the old one as a migration or re-evaluation.

Appendix D. Worked RAG optimization trace

D.1 Baseline

The sandbox baseline specification is:

```
{
  "chunk_words": 24,
  "overlap_words": 4,
  "lexical_weight": 1,
  "vector_weight": 1,
  "top_k": 2
}
```

Its canonical envelope digest is:

```
sha256:5e6885c08caea34cd8b49f977c5429e2612ca696252696db36405a5623693f3e
```

The corpus contains a small set of gardening documents. The workload contains nine queries with relevant document IDs. It is deliberately small enough that every artifact and event can be inspected.

D.2 Build plan

The build plan has port:

```
[ragtoy.spec/v1, ragtoy.corpus/v1]
  → [ragtoy.index/v1]
```

and syntax:

```
Seq(
  Primitive(ragtoy.chunk/v1),
  Primitive(ragtoy.index/v1)
)
```

The chunk operation consumes spec and corpus and returns a chunk set. The index operation consumes the chunk set and returns one value containing lexical terms and hashed semantic vectors. The plan ID is:

```
sha256:85b789bed04ef6b1de388011166964e8d1174427cc19723a52b4cd22c307ed57
```

Static analysis:

Field	Value
operations	ragtoy.chunk/v1, ragtoy.index/v1
effects	cpu, artifact.write
dependencies	corpus normalization, chunk, lexical index, vector index
deterministic	true
cacheable	true
work hint	6
critical path	6
memory hint	1 MiB

D.3 Query plan

The query plan port is:

```
[ragtoy.index/v1, ragtoy.spec/v1, ragtoy.case/v1]
  → [ragtoy.trial/v1]
```

and syntax:

```
Seq(
  Primitive(ragtoy.retrieve/v1),
  Primitive(ragtoy.measure/v1)
)
```

Its ID is:

```
sha256:af800326216f634edf702fb2ce25e93db4cedacaa8bf655959b4bb85415d29b7
```

D.4 Candidate construction

Consider the candidate `chunk-34`. Its patch is:

```
{
  "id": "chunk-34",
  "optic": "ragtoy.spec.chunk_words",
  "value": {"kind": "int", "int_value": 34},
  "classes": ["knowledge", "relevance"],
  "targets": ["index.chunk"],
  "closure": [
    "index.chunk",
    "index.lexical",
    "index.vector",
    "query.channels",
    "eval.retrieval"
  ],
  "hypothesis": "larger chunks preserve context"
}
```

The chunk-size lens reads 24 and writes 34. Its registration law checks:

```
Put(base, Get(base)) = base
Get(Put(base, 34)) = 34
Put(Put(base, 14), 34) = Put(base, 34)
```

The resulting spec is canonicalized and bound with baseline and patch to form candidate ID. The dependency closure says this candidate needs a distinct build.

By contrast, `lexical-1.75` targets `query.fusion`; its build-relevant projection remains `(24,4,corpusDigest)`, so it shares the baseline index artifact.

D.5 Build-key interpretation

The sandbox runner computes:

```
buildSpec := struct {
  ChunkWords  int
  OverlapWords int
}{s.ChunkWords, s.OverlapWords}

key := core.Sum(
  "ragtoy/build-key/v1",
  canonical(buildSpec),
  corpusEnvelope.Digest,
)
```

This is a manual approximation of dependency-derived identity. It is correct for the toy domain because only chunk and overlap affect the index. A production interpreter would derive the relevant parameter projection from plan dependencies and operation descriptors.

On cache hit, the artifact store reads the retained index envelope and validates its digest. The trial trace records `ragtoy.build_reused` and `ragtoy.build_cache hit=true`. On miss, the build plan executes and the output envelope is stored content-addressably.

D.6 One paired coordinate

For case `q-prune`, repeat 0, and candidate `lexical-1.75`, the campaign derives one seed from campaign, case, and repeat. It executes:

```
baseline coordinate:
  case=q-prune, repeat=0, arm=baseline

candidate coordinate:
  case=q-prune, repeat=0, arm=candidate,
  candidate=sha256:0a37...
```

Both results are successful. Baseline metrics are:

```
{
  "recall": 1,
  "mrr": 1,
  "hit_rate": 1,
  "scored_chunks": 12,
  "context_words": 48
}
```

The candidate has the same metrics for this case, but different component scores. The native output records lexical, vector, and combined values. This distinction matters: generic metrics can tie while the trace shows a changed retrieval mechanism.

D.7 Comparison

For candidate `c`, the engine gathers $9 \times 3 = 27$ baseline and 27 candidate results. For recall, it computes oriented deltas per coordinate and aggregates mean and worst case. Domain failures would remain as failed cells and be handled by gates before metric preference.

The `chunk-34` candidate has mean recall `0.944444` versus baseline `0.916667`, oriented mean change `0.027778`, and passes all hard gates. `topk-1` and `vector-1.75` have mean recall `0.861111` and fail a nonregression gate.

D.8 Decision sequence

The RAG policy evaluates:

1. complete paired coverage;
2. no failed arm cells;
3. worst-case recall nonnegative;
4. mean recall nonnegative;
5. context-word upper bound;
6. worst-case hit-rate nonnegative;
7. recall preference score.

Only after every gate passes is mean oriented recall used for deterministic candidate selection. `chunk-34` is selected because it is the only passing candidate with positive preference score.

This decision is local to the toy corpus and workload. The artifact states the evidence; it does not generalize the chunk-size claim.

D.9 Common-prefix diagram

For query-only `lexical-1.75`:

```
corpus — chunk(24,4) — index —————┐
                                         │ retrieve(weights 1,1) — measure
                                         │ retrieve(weights 1.75,1) — measure
```

For build-changing `chunk-34`:

```
corpus — chunk(24,4) — index — retrieve — measure
      └───┬─── chunk(34,4) — index — retrieve — measure
```

The first comparison shares the build prefix; the second shares only corpus input. This is the central operational benefit of causal parameterization.

Appendix E. Campaign event trace and replay

E.1 Event prefix

The RAG campaign contains 239 events. Its first events are conceptually:

```
{ "sequence": 1, "type": "campaign.started", "campaign_id": "sha256:5489..." }
{ "sequence": 2, "type": "candidate.registered", "candidate_id": "sha256:0a37..." }
{ "sequence": 3, "type": "candidate.registered", "candidate_id": "sha256:4bf4..." }
...
```

Candidates are sorted by content ID before registration, so enumeration order from the space does not determine event order.

After registration, trial events contain exact coordinate, terminal status, metric vector, optional output envelope, observations, native artifact reference, and timestamps.

E.2 Baseline reuse in coordinates

The engine uses one baseline candidate identity across candidate comparisons. A baseline cell for a case/repeat is written once and reused in every report. Candidate cells include candidate identity. This yields:

$$N_{\text{trials}} = |W| \cdot R \cdot (1 + |C|)$$

rather than $2|W|R|C|$, when all candidates share the same baseline execution semantics. In the toy campaign:

$$9 \cdot 3 \cdot (1 + 7) = 216$$

trial events. Adding one start, seven candidate registrations, seven reports, seven decisions, and one completion yields:

$$1 + 7 + 216 + 7 + 7 + 1 = 239.$$

This arithmetic is also a custody invariant. A different event count must be explained by failures before registration, adaptive proposals, retries represented as separate attempt events, or a changed protocol.

E.3 Report and decision events

After all cells for one candidate exist, the engine appends `comparison.recorded`. It then applies policy and appends `decision.recorded`. Reports and decisions are immutable values keyed by candidate ID.

The reducer rejects a decision before its report, a duplicate report, or an unknown candidate. It does not currently verify that a report can be recomputed from prior cells; the engine constructs it correctly. A stronger store validator can recompute reports during replay or store a report input digest.

E.4 Completion

After all candidate decisions, the engine selects the highest preference score among passing candidates with candidate-ID tie-break and appends:

```
{
  "type": "campaign.completed",
  "selected": "sha256:ad2627fa..."
}
```

The reducer marks state terminal. A subsequent engine invocation loads events and returns without appending. This property prevents a new binary from silently extending an old terminal experiment.

E.5 Interruption and resume

The test store injects failure after a fixed append count. The first engine invocation returns an error after a valid prefix. No in-memory transition occurs for the failed append. The second invocation:

1. loads and reduces the prefix;
2. reconstructs registered candidates and completed cells;
3. selects only missing semantic work;
4. appends from `LastSequence+1`;
5. reaches the same terminal selection;
6. contains no duplicate trial keys.

This is a semantic resume. There is no special checkpoint branch that tries to reconstruct hidden loop indexes.

E.6 Distributed extension

A distributed event type should add command/attempt information without changing the accepted cell uniqueness law:

```
trial.requested
trial.attempt.started
trial.attempt.observed
trial.completed ← at most one accepted terminal cell
```

Attempt events support operational diagnosis. The report still consumes one terminal result per coordinate. Fencing and expected-sequence append prevent stale workers from committing after ownership changes.

E.7 Replay audit

A replay audit should:

- validate every event schema and campaign ID;
- verify contiguous sequence;
- verify all referenced artifacts;
- recompute candidate IDs from baseline/patch/spec;
- recompute trial keys;
- validate metric definitions and finite values;
- optionally recompute reports and decisions;
- compare terminal selected ID;
- emit an audit artifact.

This audit can run independently of the original runner and providers when native material is retained.

Appendix F. Empirical mapping to the supplied repositories

F.1 Scope

The supplied snapshot contains five relevant scopes:

Scope	Go files	Nonblank Go lines	Go test functions
ragkit	173	17,743	273
ragopt	45	5,925	42
RAG-TTC	515	76,705	906
GEC	200	28,668	252
TTC Garden backend	70	8,485	108

Counts describe the analyzed archive and simple static rules. They are not measures of quality or deployed scope.

F.2 `ragopt` evidence

The reviewed `ragopt` packages and design records establish:

- candidate bundles with one mutable asset and locked context;
- suite/case identity;
- repeated baseline/candidate cells;
- local run stores and resume;
- strict paired comparison;
- explicit missing metrics;
- ordered gates and reports;
- a product adapter pattern in RAG-TTC.

This supports retaining `ragopt` as the campaign/custody layer. The proposed backbone adds compositional system, parameter, and plugin semantics beneath or beside it rather than replacing its evidence discipline.

F.3 `ragkit` evidence

Relevant `ragkit` areas include:

- domain types and evidence identity;
- chunking and representation packages;
- embedding and flow execution;
- lexical/vector indexes and immutable bundles;
- retrieval, collapse, fusion, reranking, context, answer contracts;
- evaluation metrics and deterministic ordering;
- boundary tests that constrain provider/UI dependencies.

These packages supply natural objects and generators for a RAG signature. They do not currently provide a domain-neutral free plan/Para/optic/campaign doctrine, which is why the backbone should remain separate or in a minimal shared module.

F.4 RAG-TTC evidence

RAG-TTC contains:

- a copied common RAG substrate substantially overlapping `ragkit`;
- full corpus index builds with representation generation and embedding caches;
- committed-Git source snapshots and workspace artifacts;
- exact vector search and an HNSW candidate/bakeoff;
- route-specific lexical/vector/connected retrieval;
- model-invoked search with turn-scoped evidence;
- tool-loop and answer evaluation;
- persistent chat/session runtime.

It is therefore the best initial integration target for both low-level plans and coarse campaign grafts.

F.5 GEC evidence

GEC contains:

- a startup-opened immutable bundle and verified corpus;
- server-owned access scopes and source roles;
- lexical synonyms;
- weighted hybrid fusion;
- optional cross-encoder reranking and fallback;
- retrieval/answer evaluation and parameter sweep;
- administrative tool projection and evidence labels;
- snapshot plus WebSocket frontend state.

The source order shows filtering after retrieval and optional reranking, motivating the authorization-before-remote-disclosure law. Synonyms and reranker settings outside bundle identity motivate behavior-complete release/candidate materialization.

The imported `internal/knowledgebuild` source is absent from the supplied archive, so exact production indexing behavior could not be inspected directly.

F.6 Garden evidence

Garden contains:

- intent-aware search routes;
- structured product facts and exact provenance;
- connected retrieval and fallback;
- per-conversation evidence state;

- grounded source/product widgets;
- multi-turn calibration against a running chat service.

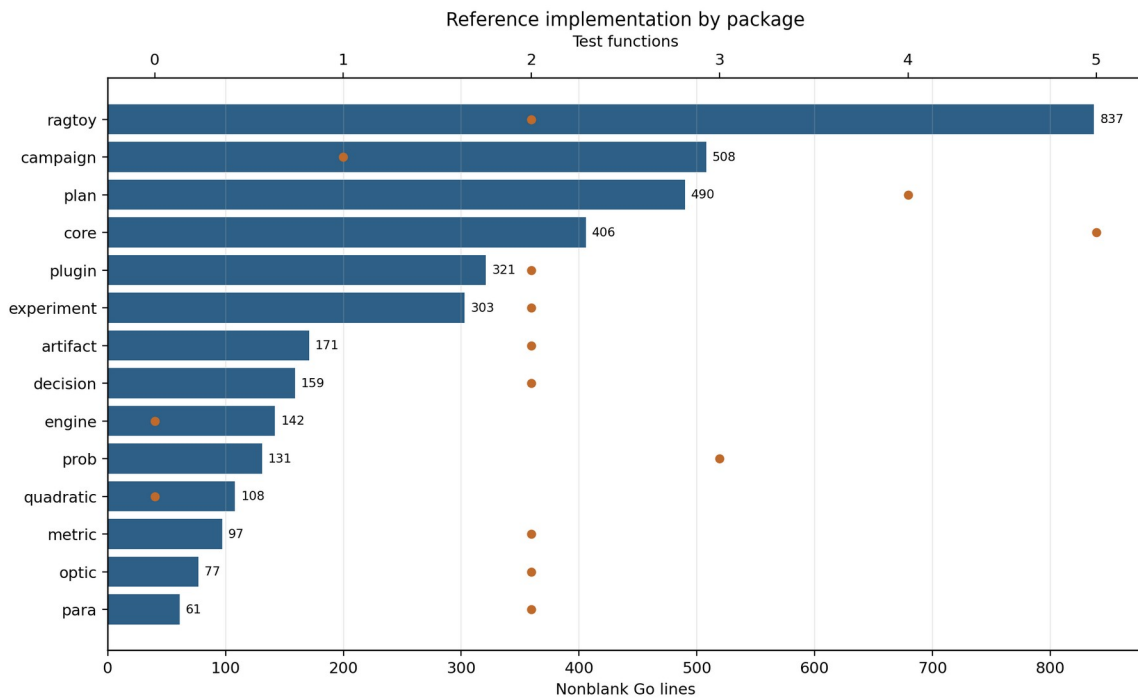
This supports the high-level graft design. Its user outcome is a trajectory and typed presentation, not a single retrieval vector.

F.7 Current-to-target matrix

Current mechanism	Target backbone role
ragkit operation packages	plugin generators and RAG-domain implementations
ragkit/flow	execution interpreter substrate/resource handler
ragkit/indexbundle	material/release artifact interpretation
ragopt candidates/runs	high-level experiment and campaign custody
RAG-TTC tool-eval adapter	model for coarse Runner graft
GEC RRF sweep	specialized proposer/runner over frozen channel artifacts
RAG-TTC ANN bakeoff	backend refinement certification campaign
Garden calibration	high-fidelity session runner
product config files	typed release schemas and optics
hand-coded cache keys	dependency/plan-derived artifact interpretation
ad hoc promotion instructions	immutable gate report plus external activation

F.8 Implementation scale

The `opfield` sandbox contains a small standard-library-only Go module with packages for core identity, plugins, plans, execution, artifacts, optics, Para, finite probability, metrics, experiments, decisions, campaign state, and two domains. At final verification it passes unit tests, `go vet`, and the race detector. Coverage is intentionally uneven: domain and campaign packages are exercised strongly, while a production-grade system would add much more fuzz, property, and failure testing.



The sandbox is deliberately small relative to the supplied RAG systems.

F.9 Analysis limitations

- The repository archive is a development snapshot, not a production deployment.
- GEC's knowledge-build package is missing.
- The supplied RAG repositories require a newer Go toolchain than was available offline during the prior broad study; empirical claims here rely on source analysis and tests.
- The new `opfield` module itself builds and runs under the available Go 1.23 toolchain.
- No production provider, traffic, latency, cost, or user-outcome experiment was performed.
- Mathematical results are a mixture of standard constructions, proof sketches, and executable laws; the Go code is not a formal proof.

Appendix G. Selected bibliography

G.1 Categories, monoidal structure, and computation

Mac Lane, Saunders. 1998. *Categories for the Working Mathematician*. Second edition. Springer.

Selinger, Peter. 2011. “A Survey of Graphical Languages for Monoidal Categories.” In *New Structures for Physics*, edited by Bob Coecke. Springer.

Joyal, André, Ross Street, and Dominic Verity. 1996. “Traced Monoidal Categories.” *Mathematical Proceedings of the Cambridge Philosophical Society* 119 (3): 447–468.

Fritz, Tobias, and Wendong Liang. 2023. “Free gs-Monoidal Categories and Free Markov Categories.” *Applied Categorical Structures* 31: 21.

The free-signature and wiring construction in this thesis uses standard category-theoretic practice. Explicit copy and discard are presented in a dataflow-oriented doctrine rather than assumed for arbitrary effects.

G.2 Parameterized systems and compositional learning

Fong, Brendan, David I. Spivak, and Rémy Tuyéras. 2019. “Backprop as Functor: A Compositional Perspective on Supervised Learning.” In *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science*.

Cruttwell, G. S. H., Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. 2022. “Categorical Foundations of Gradient-Based Learning.” In *Programming Languages and Systems: ESOP 2022*.

These works motivate [Para](#) and compositional learning structure. The present thesis uses the same architectural idea for discrete, constrained, stochastic, and non-gradient optimization.

G.3 Optics

Riley, Mitchell. 2018. “Categories of Optics.” arXiv:1809.00738.

Pickering, Matthew, Jeremy Gibbons, and Nicolas Wu. 2017. “Profunctor Optics: Modular Data Accessors.” *The Art, Science, and Engineering of Programming* 1 (2).

The sandbox implements only total lenses. The general architecture anticipates prisms, traversals, affine optics, and dependent validation.

G.4 Probability and Markov categories

Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics.” *Advances in Mathematics* 370: 107239.

Giry, Michèle. 1982. “A Categorical Approach to Probability Theory.” In *Categorical Aspects of Topology and Analysis*. Springer.

Kock, Anders. 1972. “Strong Functors and Monoidal Monads.” *Archiv der Mathematik* 23: 113–120.

The finite-distribution package is an educational approximation. Production trial semantics are sample- and artifact-based but retain the kernel/coupling distinction.

G.5 Effects and operational semantics

Plotkin, Gordon D. 1981. “A Structural Approach to Operational Semantics.” DAIMI FN-19. Reprinted in *Journal of Logic and Algebraic Programming* 60–61 (2004): 17–139.

Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55–92.

Plotkin, Gordon, and Matija Pretnar. 2009. “Handlers of Algebraic Effects.” In *Programming Languages and Systems: ESOP 2009*.

The thesis uses explicit outcomes and interpreters in Go while drawing on these accounts of effectful composition.

G.6 Open dynamical systems and compositional decisions

Myers, David Jaz. 2021. “Double Categories of Open Dynamical Systems.” arXiv:2005.05956.

Ghani, Neil, Jules Hedges, Viktor Winschel, and Philipp Zahn. 2018. “Compositional Game Theory.” In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*.

Coecke, Bob, Tobias Fritz, and Robert W. Spekkens. 2016. “A Mathematical Theory of Resources.” *Information and Computation* 250: 59–86.

These works motivate open interfaces, explicit continuation/environment dependence, and ordered resource semantics. The campaign model here is a simpler event-sourced state machine.

G.7 Experimentation and statistical design

Fisher, Ronald A. 1935. *The Design of Experiments*. Oliver and Boyd.

Cochran, William G., and Gertrude M. Cox. 1957. *Experimental Designs*. Second edition. Wiley.

Owen, Art B. 2013. *Monte Carlo Theory, Methods and Examples*.

Howard, Steven R., Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. 2021. “Time-Uniform, Nonparametric, Nonasymptotic Confidence Sequences.” *The Annals of Statistics* 49 (2): 1055–1080.

Paired coordinates and common random numbers follow established variance-reduction and experimental-design principles. Online canaries require procedures valid under sequential monitoring.

G.8 Retrieval and RAG

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems* 33.

Karpukhin, Vladimir, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. “Dense Passage Retrieval for Open-Domain Question Answering.” In *Proceedings of EMNLP 2020*.

Cormack, Gordon V., Charles L. A. Clarke, and Stefan Büttcher. 2009. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” In *Proceedings of SIGIR 2009*.

Malkov, Yu. A., and D. A. Yashunin. 2020. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–836.

These sources provide retrieval mechanisms and evaluation context. The thesis is concerned principally with the compositional architecture of optimizing such systems.

G.9 Software testing and distributed protocols

Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of ICFP 2000*.

Lamport, Leslie. 1994. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16 (3): 872–923.

Herlihy, Maurice P., and Jeannette M. Wing. 1990. “Linearizability: A Correctness Condition for Concurrent Objects.” *ACM Transactions on Programming Languages and Systems* 12 (3): 463–492.

These works motivate property-based law testing, model checking, and linearizable event/activation stores.

Appendix H. Glossary

Algebra. An assignment of meanings to primitive and structural constructors. In the implementation, `plan.Algebra[R]` supplies the cases consumed by `Fold`.

Artifact. Immutable retained material with media type, content digest, size, and storage reference. Artifacts are richer evidence than generic metric values.

Baseline. The fixed incumbent specification or release against which candidate trials are paired.

Candidate. A baseline, lawful intervention, resulting specification, and content-derived identity.

Campaign. The durable protocol that registers candidates, executes exact trial coordinates, compares pairs, applies policy, and records a terminal selection.

Capability. An explicitly granted resource or authority such as artifact access, network endpoint, secret, clock, or random source.

Catamorphism. A unique structural fold from recursively defined syntax into an algebra. `plan.Fold` is the executable form.

Closure. The transitive downstream dependency set affected by an intervention's direct targets.

Codec. A schema-specific canonical encoder/decoder used at generic plugin boundaries.

Common random numbers. A coupling technique that supplies baseline and candidate trials with corresponding random inputs to reduce variance when outcomes are positively correlated.

Conformance certificate. Versioned evidence that a plugin implementation satisfies generic and domain-specific laws under a declared test envelope.

Coupling. A joint distribution over baseline and candidate outcomes with the correct marginal distributions.

Dependency interpretation. A fold or analysis that maps a plan to the semantic nodes and artifacts on which its output depends.

Descriptor. The immutable semantic declaration of a plugin operation: ID/version, ports, effects, dependencies, determinism, cacheability, and resource hints.

Domain graft. A coarse integration that implements space, proposer, workload, runner, policy, and store around an existing application.

Effect. A declared interaction beyond pure data transformation, such as network, remote disclosure, artifact write, randomness, clock, or secret access.

Envelope. A canonical schema-bearing payload with a domain-separated digest.

Fidelity. A level of evaluation evidence, from static laws through retrieval, answers, sessions, load, shadow, and canary.

Free wiring category. The category generated by typed primitive operations and fixed structural constructors, subject only to the selected wiring laws.

Gate. An ordered eligibility predicate returning pass, fail, or indeterminate with evidence.

Generator. A primitive typed operation symbol contributed by a plugin.

High-level plugin surface. The campaign ports used to graft a complete domain or application without exposing internal plans.

Intervention. A typed lawful patch plus semantic class, direct targets, dependency closure, and hypothesis.

Interpreter. A structure-preserving meaning assigned to plans, such as execution, effects, cost, provenance, or deployment.

Law. A structural or domain equation/invariant whose evidence is checked at registration, test, or certification time.

Lens. A lawful getter/setter optic for focusing a local parameter inside a larger configuration.

Low-level plugin surface. Registration of schemas, primitive generators, descriptors, codecs, implementations, and laws.

Markov kernel. A mapping from inputs to probability distributions over outputs; the denotation of a stochastic stage or trial.

Material identity. Identity of exact retained bytes, distinct from semantic plan or campaign identity.

Metric direction. Maximize or minimize declaration used to orient candidate-minus-baseline differences so positive means improvement.

Missingness. The explicit absence of a metric or trial cell. It is not silently converted to zero or dropped.

Monoidal product. Parallel composition of ports, plans, or parameter objects, written $\otimes$.

Native artifact. A domain-owned trial record retaining details that generic metrics cannot express.

Observation. A typed trace fact such as cache hit, provider call, fallback, rank contribution, cost, or warning.

Open dynamical system. A state-transition system with declared interfaces through which it can be composed with an environment. Campaigns and release controllers have this character.

Operation plugin. A plugin contributing one or more primitive generators and their interpretations.

Optic. A compositional abstraction for focusing and updating part of a larger structure; lenses are one optic kind.

Oriented delta. Candidate-minus-baseline change multiplied by metric direction.

Para. The construction whose arrows are parameterized systems $P \otimes A \rightarrow B$ and whose composition tensors parameter objects.

Pareto front. Eligible candidates not dominated across all selected oriented metrics.

Plan. Serializable intensional wiring built from primitive operations and fixed structural constructors.

Plan identity. Content identity of normalized plan syntax and captured operation descriptors.

Plugin conservativity. The property that registering new generators does not change meanings of old plans.

Policy. The explicit ordered decision object that defines eligibility and preference.

Port. An ordered list of schema wires forming the domain or codomain of a plan.

Primitive. A plan node referring to one registered generator descriptor.

Proposer. A component that selects legal interventions to evaluate from a space and, for adaptive methods, campaign history.

Reparameterization. A map from optimizer coordinates into a component’s implementation parameter object.

Refinement. Evidence that a concrete implementation realizes an abstract operation under equality or a declared observation relation.

Registry snapshot. Immutable set of plugin, schema, and operation descriptors under which plans and campaigns resolve.

Report. Paired comparison artifact for one candidate, retaining coverage, failures, metric distributions, and oriented changes.

Runner. The high-level port executing one complete domain-native trial.

Semantic class. Classification of an intervention as operational, approximation, relevance, knowledge, policy/security, interaction, presentation, or another domain-defined class.

Semantic key. Content identity of all behaviorally relevant inputs to an operation, used for sound artifact reuse.

Signature. The typed vocabulary of schemas and primitive generators available to plans.

Space. The typed baseline configuration, lawful intervention family, and application function for one optimization domain.

Structural map. Identity, permutation, copy, or drop node whose semantics is owned by the kernel.

Tensor. Parallel wiring of plans or pairing of parameter objects.

Trace equivalence. A declared relation over executions that may ignore operational interleaving while preserving protected outcomes, effects, artifacts, and lineage.

Trial coordinate. Stable case, repeat, arm, and candidate identity under which one terminal result is stored.

Typed openness. The ability to add new domain operations and schemas through plugins while retaining kernel-controlled composition and validation.

Universal property. The result that an interpretation of primitive generators extends uniquely to all free plans through a structure-preserving fold.

Workload. Immutable identified cases, inputs, strata, and labels used by a campaign.